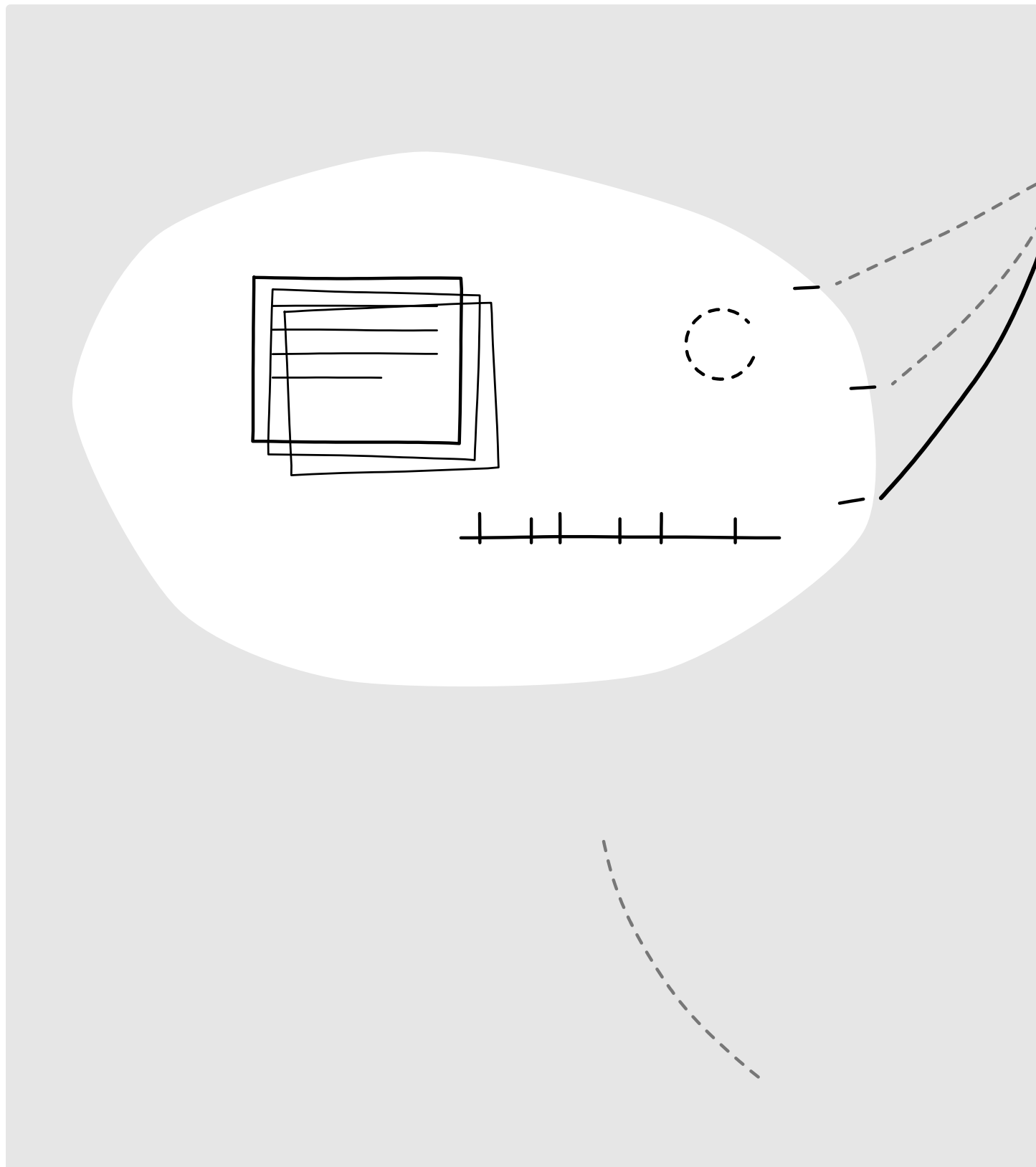


Schema Engineering：让工作存在于模型之外

从通用能力到可提交工作的弱编译、工作编纂与分层治理方法



摘要

概率模型已经能够搜索、比较、解释、起草和调用工具，却不能仅凭一次输出取得正式工作所需的事实效力、行动权限、完成状态和责任归属。一次 Run 会结束，模型会替换，上下文会压缩；Matter、Artifact、Review decision、未完义务与适用边界仍然必须继续存在。正式工作不能与一次生成同时出现或消失。

专业 Agent 的另一项瓶颈也不只是模型能力。行业头部专家可以借助 frontier Agent 亲手跑通高质量 Demo，但 Demo 往往依赖一层未被记录的 Human Harness：专家选择任务和材料、识别遗漏、调整搜索与上下文、修复失败，并判断什么可以交付。原作者离场后，这些能力常常随之消失。

Schema Engineering 是一种面向 Agent 的弱编译与工作编纂方法。它不是把工作简化成字段表，而是把专家在真实工作中演示出来的隐性义务、状态、权限、证据、完成标准与裁决边界编订为可版本化、可执行、可验证、可恢复的 Work Contract，并以同一套 Schema 治理 Sources、Events、State 与 Artifacts 之间的来源、版本和演化关系。Work Contract 据此约束模型与工具如何参与工作，规定候选结果如何通过 Evidence、Completion、Authority 与 Review 进入正式状态，并把生产中的 Revision 与 Failure 转化为 Eval、Environment 和后续系统改进的语义基础。

它不是某一种垂直 Agent 产品的宣言，而是 Agent 参与有后果工作时的编纂与承诺治理。Professional Work Runtime 是这套 Kernel 在高责任、高语义密度场景中的完整 profile；通用 Infra primitive、宿主可加载的 Work Extension、既有系统中的 embedded capability、sidecar、垂直产品与组织运行协议，都可以成为实现形态。产品负责把通用治理原语编译为具体人群认识的工作对象，组织负责提供最终 Authority、Accountability 与成果接受。

这一方法同时治理 Run 的两端。输入侧把 Stable Contract、Current Semantic State、相关资源与可检索历史投影为 task-specific Context；输出侧把 Model / Human Proposal 视为 Candidate Change，只有经过验证、授权和适用的 Review，才能写入 Committed Event 并更新 Current Semantic State。单次输出质量因此不等于跨时间工作的质量；只要 output 会成为后续工作的 input，运行之后“留下什么、失效什么、下一次优先看什么”就是系统能力的一部分。

在 Continuity Profile 中，输入侧还可以被压缩为一条运行管线：Store → Govern → Retrieve → Compile。Store 保存 Sources、Events、Artifact versions、Raw History 与正式状态；Govern 通过 Work Contract、commit protocol、status、version、Authority 与 applicability 使这些对象具有可选择的语义；Retrieve 返回潜在相关对象；Compile 才为当前 Assignment、role 与 task stage 生成最小充分的 Context Projection、Human Work Surface 与 Retrieval Index。这是既有 ontology 的运行投影。Matter repository 不是 Prompt，Work Contract 也不是一份更大的 system prompt；存在于用户工作空间中的信息，不等于必须存在于当前模型 Context。

这条管线同时解耦总工作容量与单次 Attention 成本。Matter、历史、机构知识和可用能力可以持续增长，一次 Run 激活的 working set 不应随总存量线性增长。通用 Agent 的含义因而不是 everything loaded，而是 anything governed is addressable：它能够进入不同工作空间，并可靠取得当前任务所需的最小充分 Projection。

对于高频、边界稳定的任务族，这种投影可以进一步发布为 **Compiled Work Expert**：一份经过版本化、E2E 验证和权限收敛的激活配置。Routine work 优先由机构 policy 或少量 Expert routing 选择这类配置；只有未覆盖、跨域或低置信度任务才退回受限的 primitive composition。Compiled Expert 是 Agent Extension 的发布投影，不是人格化 Agent，也不新增一套专业 ontology。

这一方法完整覆盖三条互相嵌套、不能混写的链：

- A. Expert Demonstration
 - Invisible Human Harness Extraction
 - Work Contract
 - Agent Extension
 - E2E Acceptance

- B. Stable Contract / Current Semantic State / Resources
 - Context Projection
 - Model / Human Proposal
 - Candidate State Change / Candidate Artifact Version
 - Validation / Evidence / Authority / Review
 - Committed Change
 - Updated Current Semantic State / Active Artifact

- C. Production Use / Review / Revision / Failure
 - Failure Attribution
 - Eval / Environment / Regression
 - Contract / Evaluator / Harness / Context / Tool Refinement
 - Selective Model Post-training, only when justified

第一条把 Expert Demonstration 编订为自带 Validator、Evaluator、Review boundary 和 E2E release suite 的 Agent Extension；第二条把 Current Semantic State 投影为有限 Context，再把概率性 Proposal 经 typed commitment interface 写为 Committed Change；第三条在 Extension 进入生产后，把失败先归因到 Contract、Evaluator、Context、Tool、Harness、Model 或机构分歧，再决定改进对象。模型提出并执行，确定性系统维护不变量，Evaluator 测量已明确规定的语义，人承担不可约的专业判断。

横向三条工作链之外，系统还具有一条不能混写的纵向分层：

Model Capability Layer

→ Agentic Runtime / Harness

→ Work Extension

→ Commitment Boundary / System of Record

→ Person / Organization / Downstream Work

上游扩展 reasoning、knowledge、tool use 与执行边界；下游把通用能力收敛为带有领域语义、HITL、Review 和 accepted-work-product 标准的工作渗透面。每层拥有自己的 Contract、Eval、长期资产和改进周期，不能借用相邻层的成功证明自身正确。

法律提供了高要求实例，但不构成外延边界。财务、医疗、咨询、研究、工程、合规、审计等工作同样需要来源、权限、版本、审阅和责任。一次性的高后果行动需要 commitment governance；跨时间、具有状态、又缺乏廉价稳定 verifier 的工作进一步需要 Matter、continuity 与 Context Projection。各领域可以共享 Runtime primitives，不能因此省略各自的专业语义。

第一部 弱编译与正式工作

Schema Engineering Kernel 同时处理两项不可约转换：专家亲自跑通的 practice 如何成为可转移的 Agent capability，以及概率性 proposal 如何取得可问责的 work commitment。Runtime 对象、Contracts、Agent Extension 和 E2E 都服务于这两项转换，不形成彼此竞争的 ontology。

1. 概率性提议与组织承诺

模型输出首先是一项提议。提议可以并列、撤回、试探和互相矛盾；组织承诺必须回答范围、依据、版本、责任与后果。语言再流畅，也不能自动把提议变成组织承诺。

一份合同审查意见可以写得完整，却仍然缺少原文坐标、适用版本、风险接受人和批准状态；一份研究报告可以列出大量来源，却没有说明哪些主张已经被采纳、哪些结论还需要复核；一个工具调用可以在技术上成功，却不一定取得业务权限。输出质量与工作效力属于不同对象。

模型也不能同时负责执行、定义事实来源、设定完成标准和作最终裁决。四项责任合一时，系统无法区分“模型认为已经完成”和“组织已经接受成果”，也无法在模型替换、任务恢复或责任追溯时重建正式状态。

P6 — 模型提出；确定性系统执行不变量；Evaluator 测量已经明确规定的语义；人裁决不可约的专业判断。

专业正确性通常没有一个廉价、完整的布尔判定器（Boolean oracle）。字段结构、权限、版本、引用坐标、算术和明确覆盖范围可以由系统检查；重要性、策略、风险接受、例外和责任仍需专业判断。混合裁决保留了这一区分。

工作的正式状态因此位于模型之外。模型可以进入和退出执行，组织承诺却不能随一次输出出现或消失。

2. Schema Engineering 与 Work Contract

Schema Engineering 是一种面向 **Agent** 的弱编译与工作编纂方法：它把专家在真实工作中演示出来的隐性义务、状态、权限、证据、完成标准与裁决边界编订为可版本化、可执行、可验证、可恢复的 **Work Contract**，以同一套 **Schema** 治理工作对象（在 **Continuity Profile** 中为 **Matter**）中 **Sources**、**Events**、**State** 与 **Artifacts** 的关系，并据此发布可通过端到端验收、能够在生产 **Review** 中继续改进的 **Agent Extension**。

它完成两项不同的转换：

Expert practice → transferable Agent capability
Probabilistic proposal → accountable work commitment

前者解决“专家亲自跑通的 Demo 如何成为其他人可用的能力”；后者解决“模型输出凭什么改变正式状态”。只写后者，Schema Engineering 会退化为 Work Commitment Runtime；只写前者，它会退化为专家 workflow 产品化。两项转换共同发生在一条持续的 Context / State 循环中：正式状态被投影给一次执行，一次执行的结果再经过窄化提交改变正式状态。

本文承接既有的工作与案件建模、来源治理、版本管理、提交控制和评测方法。Schema Engineering 的贡献在于把这些方法组织到 Agent 的工作生命周期中：从专家干预编订出可委派的 Contract，以它连接有限 Context 中的执行与模型之外的正式状态，再让失败反馈进入有边界的修订。单项状态机、来源模型或审批机制并非本文新创；需要检验的是这一组合能否改善委派、连续性与成果接受。

它的最小操作不是“给工作加上结构”，而是建立一条 typed commitment interface。模型和人都可以提出变化，但不能直接改写正式状态：

Stable Contract + Current Semantic State + selected Resources / History
→ Context Projection
→ Model / Human Proposal
→ Candidate State Change

- Candidate Artifact Version, when applicable
- Schema Validation
- Evidence Check
- Authority Check
- Completion / Review Routing
- Committed Event
- Atomic Projection: Current Semantic State + Active Artifact Reference
- New Context Projection

输入端把现实工作编排为本次执行可用的 Context；输出端把概率性结果编排为可提交、可拒收、可审阅的 Candidate Change。Runtime 不执行专家全部思维，只执行一项提议进入正式工作需要满足的条件。必要边界是候选、正式检查与可追溯提交；上图以事件记录和状态投影表达它，也可以由既有系统的事务、版本和审批记录承载。领域建模、workflow、结构化输出或 memory summarization 本身不建立这条边界。

2.1 Work Contract 的四重角色

Work Contract 是整套方法的枢纽，但不是完整的专业知识库、全部程序或 Boolean oracle。它同时承担四项逻辑责任：

1. **执行规格**：规定任务族、目标状态、输入资源与目的限制、交付物与受众、不变量、工具与 Authority、Evidence、Completion、Review、Escalation，以及中断和恢复所需保留的状态。它声明有效工作和有效转换必须满足什么，不要求冻结 Runtime 能自行选择的每一步执行路径。
2. **结果与提交校验器**：为 schema validation、deterministic validator、evidence check、completion ledger、authority check、evaluator rubric、review routing 与 accepted-work-product 判据提供语义来源。它决定哪些结果可以被机器拒收，哪些需要人裁决，哪些可以取得正式效力；它不保证专业结论必然正确。
3. **状态与 Context 编纂器**：规定哪些事实、决定、Artifact 与未完义务应当持久化，哪些信息具有时效或适用范围，哪些旧状态应被 supersede，哪些内容需要 Human confirmation，以及下一次 Run 优先披露什么。它同时治理 input 和 output，而不只帮助模型生成一次结果。
4. **Post-agentic Refinement 目标**：为失败归因、regression、Eval 与 Environment 提供稳定参照，使团队能区分“模型错了”与“产品没有定义清楚”，再决定应该修改 Contract、Validator、Evaluator、Context、Tool、Harness 还是模型权重。

Work Contract 不一定是一份单文件。它可以编译或投影为：

```
state schema
+ artifact schema
```

```
+ evidence requirements
+ completion obligations
+ authority policy
+ review / escalation policy
+ context and resource policy
+ persistence / expiry / supersession policy
+ validators
+ evaluators
+ E2E and Work Eval cases
+ version / compatibility metadata
```

这些对象可以在同一系统中物理合并；逻辑责任必须清楚，物理服务不必拆散。

2.2 为什么是“弱编译”

编译不表示完整形式验证，也不要求把专家知识全部写成规则。它是弱编译器，因为 source 主要来自不完整的 demonstration、revision、accept / reject 与隐形人工补位；专业正确性通常没有廉价且完整的 Boolean oracle；target 是确定性系统、Evaluator、Agent 与人共同组成的混合 Runtime；部分语义只能表示为 Review、Escalation 或 Accountable Decision，不能被自动裁决。

因此，弱编译器只把可靠委派需要的边界变成显式对象，显式化“判断的后果与工作边界”，不显式化“判断的全部心理过程”：

- 正在处理什么事项；
- 谁承担运行义务，谁承担最终问责；
- 可以读取、修改、批准和发布什么；
- 哪些主张需要来源；
- 当前已经知道、决定和遗漏什么；
- 什么条件才算完成；
- 哪些判断可以由系统裁决；
- 哪些判断必须由人承担；
- 修改、批准和升级如何改变正式成果；
- 哪些结果可以进入长期状态，哪些只属于本次执行。

完整编码专业知识既昂贵也脆弱。专家不一定能完整陈述判断过程；可以陈述的规则也可能只适用于某一机构、客户、时点或个人习惯。Schema 不追求完整描述整个领域，只保留可委派、可验证、可恢复、可审阅和可继续工作所需的最小显式语义。

Schema 也不是隐性知识本身，更不是对专家 cognition 的序列化。它是隐性专业判断显影后的中间表示：只抽取那些必须跨时间继续存在、必须改变后续行动、必须支持 Review，或者必须能够验证、撤销和追责的工作语义。其余判断仍可以留在人、模型推理、Source、Context 与情境化 Review 中。

2.3 Schema 作为工作纲要与目录

Schema 规定工作对象如何被识别、关联和区分状态，以及哪些变化在什么条件下具有意义。它使不同人员和执行实例能够围绕同一对象、版本与来源比较判断，而不必先重建彼此的对话。类型、状态机、权限规则、来源关系与审阅条件共同构成这一中间表示；JSON Schema 只是其中的数据形状约束。

在这里，“纲要体”不是把复杂工作粗暴摘要成几句话，而是在有限篇幅和有限 attention 中压缩现实，同时保留足以恢复原有关系的结构。一个合格的 Matter state 不只回答“发生了什么”，还应当能够回答：

- 这个判断来自哪份材料、哪次沟通和哪个版本；
- 哪些事实已经确认，哪些仍是推断或 working assumption；
- 哪个结论覆盖了哪个旧结论，何时开始生效；
- 哪些来源彼此冲突，哪些关系只是限定而非支持；
- 当前状态为什么演化到这里；
- 必要时怎样返回原文、旧版本与执行 trace。

这接近目录学所强调的“辨章学术、考镜源流”：编目不是把材料分进抽屉，而是借由出处、版本、关系、沿革和适用范围，保存知识的内部秩序。这里借用的不是古典术语的装饰，而是一项数据治理要求：来源、版本、状态、冲突与撤销一旦在压缩中丢失，留下来的自然语言即使语义相似，也不再具有专业工作的可靠性。

Schema 不是把现实塞进数据库，而是给现实留下可追溯、可更新、可继续工作的骨架。

因此，目录学级别的数据治理本身就是工作智能的一部分。自动摘要可以充任索引或临时投影，不能取代 authoritative state；未经校验的模型输出也不能因为“写得完整”就成为下一轮的事实。

Eval 会从这套编纂关系中自然生长出来。只要系统已经明确记录：

```
state → source → judgment → transition → artifact
```

Eval 就可以检查来源是否正确且仍然有效、判断是否越过前置条件、冲突是否被披露、状态转换是否符合 Contract、Artifact 是否能够沿 provenance 回溯。它不再是后来外加的一组问答，而是对工作目

录、契约与成果的校勘机制。Schema Engineering 先解决“如何记述和继续一项工作”，再获得“如何检查它是否按已声明的标准成立”。

2.4 哪些语义值得显式化

Schema 显式化判断的后果，不显式化判断的完整过程。一个语义对象值得进入 Schema，通常需要满足以下大部分条件：

1. 它会改变正式状态，例如接受、驳回、批准、发布、完成或升级。
2. 它必须跨时间、模型或人员继续存在。
3. Operator、Reviewer 与 Accountable Principal 需要对它形成共享理解。
4. 遗漏或误解会造成错误提交、重复工作、责任不清、版本漂移或高昂 Review。
5. 它足够稳定，可以用可接受的成本命名和维护。
6. 显式化会改变 validation、routing、Authority、Completion、Review 或 Context Projection，而不只是增加字段。
7. 它会改变下一次执行应当优先看到什么，或者决定本次 output 能否进入长期状态。
8. 它能够以低于预期治理收益的成本被捕获、解释和维护，不要求专家按同等比例增加表单填写与标注劳动。

低风险且可撤销的判断、只在单次推理中使用的信息、高度依赖整体语境的专业直觉、重新推断成本很低的内容，以及不影响正式状态的描述，可以继续留在 source、context 或 Reviewer judgment 中。显式化如果持续产生大量例外或错误确信，也应退回较弱的表示。

每个字段和对象都需要通过一个删除测试：移除后，恢复、权限、审计、完成、Review、Context Projection 或成果采用是否明显变差？如果长期没有可测影响，它就不应继续占据 Schema。P13 和 F12 因此也是日常产品纪律，不只在模型升级时使用。

删除测试必须连同 Capture Economics 一起执行。一个语义对象即使理论上有价值，如果只能由最昂贵的专家持续手工填报、维护成本接近或超过它节省的 Review、恢复和错误成本，也不应成为默认 Schema。优先路径应当是：

```
normal work behavior
→ implicit event / before-after capture
→ uncertain semantic inference
→ selective clarification, only when ambiguity matters
→ Candidate rule or state change
→ authorized promotion
```

而不是让专家为了未来 AI 学习，把日常工作改造成高频数据标注。

泛化到一个新场景时，不从“能结构化什么”开始，而先回答四个最低问题：

What persists?	运行结束后，哪些事实、决定和 Artifact 应当留下？
What expires?	哪些信息具有时效、版本或适用范围？
What gates action?	什么状态必须确认后才能继续或提交？
What deserves attention next?	下一次执行应优先披露什么，而不是重放全部历史？

如果一项 Schema 声明不能改变这四个问题中的任何一个，也不能改善 validation、Review、recovery 或 accountability，它很可能只是形式主义。

第二部 Work Commitment and Continuity Runtime

本部包含两个嵌套但不等同的范围。**Commitment Kernel** 适用于任何可能改变持久共享状态、触发外部行动或产生现实后果的概率性 Output，即使它只发生一次；**Continuity Kernel** 进一步适用于跨 Run、跨人员或跨时间继续存在的工作。Matter、Event Ledger、Semantic State 与 Context Projection 主要服务后者；Candidate、Authority、Review 与 Committed Change 则同时服务两者。Professional Work Runtime 是二者重叠时的高保证 profile，不构成全部外延。

3. Matter，而不是 Session

一次 Run 会结束，模型会替换，上下文会压缩，用户会隔日返回。需要持续的工作仍然必须保留材料、事实、版本、未决问题、权限和审阅结果。如果工作只存在于 transcript，连续性就会依赖对历史对话的反复压缩、对旧模型隐含推理的猜测、对一次性 prompt 的复现，以及用户记得“上次说到哪里”。

在 Continuity Profile 中，Professional Work 产品的首要用户对象是 **Matter**：具有可识别责任边界、状态连续性和成果或决定历史的最小工作承诺单元。Matter 可以是一宗案件、一项交易、一次审计、一个研究任务、一个客户项目，也可以是持续运营中产生的一项需要独立处理和追踪的工作。

Session 是对一项 Matter 的临时交互与 attention window；Run 是其中一次可停止、失败或替换的执行尝试；Matter 才是工作本身。工作必须能够在不复现完整 transcript 的情况下由新的 Session、模型或 executor 从 governed state 恢复。

P1 — Matter over Session: Session 是基础设施对象，Matter 才是用户对象。

Matter 持有 identity、resources、participants、permissions、Assignments、Artifacts、Semantic State、evidence、Events、Reviews 与 outcomes。Conversation 仍然存在，但只是一种交互表面。

P2 — Conversation is an interaction surface, not the product ontology: Conversation 是交互表面，不是工作本体。

经过治理的 Matter 可以进一步被理解为一种 **semantic repository**。它具有稳定 identity、权威当前状态、版本化 Artifact、候选变更、Review、commit 与可恢复历史；必要时，互相竞争的假设、方案或机构配置可以在独立 branch / fork 上演化，只有经过适用的 Evidence、Authority 与 Review 才进入当前权威状态。这里借用的是 repository 的治理层级，而不是要求专业用户直接操作 Git，也不是把所有工作细节都文件化。

这种 repository 的权威语义层通常远小于它所引用的原始材料和完整执行历史，语义密度却更高。它不以 boilerplate、执行日志和全部 transcript 充当当前状态，而主要保存仍然有效的 finding、evidence relation、decision、open issue、active Artifact version 与未完义务。Fork 的合并也不是文本层面的 merge，而是对哪个判断、方案或版本取得正式效力的业务裁决。

Topic-level memory 位于另一层。它可以按主题保存偏好、项目事实、持续关注点和近期变化，减少跨 Session 的重复说明；但 Topic 回答的是“这些信息关于什么”，不必回答 objective、Accountable Principal、Artifact、Authority、Completion、Review、正式版本与提交历史。因而，从 transcript summary 转向可编辑、可持续更新的 topics，是从 chronology 向 semantic persistence 的推进，却不自动构成 Matter management。

Topic organizes remembered information; Matter organizes accountable work: Topic 组织记忆内容，Matter 组织可问责的持续工作。

用户返回一项 Matter，不是寻找某个自动生成标题的 Session；用户审阅一版 Artifact，不是从消息流中猜测哪段回答已经生效；用户恢复工作时读取当前状态，不必重放完整聊天。

Portfolio、Queue、Program 或 Practice 位于 Matter 之上，用来组织多个 Matter。持续监控、周期性合规、长期账户管理和工单队列通常由这些上层对象管理，并在出现独立责任、状态或成果时创建 Matter。把整项长期职责塞进一个 Matter，会使 Matter 退化为无边界 workspace；把每条微小事件都建成 Matter，又会造成过度建模。

普通聊天、角色陪伴和开放探索可以共享持久化 Runtime primitives，却不必共享专业工作的 Evidence、Completion、Authority、Operational Responsibility 和 Accountability 语义。Matter-first 是专业工作对象的选择，不是把所有长期对话改名为 Work。

4. Work Commitment and Continuity Runtime

Runtime objects 表示逻辑责任、状态和权威边界。下文采用事件溯源（event sourcing）作为参考实现：Committed Event 保存生效变化，reducer 从事件生成 Current Semantic State。既有系统也可以用事务状态表、版本化成果和可审计的提交记录实现同一语义；它需要保持唯一 canonical owner、验证后的写入与可恢复状态，不必从事件日志重建全部历史。一次性 consequential action 可以只实现 Candidate → Authority / Review → Committed Change；当工作需要继续存在时，再引入 Matter 与 continuity objects。

这里需要区分两种“最小”：

- **Semantic Minimum** 是完整描述高保证持续工作所需的逻辑闭环，也就是本节的 Continuity Runtime Kernel；
- **Deployable Minimum** 是让用户第一次感受到工作不再只存在于 Chat 的最小产品切片，通常只实现一条真实的 commitment boundary。

Semantic Minimum 定义扩展方向，不是产品采用的前置建设清单。Deployable Minimum 可以把多个逻辑对象压在一套现有系统或单体实现中。完整 Kernel 由以下对象构成：

```
Matter
├─ Resources / Context
├─ Assignment / Work Contract
├─ Operator
│   └─ Lane
│       └─ Run
│       └─ Delegated Operator
├─ Accountable Principal
├─ Event Ledger
├─ Semantic State
│   └─ Evidence Graph
├─ Artifacts
└─ Review
    └─ Revision / Outcome
```

4.1 Matter

Matter 是最小工作承诺单元，定义责任边界、资源、参与者、权限、当前事实、正式成果和决定历史。它跨模型、跨 Conversation、跨人员继续存在，可以容纳多个 Assignment 和同一 Artifact 的多个版本；Portfolio、Queue、Program 和 Practice 只负责组织多个 Matter。

4.2 Assignment 与 Work Contract

Assignment 是 Matter 内一项有边界的委派，不是自然语言 prompt 的别名。它至少说明 objective、deliverable、audience、constraints、deadline、authority、completion requirements 与 Review requirements。

Assignment 的状态可以包括 not started、in progress、blocked、awaiting review、completed、superseded 与 cancelled。状态由 Runtime 维护，不能只存在于 Agent 的自然语言自述。

Work Contract 是约束 Assignment 并支撑同一任务族复用的契约集合。它在此处作为执行规格和提交校验的语义来源，并在生产后成为失败归因与改进的参照。Matter 是工作对象，Assignment 是一次有边界的委派，Work Contract 是可版本化的规范；三者不需要合并成新的上位名词。

4.3 Operator 与 Accountable Principal

Operator 对某一范围承担稳定的 **Operational Responsibility**，可以是人、主 Agent 或被委派的 Agent。这种责任只表示谁必须执行、恢复、交付或升级工作，不表示谁最终承担专业、组织或法律后果。

Authority 说明谁被允许读取、提出、修改、批准、发布或委派。**Accountability** 说明谁最终接受专业判断和组织后果。Accountable Principal 必须是能够承担这项问责的人或组织角色；Agent 可以取得部分 Operational Responsibility 和 Authority，不能自行取得不可约的 Accountability。

新的 Operator 只有在 Operational Responsibility、scope、tools、Authority、completion obligation 与 escalation path 相对独立时才成立。每个 Assignment 需要明确 Operator、授权范围和 Accountable Principal，三个字段不能互相代替。

并行调用、检索线程或后台任务不自动形成 Multi-agent。运行义务和最终问责都不能从计算拓扑反推。

4.4 Lane

Lane 是同一 Operator 内的一条独立工作线或 execution cursor。不同 Lane 可以并行检查问题、搜集材料或起草不同部分，同时共享 Operational Responsibility 与 Authority。

P9 — Lane is parallelism; Operator carries operational responsibility; the Accountable Principal carries ultimate accountability: Lane 表示并行，Operator 承担运行义务，Accountable Principal 承担最终问责。

Lane 在取得独立 Operational Responsibility、Authority、完成义务和升级路径后，才被提升为新的 Operator。

4.5 Run

Run 是某条 Lane 的一次 activation。它接收输入、调用工具、读取或更新状态、创建或修改 Artifact、产生 Event、请求 Review，并可能停止、失败或恢复。

Run 只承载一次暂时执行，其 model invocation、sandbox、tools 与 working Context 都可以被替换。它的结束不证明 Assignment 已经完成，Assignment 完成也不等于 Matter 结束；Run 的 disposable state 也不能成为恢复 Matter 的唯一来源。

4.6 Event

Event 记录“What happened”，但候选变化和已经生效的变化必须分开：

- **Candidate Event** 记录待提交的状态变化，没有正式效力；
- **Committed Event** 已经通过 Schema Validation、Evidence Check、Authority Check 和适用的 Review Policy，可以改变 Current Semantic State。

模型和人都可以提出 Candidate Event，只有 Runtime 可以按提交协议写入 Committed Event Ledger。在适用 retention policy 内，Committed Event 不被静默改写；纠正由新的 Committed Event 表达，删除、匿名化和归档由明确的权利与保留政策处理。

典型 Event 包括 assignment accepted、resource accessed、tool called、artifact proposed、review requested、revision committed、approval granted 与 failure occurred。Committed Event Ledger 是正式状态转换的权威记录；执行 trace 和未提交候选可以另行保存，但不能混入正式账本。

4.7 Artifact

Artifact 是可以进入真实工作流的正式成果，具有 identity、type、version、provenance、status 和下游用途。文档、表格、redline、workpaper、finding、decision、plan 与结构化 issue set 均可成为 Artifact。Artifact 内容保存在版本化 Artifact Store 中；Committed Event 引用不可变的 Artifact version，Current Semantic State 只保存当前权威版本的引用和状态。

Candidate Artifact Version 可以在提交前暂存，没有正式效力。只有引用该版本的 Committed Event 写入账本后，它才成为正式 Artifact。Chat 回答也只有经过这条路径，或形成其他已经提交的状态变化，才进入正式工作。

4.8 Semantic State

Semantic State 表示“What is happening now”。每项 claim、finding 或 decision 至少携带两个互相独立的状态轴：

状态轴	可用状态	回答的问题
Epistemic Status	proposed、supported、contradicted、uncertain、working assumption、resolved	证据和认识处于什么状态
Institutional Status	draft、under review、approved、published、superseded、withdrawn	组织赋予了什么效力

证据充分不等于已经批准；已经作为业务决定接受，也不等于它被证明为客观事实。Current Semantic State 还保存 open issues、Evidence relations 与 gaps、coverage、completion status、pending decisions、active Artifact references 和 Review state。

P3 — Transcript is evidence of execution, not canonical work state: Transcript 是执行证据，不是权威工作状态。

在事件溯源实现中，Committed Event Ledger 是状态转换的权威记录，Current Semantic State 是这些事件的确定性投影，可以 checkpoint 并从账本与引用的 Artifact versions 重建。由既有 system of record 持有状态时，恢复以其正式版本和提交记录为准。Artifact Store 保存成果内容，Semantic State 保存权威版本引用；引用与实际版本不一致时，Runtime 阻止继续提交并进入修复流程。

4.9 Review

Review 是工作生命周期中的裁决动作，包括 accept、reject、revise、request further work、escalate、approve 与 promote policy。Reviewer 先提出 Candidate Decision；Runtime 检查 Authority 与 Review Policy 后写入 Committed Event，再由 reducer 更新 Current Semantic State 和 active Artifact reference。Review 不直接改写 State。

专业系统不以“一路点击 Yes”为目标。Review 只应出现在具有专业意义、权限意义或学习治理意义的节点，并且必须改变正式状态，而不只是留下评论。

4.10 生命周期

这些对象共同形成一条可恢复的工作链：

Matter created

→ Resources, Accountable Principal and permissions established

→ Assignment accepted

→ Operator and Work Contract selected

→ one or more Lanes execute through Runs

→ Candidate State Changes and Candidate Artifact Versions proposed

→ Schema, Evidence and Authority checks executed

→ Completion / Review Policy routes the candidate

- Committed Event appended
 - Current Semantic State and active Artifact reference projected atomically
 - Outcome and reusable signals recorded

这条链不要求线性推进。Review 可以把成果退回执行，Assignment 可以拆给新的 Operator，新证据可以重开已经关闭的问题，新 Artifact 也可以取代旧版本。每次回转都通过 Committed Event 改变 Current Semantic State 和 active Artifact reference；历史记录与当前状态因此保持一致。

5. 历史、状态、上下文与输出不能混为一体

持久工作需要分别回答四个问题：

问题	承载对象	作用
发生过什么？	Committed Event Ledger 与 Raw History	正式状态转换、执行追溯
现在是什么？	Current Semantic State	当前事实、决定、覆盖、未决义务
下一次执行应看见什么？	Context Projection	面向特定 Operator、Lane、Run 的临时视图
这次运行产生的什么可以留下？	Candidate Change + Commit Protocol	把 output 区分为临时结果与正式状态更新

Transcript 不能同时承担四项职责。历史越长，完整注入的成本越高；反复 summary 又会把暂时解释写成事实，并在多轮压缩中积累漂移。未经治理的 output 如果直接进入 memory，还会把一次推测、旧版本或未通过 Review 的结果带入后续执行。

对于能够把未来决策所需信息表达为当前语义状态的长程工作，规范执行基底应是 Current Semantic State，而不是累积 Transcript。History 仍属于 Evidence 与溯源层：它在需要时被检索和披露，不默认重放为执行上下文。

连续性由三层持久内容和一条提交路径共同组成：

5.1 Stable Contract

Stable Contract 保存长期稳定的 role、institution policy、Assignment semantics、professional schema、permissions 与 tool authority。

5.2 Current Semantic State

Current Semantic State 保存双轴 claim status、开放问题、Evidence Relations、coverage、active Artifact reference、completion、decision 与 pending Review；它由指定的 system of record 维护，在

事件溯源实现中为 Committed Events 的当前投影。

5.3 Raw History / Evidence

Raw History / Evidence 保存完整 transcript、原始材料、工具轨迹、Candidate Events、执行 trace 与旧版本成果。它们默认可检索，不默认全量注入，也不能直接改变 Current Semantic State。

P4 — Continuity comes from stable contracts, current Semantic State and retrievable history: 连续性来自稳定契约、当前语义状态与可检索历史，不依赖不断压缩 chronology。

按工作域隔离持久内容、保留可检索历史和按需生成工作集，只解决 recall boundary 与 attention allocation；它们不自动建立 authoritative Semantic State、Evidence status、Authority、supersession 或 commitment governance。

5.4 Store → Govern → Retrieve → Compile

Context 是面向当前 Assignment 的临时工作视图。它从持久内容中选择本次执行有权使用、与当前义务相关的对象，并保留判断所需的状态、来源和限定。Continuity Runtime 的输入侧由四项责任组成：

```
Store
→ preserve Sources / Events / Artifacts / Raw History / Current Semantic State

Govern
→ assign identity / status / version / provenance / authority / scope / expiry

Retrieve
→ locate relevant objects within the authorized scope

Compile
→ represent the working set for this recipient, task and stage
```

Govern 确定对象的效力与使用边界，Retrieve 定位候选材料，Compile 将其组织为可用于当前工作的表示。相关性不能授予访问权限；对象可读，也不表示其中的主张已获支持或已经生效。披露由接收者、Assignment、用途和当前规则共同约束，规则在数据交给相应接收者之前实施。

Context Projection、Human Work Surface 与 Retrieval Index 分别服务执行、裁决和未来定位。它们可以采用不同颗粒度，但同一对象的身份、版本、来源与效力必须一致。执行者可以只看到与当前问题相关的事实和 Evidence，Reviewer 则需要候选差异、未解冲突与决定后果。投影省略的内容不因此成

为否定事实；影响判断的未知、限定和分歧应在视图中保留，材料不足时通过授权路径补充，或将问题交回裁决。

Govern layer 的每项规则需要 owner、scope、version、review path、disagreement representation、deprecation 与 rollback。若这些规则只能由少数专家永久手工维护，或其结构化成本与使用量同比增长，Runtime 只是把隐形 Human Harness 改写成配置劳动。

投影随任务、权限和状态变化而重建。Stable Contract、Current Semantic State、选定的 Resources / History 与当前 Assignment 构成其输入，正式状态与受 retention policy 保护的 Evidence 保留在投影之外。Recovery 因而是恢复有效状态并重新生成工作视图；已经失效的权限或旧视图不能仅因执行续行而重新生效。

删除、截断、摘要、压缩、折叠与重载属于 **Context Mutation**，改变本次 Run 能注意、比较和引用的内容。其作用限于工作视图；需要长期保留的事实与决定仍走正式提交路径，Raw Evidence 仍按 retention policy 保存。影响恢复、来源完整性或重要判断的 mutation 应保留来源、作用域、变更理由和恢复路径。

5.5 Run 的双向治理

一次 Run 不是从“全部记忆”直接生成“新的记忆”，而是位于两次治理之间：

```
Stable Contract + Current Semantic State + selected Resources / History
→ Task-specific Context Projection
→ Model Attention / Tool Execution
→ Candidate Output
→ Schema / Evidence / Authority / Completion / Review
→ Committed Event + Committed Artifact Version
→ New Current Semantic State
→ Next Context Projection
```

输入侧决定哪些材料以什么状态进入 attention；输出侧决定哪些结果取得正式效力。未经接受的 finding、plan 与候选成果可以为继续执行和 Review 被保存、检索并进入授权视图，但必须保留候选身份、来源和适用范围。只有通过相应 Contract 的变化，才能更新 authoritative state 或 active Artifact。可见性服务于工作，提交决定效力。

提交边界限制错误取得正式效力，来源与认识状态则约束它在后续工作中如何被使用。候选中的错误仍可能随共享和检索传播，因此后续视图必须保留其未获支持、存在分歧或尚待审阅的状态；正式记录一项 working assumption，也不使它成为已证实事实。发现偏差后，系统应定位受影响的候选、状态和

成果，修订其效力，再从经核对的状态继续工作。长程可靠性由这条发现、限制、纠正与恢复的链路支撑。

P18 — Context is a projection; output is a candidate state update: Context 是针对当前工作生成的投影；Output 只是候选状态更新，不能默认沉淀为 Memory。

这一区分使系统可以保留丰富的探索，而不让 brainstorming、已撤回判断、失败尝试和未审阅结果污染当前状态。它也说明 one-shot quality 与 longitudinal work quality 属于不同指标：一次生成可以很好，跨时间工作仍可能因状态未更新、版本覆盖错误、来源失效或错误沉淀而失败。

5.6 Attention economics

更大的 context window 会降低一部分检索和压缩压力，但不会取消信息治理。至少有三个约束继续存在：

- 1. 可见信息不等于有效信息。模型“看得到”全部历史，不表示它会把当前有效、最承重的信息赋予正确权重。
- 2. Semantic similarity 不等于 epistemic status。已确认事实、旧结论、用户猜测、已撤回决定、权威来源和 brainstorming 可能语义相近，却不能被平等使用。
- 3. Output 会成为下一轮 Context 的上游。未经验证的结果如果直接沉淀，会累积为错误状态、错误摘要和错误检索索引。

没有结构化治理时，系统往往反复支付同一笔成本：

全部历史

- 塞入 Context
- 模型重新辨别有效与失效状态
- 再摘要或压缩
- 丢弃大部分
- 下一次重新开始

Schema Engineering 把它改写为：

Raw History / Evidence

- 一次编订与提交
- Current Semantic State
- task-specific Context Projection

前者反复支付 token、latency、retrieval、context construction、状态重判和 attention dilution；后者把已经解决过的状态判断保存下来，只在当前任务需要时渐进披露。更多 Context 因而不能自动等价于更好的 Context。上下文窗口扩张类似仓库扩容，不会使目录、版本和权威关系失去价值。

这种 semantic repository 对 attention economics 的关键作用，不是再造一份精简 transcript，而是把体量庞大的材料与 trace 同体量较小、语义密度较高的权威状态分开。同样的 Context budget 不再主要装入需要重新辨别有效性的 chronology，而是装入已经过提交的事实、判断、关系、版本、适用边界和未完义务。

Context Projection 也不是对整座 repository 重新摘要，而是从权威状态编译出的本次执行视图。相同模型与 Context budget 下，这种编订可以承载更长的 Matter horizon 和更高的语义复杂度；它拓展的是由工作基础设施支撑的有效能力边界，不等于模型参数本身已经获得同等提升。

Schema Engineering 控制的是 Reality、Sources、State、Contract、Projection 与 Output 之间的转换。它不试图比模型更聪明，而是在有限 attention 下，把模型能力配置给正确的现实。

Memory 也不再是一个不断增长的“模型记忆罐”。需要持久保存的是专业对象、正式状态、来源关系与检索权；模型每次只获得与当前执行有关的投影。

这可以进一步表述为工作知识层的 **sparse activation**：

```
Total governed work knowledge and capability
→ Matter / Contract / Catalog boundary
→ retrieve potentially relevant objects and packs
→ compile a minimal sufficient working set
→ Model Attention / Tool Execution / Human Review
```

目标不是把总存量压缩为一份永久摘要，而是使总容量与单次激活成本解耦。随着 Matter、Artifact、历史和可用 capability 增长，per-run Context 应主要随当前 Assignment 的 working set 变化，而不是随整个 repository 线性增长。Compile layer 同时控制两类错误：under-inclusion 会遗漏有约束力的事实、active version 或 Authority；over-inclusion 会引入已 superseded、越权或无关对象，增加 token、attention dilution、状态重判与 stale-context 风险。

这一结构可以与模型内部 Mixture-of-Experts 作有限类比：MoE 稀疏化模型容量上的计算，Matter Runtime 稀疏化工作知识和 capability 上的 Attention。二者都尝试把总容量与单次成本解耦；但模型 expert 通常是同构参数模块并由 learned router 激活，Matter、Work Contract、Evidence、Authority 与 Tool 则是异构、状态化、带来源和现实后果的对象。这里的 routing 更接近语义编译、least-

privilege composition 与 governance gate，不能由 MoE 的 top-k gating 直接替代。类比说明 scaling principle，不构成实现或正确性的证据。

6. Contract Discipline

专业工作由一组互补契约约束。单一 schema 无法同时表达来源、权限、完成、成果和裁决。

6.1 授权范围内广泛探索，正式边界上窄化提交

Assignment 分别约束可以读取的资源、允许提出的变化，以及能够提交或向外传递的结果。读取权不授予提交权，合法读取与合法调用工具也不能自动组合成向任意接收者披露信息的权限。目的与接收范围因此约束 Context、工具参数和交付物中的信息传播，外部传递须在发生前满足相应授权条件。

P5 — Explore broadly within authorized scope; commit narrowly across the formal boundary: 在授权资源范围内广泛探索；只有通过带有明确类型、证据、权限和问责主体的窄化边界，才能改变正式状态。

模型可以在此范围内搜索、比较、试探和提出替代方案。候选变化须绑定目标对象、依据版本与前置条件，再由 Evidence、Authority、Completion 与 Review 决定能否取得正式效力。探索可以保持开放，提交则必须具体到对象及其后果。

6.2 Evidence Contract

需要事实依据的正式主张必须关联机器可读的 Evidence Relation。来源坐标只说明材料在哪里，不能说明材料支持、反驳还是限定某项主张。最小关系包括：

```
claim
source
source_version
location
relation: supports / contradicts / qualifies / derives-from
extraction_actor
derivation_method
effective_time
status
```

模型可以提出引文、主张和候选关系；Runtime 负责解析来源、版本、坐标和关系类型。可解析引用只确认定位，语义上是否支持主张，由 Evidence Contract 指定的可验证规则、Evaluator 或 Reviewer

判断；未通过该判断的关系仍是候选。垂直 Contract 定义 source authority、precedent hierarchy、freshness 与 admissibility，通用 Runtime 不给不同领域的来源设定统一权重。

P7 — Provenance is state, not prose: 来源关系属于工作状态，不能只写在说明文字里。

无来源关系的事实主张不能进入要求事实承诺的正式状态。可用来源不只包括文档和数据库，也包括用户或专业人士的明确陈述、系统观察和工具返回、可以复算的推导结果。没有足够依据的主张可以保留为 proposed、uncertain 或 working assumption，但不能伪装成已经确认的事实。

Evidence Contract 必须区分事实、推论、偏好、策略与人类裁决。一个来源可以支持事实前提，却不能替 Reviewer 承担风险接受或策略选择。

6.3 Completion Contract

Run 可能因 token、工具失败、不确定性或局部成功而停止，这些都不构成专业完成。Assignment 只有在 Completion Contract 满足，或授权人明确豁免、修改该 Contract 后，才进入 completed state。

Completion ledger 可以包括：

- required Artifacts；
- required issue families 与 coverage；
- unresolved questions；
- evidence gaps；
- mandatory checks；
- dependency state；
- pending external input；
- required approvals；
- known exceptions。

P8 — Completion must exist outside the model's self-assessment: 完成条件必须存在于模型自评之外。

可执行的 completion inventory、evidence procedure 与 continue / stop gate 可以改变长程执行的结果；它们仍只能证明被表达的完成条件得到执行，不证明一般专业正确性，也不取代 Evidence、Authority、Matter continuity 或 accepted-work-product 的独立检验。

系统应使“不完整”可见，但不能假装所有完成条件都能机械裁决。机器可验证义务和人类裁决点需要分别表示。

6.4 Authority Contract

Tool capability 与业务 authority 不是同一个对象。模型可能技术上能够创建文件，却没有权力发布；可以生成邮件，却没有权力发送；可以提出修改，却不能改变 approved state。

Runtime 至少应区分 read、propose、modify draft、modify approved state、approve、publish、transmit externally、execute irreversible action、delegate 与 promote policy。取得 Authority 不等于取得 Accountability；自动化可以准备材料、提出建议和执行可撤销步骤，不能替 Accountable Principal 作出最终问责决定。

用户看到的是业务语义：

write_file	→ 接受这项成果修订
run_external_tool	→ 把这份工作成果发送给客户
resume_session	→ 返回当前 Matter
create_subagent	→ 委派一项边界明确的运行义务

技术操作不应自动成为用户心智对象。

6.5 Artifact Contract

Artifact Contract 定义成果类型、结构、版本、状态、引用关系、审批条件和下游可用性。Committed Event 必须引用具体 Artifact version，Current Semantic State 只能把已经提交的版本设为 active。产品价值以可采用的 work product 计量，不以聊天回答的长度或流畅度计量。

6.6 Review Contract

Review Contract 定义谁在何种状态下可以接受、驳回、修订、要求补充、批准和提升规则。有效 Review 还要求具备 Authority 的人能在有限 attention 下形成独立判断。Human Work Surface 因此按 decision unit 编译当前状态、Candidate delta、Evidence、自动检查、未决问题、可逆性、Authority requirement 与 state consequence；Raw Trace 保持可检索，而不成为默认审阅对象。Review 产生 Candidate Decision，只有通过 Authority Check 并写入 Committed Event 后，才改变 Semantic State 或 active Artifact reference。

6.7 Escalation Contract

Escalation 是 Runtime 中的正式动作，不是模型失败后的兜底。合格 escalation 至少表达：

```
issue
+ evidence
+ alternatives
+ recommendation
+ reason_for_escalation
```

系统不迫使模型在信息不足或权限越界时给出答案，而是把不确定性送到具备相应 Authority 或 Accountability 的主体。

第三部 从专家演示到 Agent Extension

7. Expert Demonstration 与隐形 Human Harness

专家 Demo 不是产品能力，却是比静态 SOP 更接近真实工作的可运行 source material。专家不必先写出完整 SOP、Schema 或 rubric；Schema Engineering 团队可以从以下材料抽取可转移的工作语义：

- 真实或合成 Matter 上的完整示范；
- prompt、context、tool 和 source 的选择；
- 反复重试、失败恢复和执行路径变化；
- before / after revision；
- accept、reject、request further work 与 escalation；
- 对 materiality、coverage 与 deliverability 的判断；
- 哪些结果进入下游流程，哪些被推翻。

因此，编订不是简单地“采访专家写需求”，而是：

```
观察 → 抽取 → 分层 → 编订 → 反例验证
```

专家做出的模型 prototype 往往由一层未被记录的人工 Runtime 支撑：

```
Expert selects task and context
→ Model attempts
→ Expert notices omission or error
→ Expert changes framing, source or search
→ Model retries
```


- Expert judges materiality

→ Expert decides what is deliverable

Prototype 制作者亲自操作时，这些干预容易被误认为模型能力；prototype 交给其他人后，任务选择、上下文拼装、遗漏识别、失败恢复和完成判断同时消失。产品化需要外部化足以支持委派、验证、恢复和审阅的干预，不需要复制专家全部思维。

编订时必须按责任、适用范围与稳定性分层：

层级	例子	编译目标
Domain invariant	稳定的专业义务、来源层级、必查事项	Product Contract / Evaluator
Institution convention	团队审批、格式、风险偏好	Versioned institution configuration
Client / Matter context	特定客户、交易、时点要求	Client / Matter context
Individual habit	个人表达、捷径、工具偏好	User preference，默认不推广
Irreducible judgment	风险接受、策略、重大例外	Expert Review / Accountable Decision

可重复、低争议且可机械验证的动作可以落为 Validator 或 deterministic policy；可委派但仍需判断的步骤落在 Assignment、Review 或 Escalation。这是对上述语义分层的实现选择，不是第二套分类。

同一 prototype 往往混合 domain invariant、institution convention、client or Matter context 与 individual habit。出现频率不能自动把个人习惯提升为机构规则，也不能把机构规则写成领域真理。

产品化的完成判据是：prototype 作者离开后，其他合格用户能否依靠显式对象和状态委派同类工作，专家 Review 时间是否下降，accepted-work-product rate 是否提高。如果专家仍然需要以接近原来的频率介入，产品只能支持 expert augmentation，还没有形成可委派的 Agent Extension。

8. Agent Extension、Work Extension 与 E2E 发布

Agent Extension 是由 Work Contract 约束、能够在特定任务族中产生可采用成果，并自带执行边界、结果校验、Review / Escalation 语义和 E2E 发布门的可版本化专业能力包。

它不限定物理形态，可以实现为 package、plugin、skill、workflow、service、sidecar、embedded module 或其他宿主可加载单元。但 Agent Extension 不等于 prompt + tools + workflow；成熟 Extension 至少包含以下逻辑对象：

```
Agent Extension
├─ Task family and applicability
├─ Work Contract
├─ Context / resource policy
├─ Artifact / evidence / completion contracts
├─ Authority contract
├─ Review / escalation policy
├─ Validators / Evaluators
├─ Recovery behavior
├─ Golden / adversarial / boundary cases
└─ E2E release suite
```

Prompt、tool definition 和 workflow 只是可折旧的实现部分，不构成 Extension 的全部身份。

8.1 Work Extension 作为 Runtime 封装

Agent Extension 表示不依赖具体宿主的逻辑能力身份；当它被封装为可由通用 Harness 加载、替换或组合的运行单元时，本文称其实现形态为 **Work Extension package**。二者不是两套专业 ontology：前者保存可移植的工作语义，后者提供面向某一 Runtime 的编译目标与部署封装。

```
Work Extension package
├─ Semantic core
|   └─ Work Contract / State / Artifact / Evidence / Completion / Authority / Review
├─ Runtime adapter
|   └─ required services / tools / events / model and harness compatibility
├─ Human Work Surface
|   └─ inspection / comparison / provenance / correction / review / approval / escalation
├─ State compatibility
|   └─ version / migration / rollback / canonical-owner mapping
└─ Evaluation and release
    └─ Validators / Work benchmark / boundary cases / E2E suite
```

通用 Harness 可以提供 mount / unmount、dependency resolution、event、tool、permission、session、trace、recovery 与 UI slot 等机制；Work Extension 决定这些机制在具体工作中意味着什么。Harness 可以提供“一次性允许、拒绝或询问”的 approval primitive，Extension 必须把它编译为“接受这一版成果”“批准这项风险”“要求补充证据”或“把决定升级给谁”，并规定该操作改变哪项正式状态。HITL 因而不是通用确认弹窗的同义词，而是 Work Contract 的人类裁决面。

这一分层允许 model、Agent loop、Harness 与 Work Extension 以不同速度演化。Model protocol、reasoning loop、streaming、generic tool calling、adapter 和 renderer 通常比 work state、

Evidence、Authority、validation、commitment 与 recovery 更易折旧。成熟 Work Contract 因而优先声明目标状态、不变量、充分证据、禁止转换和提交条件，把具体 planner / worker / reviewer 路径留给可替换 Runtime；更强的模型可以减少 imperative orchestration，不能因此取得更宽的副作用或正式提交权限。Runtime composability 只证明组件能够被组合、替换或撤销，不证明 Work Contract 正确，也不能绕过 state migration、canonical ownership 与 E2E Acceptance。

一种非规范性的实现形态是 **Sparse Work Harness**：

```
Shared Core
  agent loop / tool protocol / event / permission / artifact IO / compiler
+ Work Primitive Packs
  schema / instructions / tools / verifier / transition / surface / policy
+ Matter State
  Current Semantic State / versions / evidence / decisions / open obligations
+ Context Compiler
  activate capabilities + project Matter State + mount gates + exclude noise
```

Work Primitive Pack 不是人格化 Agent，Matter 也不是可执行 expert。Compiler 输出的不是单一 expert ID，而是一份 executable context plan：本次 Run 允许装载哪些 schema、tools、verifiers 与 Human Work Surface，投影哪些 Matter slices，明确排除哪些旧版本、其他 Matter 与越权 capability。Model 可以提出“我认为需要什么”；Harness 必须根据 Contract、role、stage 与 policy 决定“允许装载什么”。由于 capability activation 会改变数据访问、Tool Authority 和现实行动范围，routing 必须采用 learned proposal 与 deterministic permission / commitment gate 的组合。

稀疏组合也不要求每个 turn 动态热插拔。Activation lifetime 可以按 organization → role → matter → stage → run 分层：越靠左越适合预编译，越靠右越适合动态投影。实践上可以采用 cold composition / stage-bound reconfiguration / hot execution，在 Matter 或 stage 内保持 tool definitions、Contract prefix、permissions 与 Surface 相对稳定，只让 Current Semantic State 和 Evidence disclosure 随 Run 变化。Modularity 使 sparse composition 可行；预编排组合才使专家 orchestration 可以被分发。

8.2 Compiled Work Expert 与三层激活

Compiled Work Expert 是一份经过版本化、评测和权限收敛的 activation profile：它把一个或多个 Work Extension / Work Primitive 与机构配置、适用任务族、Review / Escalation policy 和 E2E 证据绑定为可复用工作能力。它不是模拟某位专家的人格，也不是单个 Tool、Matter instance 或新的 Contract 类型。

逻辑对象应当分开：

Work Primitive

→ 原子 schema / tool / verifier / renderer / permission capability

Work Extension

→ 面向某一 Harness 的可分发领域封装

Compiled Work Expert

→ 针对高频任务族或 stage 的预编译、已验证 activation profile

Run Plan

→ Compiled Expert + current Matter State + role + stage 的本次执行计划

成熟 Runtime 可以采用三层 activation:

1. **Preset binding**: Matter type、institution policy、role 与 stage 已经足以确定 Expert，不需要模型路由；
2. **Expert routing**: 模型或规则只在一组已经批准、彼此兼容的 Expert 中提出选择，Runtime 再检查 applicability、version 与 permission envelope；
3. **Primitive composition**: 仅用于无合适 Expert、跨域、低置信度或 frontier exploration；默认处于 least-privilege、Candidate-only 边界，不得直接取得 approve、publish、external transmit 或 irreversible authority。

预编排的价值不只是减少 routing entropy。Expert 以完整工作能力作为评测单位，使 schema + retrieval + tools + verifier + permissions + Human Work Surface + transitions 可以共同接受 Work benchmark 和 accepted-work-product 检验；模型不必在每个 Run 中重新发明 capability graph。一个 primary Expert 可以调用 Contract 明确允许的 auxiliary Experts，但不能从 Registry 的任意子集自由取得组合权限。

“从动态组合编译成 Expert”本身是一项治理过程，而不是成功 trace 的自动缓存：

frontier composition / expert intervention

- Candidate Expert Profile
- applicability and exclusions
- provenance and source demonstrations
- Contract / primitive dependency versions
- held-out E2E, risk-stratified Review and reversal window
- release Authority
- Committed Expert Version

- production monitoring
- revalidate / recompile / suspend / deprecate

Candidate Expert 至少应记录 expert_id / version、semantic owner、适用与排除范围、来源示范和 correction、依赖的 Contract / Tool / model / Harness version、评测样本与 Reviewer 分布、已知限制、最后验证时间、freshness trigger、abstention / fallback、rollback 与 deprecation path。不同领域不必共享一个“成功次数”阈值；法律、咨询、医疗等慢反馈场景可以使用 accepted-work-product、Reviewer disagreement、后续 reversal、风险分层和观察窗口共同裁决是否足以发布。

Compiled Expert 既是资产也是被冻结判断的责任。法规、source catalog、institution policy、base model、Tool provider、Matter distribution 或 Reviewer outcome 变化，都可能使它过时。生产中的 edge case 必须能够让 Expert abstain 或升级到 Human / frontier composition，并形成 Candidate Expert Change；不能为了维持 fast path 而把未覆盖事项强行解释为已覆盖。

因此，fast path / slow path 只能作有限产品类比。Compiled Expert 是经过治理的默认路径，primitive composition 是受限的探索路径；二者不保证产生相同结果。产品化不是单向瀑布，而是：

- Compiled Expert execution
- uncovered case / drift / disagreement
 - abstain or escalate
 - Human expert / bounded primitive composition
 - structured trace and outcome
 - Candidate Expert revision or new Expert
 - E2E revalidation and release

这条回路把 Expert frontier 与 productized work 保持连接，同时防止 active Expert 被 improver 或模型静默改写。

8.3 Extension 的发布判据

Demo 只有同时满足以下条件，才被提升为 Agent Extension：

1. 原作者不再承担逐步驾驶；
2. 其他合格用户能够启动并完成同类工作；
3. 输入、适用范围和不适用边界清楚；
4. required artifacts、evidence、completion 与 review requirements 可以被查询；
5. failure、interrupt、return-for-revision 和 recovery 有可验证行为；
6. 结果能被 Reviewer 作为 accepted work product 接受并进入下一步流程；

- 7. E2E suite 覆盖正常路径、关键反例、权限边界、恢复和模型替换；
- 8. 专家 Review 时间、成果接受率或恢复成本至少一项得到可测改善，且高风险指标不恶化；
- 9. 发布版本具有可查询的 semantic owner、provenance、applicability / exclusions、依赖版本、last validation、freshness trigger、abstention、fallback 与 rollback；
- 10. 未覆盖或发生 drift 的生产事项能够 fail closed、abstain 或升级到 Human / frontier path，并以 Candidate Change 进入重新评测，而不是静默修改 active Expert。

8.4 E2E 的测试对象

E2E 不是页面点击完成、Agent 停止生成或文件成功写入。它测试一套 Extension 能否在声明范围内稳定完成工作：

Representative Matter / Task

→ authorized resources and context

→ Agent execution

→ candidate artifacts and state changes

→ contract validation

→ review / revision / escalation

→ accepted work product

→ downstream handoff or recoverable continuation

完整 suite 可以混合 deterministic checks、schema / evidence / completion / authority validators、evaluator rubric、expert review、accepted-work-product outcome、recovery and replay tests 与 equivalent-interface perturbation tests。E2E 不要求所有专业正确性被自动化，也不能因全绿就宣称整个领域已被形式化。它需要同时观察执行是否成功、状态是否正确提交以及成果是否构成可接受的工作，不能把三者压成一个不透明总分。

8.5 Model、Agentic 与 Work benchmark

专业 Agent 的 Eval 至少包含三层，它们回答不同问题：

层级	主要测试对象	典型问题
Model benchmark	reasoning、knowledge、instruction following	模型是否知道、是否能推理出正确候选
Agentic benchmark	agent loop、harness、tool use、recovery	Runtime 是否能规划、调用、重试、跨工具执行并保持目标
Work benchmark	Work State、专业判断、Artifact 与承诺边界	在真实工作状态下，动作与判断是否组成可采用的工作

Agentic benchmark 相对容易规范化。输入、动作空间、目标状态和技术成功条件通常可以预先定义，例如 tool calling 是否正确、长上下文检索是否稳定、失败后能否 recover、跨文件操作是否完成。它主要测量 Runtime 内部的执行质量。

Work benchmark 的语义来源则位于 Runtime 之外。它不先问“Agent 有没有把流程跑完”，而问“在这个 Matter 的事实、权限、版本与责任条件下，它形成的判断、中间状态和最终 Artifact 是否满足 Work Contract”。测试可以由 Runtime 执行，但 acceptance criteria 必须来自领域、机构和 Reviewer，而不能由 agent loop 自己定义。

例如，一个法律 Agent 顺利完成：

读取材料 → 搜索法规 → 起草 memo → 写入文档

所有 tool call 都可能成功，任务也可能正常停止，但工作仍然失败：

- 它识别错了 Matter 的真正争议焦点；
- 应当要求客户补充事实，却直接开始起草；
- 使用了已失效版本或不适用的 jurisdiction；
- 选择了法律上成立、商业上却不可接受的方案；
- 忽略了证明力不足、冲突来源或未决义务；
- 交付件形式完整，却仍不能被 Reviewer 作为 accepted work product 接受。

这些失败不能仅靠“更好的 planner”或“更多 tool coverage”定义。Work Contract 需要先声明：

Context / Work State
→ Required Judgment
→ Intermediate Artifacts and State Transitions
→ Evidence / Authority / Completion / Review
→ Acceptance Criteria

然后从同一 Contract 派生 deterministic validators、Evaluator rubric、adversarial cases、expert review points 与 accepted-work-product outcome。Benchmark 是 Work Contract 的测试投影，不是后来另造的一套 QA，也不是系统设计的起点。

P19 — Work benchmarks are derived from Work Contracts; runtime success cannot substitute for work acceptance: Work benchmark 从 Work Contract 派生；Runtime 执行成

功不能代替工作成果被接受。

这三类 benchmark 同时对应三层架构、三类 Contract 与三组长期资产：

层级	自有 Contract	主要长期资产	主要任务	成功证据
Model Capability Layer	Context / output interface、能力与安全边界	weights、训练数据与管线、推理栈、Model Eval	扩展 reasoning、knowledge、generation 与候选行动边界	held-out Model benchmark、跨任务能力与成本
Agentic Runtime / Harness	observation / action、tool、session、event、plugin、permission、trace、recovery contract	Harness kernel、service / plugin ABI、tool adapters、sandbox、session / event infrastructure、Agentic Eval	把模型能力变成可执行、可组合、可恢复、可观测的行动	Agentic benchmark、tool success、recovery、trace invariant、跨模型复现
Work Extension	Work Contract、Context / State、Evidence、Completion、Authority、Review、accepted work product	领域语义、机构配置、HITL UX、Validators、Work Eval、failure distribution、accepted / reversal outcomes	把通用能力收敛为特定人群可依赖的工作渗透面	Work benchmark、Expert Review、accepted work product 与 downstream adoption

Coding 同时跨越后两层。一次 bounded issue 主要检验 Harness 执行；多日、跨迭代的软件开发还需要长期 Specification、持续变化的 Artifact State、关于已验证行为与未解失败的 Evidence State、每轮受限目标、role-specific Context 与 Authority，以及独立于实现者自评的 Acceptance。Repository、toolchain、test、CI 与 PR lifecycle 提供 machine-readable、versionable、replayable substrate，定义软件工程 Contract 的一部分；产品意图、Architecture、Compatibility、Release Authority 与运营后果仍由 Work Contract 承担。其他专业领域沿用这组状态与责任关系时，需要以各自的 Evidence 与 Acceptance 语义替换软件测试提供的低成本 verifier。

P21 — Layered contracts, local evidence: Model、Agentic Runtime 与 Work Extension 各自拥有 Contract、Eval、长期资产和演化周期；上游能力不能代替下游成果接受，下游失败也不能未经归因就归咎于模型。

三层 benchmark 可以共同进入 E2E，但不能互相冒充。Model 分数提高不自动证明 Harness 有效；Agentic benchmark 全绿不自动证明工作正确；Work benchmark 也不能反向声称所有专业判断已经被形式化。真正稳定的顺序是：先定义 work-state contract，再从 contract 派生 benchmark，并以真实 Review 和 downstream adoption 校验其外部效度。

8.6 Accepted work product

Accepted work product 指由具备相应 Authority 的 Reviewer 在预定流程节点接受、可以进入下一步业务流程、不需要实质性修改的 Artifact version。Accepted-work-product rate 是 Extension 发布和迭代的核心指标，不是单一模型 benchmark 的替代名；它必须按任务族、风险、机构和 Reviewer 分层，并记录 reversal。

P17 — A demo becomes an Extension only when its hidden human harness is compiled and it passes E2E without its author: 只有当隐形 Human Harness 被编译，并且原作者离场后仍能端到端交付可采用成果，Demo 才成为 Extension。

8.7 产品吸收 Agent 复杂度

Frontier Agent 的能力越广，可配置对象越多。Session、Memory、context window、compaction、handoff、tool call、MCP、plugin、subagent、system prompt 和 model-specific recovery logic 属于构建执行系统的概念，不是专业用户完成工作的概念。

用户需要管理的是 Matter、Assignment、resources、issue、evidence、Artifact、revision、approval 与 outcome。

P10 — Agent complexity should be absorbed by the product: 用户面对专业对象和业务裁决，产品吸收 Session、Memory、Tool、Subagent 与 Context mechanics。

复杂度没有消失。产品把它从用户的逐次配置移入可复用的契约、默认值、路由、恢复与审阅设计。一次产品判断可以服务多次任务，用户无需每次重新发明工作方法。

完全隐藏技术层并非无条件成立。部分专业用户可能需要诊断 Context Projection、工具失败或委派拓扑。产品可以采用 progressive disclosure：默认呈现领域对象，在诊断需要出现时暴露技术证据，但不把技术对象设为主工作面。

8.8 一个完整的提交、退回与恢复过程

一项合同审查 Matter 的目标是形成可以交给业务负责人的风险清单和修订稿。法务负责人是 Accountable Principal；主 Operator 承担审查、恢复、交付和升级义务；Authority Contract 允许它读取合同、提出 finding 和修改 draft，不允许批准或对外发送。

1. Matter 创建

→ 建立原合同、标准条款库、参与人和权限边界

2. Assignment 接受

→ Work Contract 要求覆盖责任、付款、终止和数据义务

→ Completion ledger 登记四类义务与两个交付物

3. 主 Operator 建立两个 Lane

→ Lane A 检查责任与终止

→ Lane B 检查付款与数据义务

4. Lane A 提出 Finding F-17

→ Epistemic Status = proposed

→ Institutional Status = draft

→ Candidate Event 还没有进入 Committed Event Ledger

5. Evidence Check 失败

→ Finding 只有原文摘录, 没有 source version 和 location

→ F-17 保留为 proposed, 不能进入 accepted state

6. Lane B 定位合同版本与条款坐标

→ 提出候选 Evidence Relation = source supports claim

→ Candidate State Change 再次提交

7. Schema、Evidence 与 Authority checks 通过

→ 指定 Reviewer 确认原文在适用范围内支持该事实主张

→ Runtime 写入 Committed Event

→ reducer 把 F-17 投影为 supported / under review

8. Completion ledger 仍显示数据义务未覆盖

→ Operator 不能宣布 Assignment 完成

→ Context Projection 为下一次 Run 加入缺口和相关材料

9. Reviewer 处理候选成果

→ 接受 F-17 的事实基础

→ 退回风险等级判断

→ Candidate Decision 通过 Authority Check 后写入 Committed Event

10. Revision 生成 Artifact v3

→ Artifact Store 保存不可变版本

→ Committed Event 引用 v3

→ 同一次投影把 Current Semantic State 与 active Artifact reference 更新到 v3

11. 法务负责人批准发布

→ Authority 允许 approve, 不允许 Operator external transmit

→ Accountable Principal 的批准写入 Committed Event

→ 发布动作由具备 transmit Authority 的角色执行

12. 新 Run 恢复

→ Runtime 从 Committed Event Ledger 重建 Current Semantic State

- 读取 active Artifact v3 与未完成义务
- 生成新的 Context Projection, 不重放完整 transcript

这条 trace 同时约束 Lane 与 Operator、Candidate 与 Committed、Evidence 与 Completion、Authority 与 Accountability, 以及 Event、State、Artifact 和 Context Projection 的写入顺序。任何一步都不能由模型自述“已经完成”替代。

第四部 Post-agentic Refinement

Post-agentic Refinement 从 Kernel 推导产品运行、学习与模型策略, 但不把任何生产 trace 自动当作训练数据, 也不把任何系统改进都称为 Training。本部的命题需要分别接受成本、数据权利和产品结果检验。

9. Post-agentic Refinement 与 Review 治理

Post-agentic Refinement 是 Agent Extension 已经能够端到端运行之后, 基于生产使用、Review、Revision、Failure 与 E2E 结果, 对整个 Agentic System 进行的后续改进。

改进对象按默认检查顺序包括:

1. Work Contract 是否遗漏或错误表达专业义务;
2. Validator / Evaluator 是否过弱、过强, 或把机构偏好当成事实;
3. Compiled Expert 的 applicability、依赖版本、routing、fallback 或 freshness 是否已经失效;
4. Context Projection 和资源选择是否缺失、过时或冗余;
5. Tool / interface 是否制造 interface accident;
6. Harness / recovery 是否造成系统性失败;
7. Base model 是否存在可测、可泛化的剩余能力缺口;
8. 对剩余能力缺口或可测的质量、成本与效率收益, 判断 Selective Model Post-training 是否值得采用。

专业工作本来就通过 Review 与 Revision 推进。产品应捕获实际发生的修订, 不应要求专业人士为了未来训练, 在每次修改时额外填写 reason code、materiality、scope 和 judgment type。

P11 — Capture first, infer later, promote selectively: 先捕获事实，再推断语义，只把少数经过确认的候选提升为规则。

P11 同时是一项 Capture Economics 原则：结构应主要作为正常工作的副产品产生，而不是由专业人士额外为 Schema 填写。删除一句话、选择另一版、拒绝某来源、要求补充某法域、把候选退回或升级，本身都可以先记录为 Event 与 before / after diff；只有当行为无法消歧、且该判断对未来工作具有足够价值时，才请求最小化 clarification。

人的 correction 可以被捕获、抽象、审阅、版本化并在模型权重之外持续复用。若其仅修改自然语言 instruction 或 heuristic，它仍只是 policy memory，不等于完整 Work Contract；写回还必须受 feedback authority、semantic validation、适用范围和结果归因约束。

9.1 自动捕获的事实

系统自动记录 before / after、Artifact path 与 version、actor 与 role、Matter 与 Assignment、evidence、surrounding context、trajectory、outcome 与 learning rights。这些对象首先服务审计、恢复和当前产品运行。

9.2 机器推断的语义

系统可以带不确定性地推断 correction type、likely scope、materiality、failure family、applicability 与 Reviewer disagreement。推断结果不是正式规则，可以通过跨 Matter、跨 Reviewer 和跨版本比较校正。

9.3 人工确认的规则提升

系统准备改变未来行为时，才要求明确裁决：

- Matter 规则是否成为 client preference；
- 重复 client preference 是否成为 institution configuration candidate；
- 重复 institution correction 是否成为 product 或 domain rule candidate；
- 某类失败是否进入正式 Eval；
- 某类数据是否允许 aggregate、evaluation 或 training。

低频的规则提升决定未来行为，适合承担治理成本；高频 Review 只处理当前工作，不应退化为数据标注界面。

生产中的 revision 不一定能可靠区分 correctness、client preference、strategy 与 style。如果自动语义长期不准，Revision Event 仍然保留审计价值，学习语义需要另行采集。

9.4 Capture Economics

“能够捕获”不等于“值得捕获”。每类结构化信号至少需要同时评估：

- 当前工作本来是否会自然产生该行为或 Artifact；
- 自动推断的准确度和可校正性；
- clarification 的频率、时长与被打断成本；
- 规则未来影响的 Matter 数量、风险和可撤销性；
- 相比普通文档、自由 Review 或不保存，该结构是否降低总专家时间。

如果结构化程度只能通过同比例增加 Expert 操作成本获得，expert-trace flywheel 不能作为产品成立的前提。系统仍可保留原始 Event 与 Artifact diff，用于审计和未来重新解释，不应因无法立即得到完整语义而强迫当前用户标注。

10. Failure Attribution、Production Learning 与 Synthetic Environment

每项生产失败至少应允许落入下列一种或多种类型：

```
contract gap
validator gap
evaluator disagreement
expert profile / routing / staleness gap
context / resource gap
tool / interface accident
harness / recovery failure
model capability gap
institutional disagreement
rights / data limitation
```

失败归因本身可以是带不确定性的推断，不能自动改变正式规则。规则提升仍遵循 capture first、infer later、promote selectively。Work Contract 为这一归因提供任务边界、目标成果、accepted-work-product 标准、可验证义务、failure taxonomy、Eval rubric、Environment seed 以及权限与数据 rights 边界。它不只是 Runtime guard，也是训练目标的治理层。

结构化产品界面可以产生学习信号，不会自动形成数据飞轮。生产 trace 首先揭示由当前产品、客户选择、权限条件和使用方式共同塑造的 observed production distribution：

- task distribution；
- failure distribution；

- preference distribution;
- escalation distribution;
- environment seeds;
- accepted-work-product standard.

这些 trace 可能受客户机密、权限、上下文依赖、Reviewer 分歧和选择偏差限制，不一定适合直接训练。Documents 是内容资产；带有具体情境的 task、state、trajectory、judgment 与 outcome 才可能成为学习资产。两类资产都不能自动取得聚合、评测或训练权。

Synthetic Environment 走另一条路径：

```
Expert defines task family / rubric ontology
→ Synthetic Environment generation
→ oracle or multi-agent solve
→ deterministic checks
→ expert audits disagreements and high-value cases
→ accepted Eval / training environments
```

两条路线承担不同作用：

- Production Learning 揭示系统在哪里工作、在哪里失败、哪种偏好和升级具有现实权重；
- Synthetic Environment 以较低边际成本扩展任务族和失败模式；
- held-out production Matters 检查 synthetic system 是否只优化自己的世界。

P12 — Production traces reveal observed production distributions; synthetic environments expand them: 生产记录揭示当前产品条件下可观察到的分布并提供环境种子；合成环境扩展规模，但不能自行证明它代表完整任务分布。

如果 synthetic-only 路线在 held-out real Matters 上持续达到同等效果且成本更低，生产 instrumentation 仍然服务运行和 failure discovery，不再构成学习壁垒。如果生产失败不能形成可复用 task family，trace 只保留局部审计价值。

11. Eval、Environment、折旧与 Selective Model Post-training

Schema 中面向模型的 scaffold 与面向人和系统的工作语义具有不同寿命。

11.1 有限 Attention 下的抗折旧语义

只要主流智能系统仍然在有限 attention 下，根据当前可见 Context 产生 Output，而工作经验不能持续、可靠、低成本地写回参数，Context 与 Output 治理就不是阶段性的 workaround，而是系统能力的一部分。Context window、模型推理和 Harness 自动化可以继续增强，却不会自动消灭三项约束：

1. 可见信息不等于有效信息，窗口扩大只是把一部分 retrieval problem 转化为 attention allocation problem；
2. 模型不会天然知道每条自然语言的来源、版本、时效、适用范围、Epistemic Status 与 Institutional Status；
3. Output 会进入新的工作状态并成为后续 Context 的上游，错误沉淀会在多轮中放大。

因此，需要主动区分两类对象。

会随模型快速折旧的是 model-facing scaffold：Explicit reasoning recipe、model-specific prompt trick、过细 workflow instruction、checkpoint tool workaround、固定 prefix、特定 tool name，以及补偿当前模型缺陷的 context orchestration。模型越强、Harness 越原生，这些 procedural glue 越应被删除。

不因模型增强而自动消失的是工作语义：Artifact identity 与 version、provenance、authority、permissions、Semantic State、completion、Review、audit、institution policy、rights、temporal validity、matter isolation 与 correction / revocation。模型更强不会自动决定谁有权批准、哪种来源可以采用、哪个旧结论已被覆盖，或者组织什么时候愿意承担结果。

P13 — Delete scaffolding aggressively; preserve semantics deliberately: 主动删除会随模型折旧的脚手架，谨慎保留工作语义。

Schema 保存意义，Harness 负责执行。Harness 越强，Work Contract 中为了引导模型而存在的 procedural instruction 可以越少；但事实是什么、来源在哪里、何时有效、谁能提交、何时完成、什么必须 Review，这些语义不会因为 API、agent loop 或模型版本变化而消失。Schema Engineering 抗折旧的不是某份 schema 永远不改，而是实现层可以持续删减，现实世界的编纂与治理仍然存在。

即使未来 persistent learning 成熟，一部分显式 memory management 也可能减少，但严肃工作仍需 provenance、version control、access control、auditability、matter isolation、temporal validity、human authority 与 revocation。系统不能只知道模型“学会了”某项客户偏好，还必须知道它从何而来、适用于谁、何时被修改以及如何撤销。治理对象可能从外部 memory 部分迁移到“模型学到了什么”，治理责任不会自动消失。

这项判断仍然接受删除测试。如果模型能从 raw context 稳定推断某项契约，并在 accepted work product、责任承担、审计、恢复、多角色协作、成本和 latency 上不劣于显式契约，该契约可以降为

model-facing scaffold、按需投影或审计视图。单纯 benchmark 分数提高、context window 扩大或一次 one-shot 成功不足以得出这一结论。

11.2 Eval 基础设施与条目折旧

Eval infrastructure 管理 task generation、execution、scoring、regression、分层和复核；Evaluator、rubric、labeling guideline 与 Eval item 都是其中的 governed artifacts。它们需要 owner、source criteria、version、适用范围、disagreement、monitoring、Candidate revision、Review、deployment gate 与 rollback。单个 Eval item 会因模型普遍通过、任务分布或制度变化而失去区分力，因而分别标记 stable regression value、frontier discriminative value、distribution relevance 与 legal or institutional validity。

Schema 与 Eval 可以是同一 governed criterion 的执行与测量投影；两者一致只能说明执行符合已声明判据，不能证明判据充分。Runtime 外的评测把产品问题转为可解释信号，沿工作对象与状态变化定位失败，再把修订送回 Contract、Evaluator 或执行系统。未被编码的义务、人类分歧与外部 outcome 使判据本身也能被质疑。Schema 因而既是评测对象，也是归因的坐标；坐标不自动提供原因。

当 Evaluator reason 会进入下一轮生成、状态迁移、拒收、升级或训练信号时，evaluation trace 已经取得执行后果。Criterion、verdict 与 reason 都需要可追溯的版本和修订边界；rubric revision 与普通候选一样，不能静默改变正式规则。

P14 — Eval infrastructure compounds; individual Eval items saturate: Eval 基础设施会复利，单个条目会饱和并折旧。

Environment generation capability 也会复利；单个 environment instance 同样会饱和。

11.3 Harness Specialization 与 Interface Accident

有价值的 Harness specialization 使模型适应 state、evidence、authority、completion 与 escalation 等工作语义。脆弱的 specialization 只适应固定 prefix、tool name、field order、wrapper 或 protocol。

两者需要通过等价语义下的 tool rename、protocol substitution、field-order randomization、Context Projection 变化和环境迁移区分。收益在这些变化后消失，表示系统学到的是 interface accident，portability 低且折旧快。

11.4 Selective Model Post-training 的位置

P15 — Selective model post-training is optional; acceptance criteria come first: 模型权重训练是可选项，接受判据先于训练目标。

对于已经能够通过 E2E 的 Extension，默认改进顺序是：

```
Expert Demonstration
→ Work Contract
→ Agent Extension
→ E2E Acceptance
→ Production Use
→ Failure Attribution
→ Eval / Environment
→ Post-agentic Refinement
→ Selective Model Post-training, if residual gains justify it
```

训练也可以参与首次能力形成，或在质量相同时降低成本。它仍需依据独立声明的工作要求接受评测；产品尚未成立不能由训练计划代替，已有训练投入也不能代替成果接受。

模型权重本身不是显性知识库，而是对训练分布的隐式表示。从训练信号看，可以区分三项连续但不互相替代的积累：

```
codified human knowledge
→ foundation pretraining

software-grounded interaction trajectories
→ agentic post-training

situated expert judgment in governed Work State
→ next Eval / Environment / training candidate
```

Foundation model 大规模吸收了人类曾经文本化、编码化和公开表达出来的知识与行为；Agentic post-training 又强化了模型在计算机和软件环境中观察状态、调用工具、读取反馈、恢复失败和继续执行的程序性 policy。下一层更稀缺的信号，是专业共同体会做却很少完整写下来的情境化判断，包括何时证据不足、何时切换路径、什么变化可以进入 Current Semantic State、哪类例外必须升级，以及什么才算能够交付。这一分层不表示当前 Agent 只有程序性外壳，也不表示数据规模会自动产生新的判断能力。

Schema Engineering 可能在这里产生一类在自然形成的预训练语料中通常不存在、至少不以可执行和可归因形态存在的专业行为信号。法规、合同、论文、SOP 与最终报告记录了世界知识和部分工作成果，却很少完整记录：在某一具体 Work State 下，专家为何补充某项事实、拒绝哪项候选、采用哪个来源、何时升级、怎样修改 Artifact，以及什么条件使成果真正可以进入下游流程。

当专家借助 frontier Agent 把自己的隐形 Human Harness 编订为 Work Contract、Evaluator、Review boundary 和 Agent Extension，并在生产中留下 Committed Event、Current Semantic State、Revision、Failure attribution 与 accepted-work-product outcome 时，系统生成的不只是更多专业文本，而是“专业工作如何在契约下发生状态转换”的结构化行为记录。它不是把专家经验包装成一个静态 skill，而是把情境、判断、行动、审阅与结果保存在同一可校勘的工作结构中。

Coding 提供了一个有限类比：repository、diff、test 与 CI 先是软件生产的基础设施，后来也成为 code model 可利用的训练上游。Coding Agent 又可以帮助构建 AI 进入更多专业场景的交互面和产品；进入这些场景后，专家与模型共同编订新的 Work Contract 和 Extension，进而形成新的 Eval、Environment 与训练候选：

```
Coding Agent / product engineering
→ domain interaction surface
→ Expert + frontier model compile hidden practice
→ Work Contract / Agent Extension / Work Eval
→ governed state, revision, review and outcome
→ validated training candidate
→ broader professional behavior capability
```

这里需要扩大的不是抽象“用户量”，而是 **work penetration surface** 的有效覆盖：更多任务族的横向广度、更深的可委派 lifecycle、跨时间连续性、更接近现实后果的 commitment depth，以及能够观察 later validation / reversal 的反馈深度。只有在这些维度上形成多样、可比较、可归因的 Work State 与 judgment trajectory，规模才可能转化为模型学习信号。

由此得到的是一项待验证命题：Schema Engineering 不只消费模型能力，也可能生产能够扩展模型专业行为边界的新型训练上游。这项命题不因数据存在而自动成立。局部机构偏好、Reviewer 分歧、选择偏差、interface accident、客户机密和 training rights 都可能使这些记录不适合训练；只有经过语义归因、权利确认、held-out Work benchmark 与真实成果检验的信号，才可以进入 Selective Model Post-training。

这里的上位词是 Post-agentic Refinement，只有模型权重更新称为 Selective Model Post-training。只有当后者能够改善 accepted work product、completeness、grounding、escalation calibration、institutional adherence、cost、latency 或 tool efficiency，并且改善幅度可以测量时，它才有采用依

据。如果新的 frontier model 在 accepted-work-product Eval 上超过旧 specialized model，应直接迁移，不因旧 checkpoint 的 sunk cost 延迟替换。

P16 — Do not use training as a substitute for reliable validation; do not globalize local preference; do not let automation substitute for the accountable decision: 训练不能取代可靠验证；局部偏好不能直接推广；自动化不能取代最终问责决定。

可以训练已经能够验证的行为，以提高首次通过率、降低成本或减少重试；Validator 仍然保留判定权。自动化也可以准备材料、提出建议和执行可撤销步骤；Accountable Principal 仍然保留最终裁决资格。

没有自有模型、专有 checkpoint 或 post-training，Professional Work 产品仍然需要依靠 Matter、state、evidence、authority、Artifact 与 Review 产生真实价值。

11.5 系统组合不能替单个组件证明因果

系统表现由 model、Runtime、environment、context、Contracts 与 Evaluator 共同产生。这些组件存在交互：同一项 Harness 改动可能在不同 base model 上改善结果，也可能方向相反。组合系统在自有 benchmark 上表现更好，只能证明这套组合在相应设置下有效，不能据此把主要贡献分配给某个组件。

因此，组件判断需要独立消融、跨模型复现、接口扰动和 held-out environment。

第五部 实现形态、长期资产与证据纪律

Kernel 可以定义架构与治理边界，不能单独证明市场规模、学习壁垒、横向平台机会或通用智能。本部把实现形态、产品价值、分层资产与市场证据分开处理。

12. 适用范围、分层资产与工作渗透面

12.1 适用范围：consequential commitment 与 longitudinal continuity

Schema Engineering 不是“所有 LLM 应用都需要额外 Schema”。它有两个逐层增加的适用范围：

- **Commitment scope:** 概率性 Output 会改变持久共享状态、触发外部行动，或产生需要 Authority 与 Accountability 的现实后果；
- **Continuity scope:** 工作跨 Run、人员或时间继续存在，而 Evidence、Authority、Completion 或专业正确性不能被廉价稳定 verifier 完整判断。

一次发送、发布、合并、批准或不可逆操作即使没有长期 Matter，也需要 Candidate 与 Committed 的边界；完整 Matter、Semantic State 与 Context Projection 则在 continuity scope 中产生更高价值。一个场景越符合以下条件，显式 Work Contract 与状态治理越有价值：

- 工作跨多轮、跨人员或跨时间持续；
- 本次 Output 会成为下一轮 Input，Matter 会不断积累历史；
- 存在来源、版本、时效、状态变化与相互冲突的陈述；
- 专业正确性缺乏廉价、稳定、完整的外部 verifier；
- 错误 Context、遗漏前提或过期状态会显著影响结果；
- 存在 HITL、approval、Authority、Escalation 或最终问责；
- 用户不应承担 Session、Memory、compaction 与 Context 拼装；
- 恢复、版本协调、Review 和未完义务本身具有可见成本。

即使 frontier model 已经能够 one-shot 产出一份高质量研究报告、合同审查、投研判断或咨询建议，这层治理仍然有独立价值。下一次工作仍要判断哪些事实继续有效、哪些只是当时假设、哪些结论已经被新材料覆盖、哪些结果值得沿用、哪些中间推理不应进入长期状态。

P20 — One-shot quality is not longitudinal work quality: 单次交付质量不等于跨时间工作质量；当 Output 会进入后续工作，状态治理就是产品质量的一部分。

反过来，一次性的翻译、改写、摘要或开放 brainstorming，如果 Output 不进入正式状态、不产生后续义务，也没有版本、来源和责任连续性，额外 Schema 的收益通常很低。Schema Engineering 的目标不是把一切工作程序化，而是在需要继续、追溯和承担后果的地方补上最小结构。

一个更直接的选场景方法是把 Judgment density 与 Continuity requirement 分开观察：

	Continuity 低	Continuity 高
Judgment density 低	普通 automation；通常不需要 SE	workflow / traditional software；重点是状态流转
Judgment density 高	strong model / copilot；重点是一次性专业判断	Schema Engineering 最可能产生增量价值

Consequentiality 是另一条正交轴：即使 continuity 很低，一次发送、批准、发布、合并或不可逆行动仍可能需要 Candidate / Committed、Authority 与 Review。上表不是领域分类，而是治理 ROI 的起始假设。产品必须实测从哪个 judgment × continuity × consequence 阈值开始，显式状态和 Contract 的收益超过建模、维护与交互成本。

Coding 的表现反而说明了其他工作为何更需要这层结构。代码仓库已经天然拥有一套长期工作基础设施：

```
file system → state
git → version history
type system → schema
compiler → verifier
tests → eval
diff → change representation
CI → acceptance gate
```

Coding Agent 可以显得高度自由，是因为它背后站着几十年积累的结构、状态和验证系统。法律意见、投资判断、咨询建议、研究结论和客户管理通常没有天然的 `git + compiler + test suite`。Schema Engineering 可以被理解为给非 Coding 工作补上一套弱化、可增量建设的状态、来源、承诺与验证基础设施，而不是把专业判断伪装成可完全编译的程序。

Coding 本身也不构成绝对例外。上述 verifier 主要作用于 **code execution layer**：代码能否编译、测试是否通过、接口是否满足已经声明的检查。软件工程作为一项持续工作还要处理产品意图、架构边界、迁移策略、向后兼容、安全与性能约束、部署准备、跨团队依赖、技术债务和 release authority。Tests green 只说明已声明的 checks 通过，不说明功能、架构与运营条件已经完整实现，也不自动形成成熟软件。

Issue、spec、ADR、PR discussion、code review、release checklist 与 incident record 可以看作软件工程中分散存在的 Work Schema；工程师对遗漏、materiality、trade-off 与完成状态的手动判断，则构成一层隐形 Human Harness。工程师群体通常具有较强的手动 loop 能力，因此这层 Schema 可以更薄、更隐式，也未必值得额外封装成新的产品门槛。但当 Coding Agent 从完成局部 patch 走向 one-shot 交付成熟软件时，瓶颈会重新出现在 model 与 agent runtime 之外：什么才算实现、哪些架构变化可以接受、谁有权合并和发布、旧决定何时被 supersede，以及下一次 Run 必须优先看到哪些约束。

因此，Coding 更准确地包含两层：底层是拥有强 verifier 的代码执行，上层是仍需语义契约和专业裁决的软件工程。Schema Engineering 对前者可以极薄，对后者仍然成立。它也解释了为什么“测试全绿”不能代替 Work benchmark，以及为什么成熟软件的 one-shot 生成不只等待更强模型或更长 agent loop。

在当前范式下，能力抵达现实工作可以被理解为两个方向的持续对接：

```
Upstream expansion
Model scaling / attention / post-training
→ Agentic Runtime / coding infrastructure
→ broader candidate capability

Downstream convergence
Work Contract / Work Extension / HITL UX
→ Commitment Boundary / System of Record
→ accepted work for a concrete person or organization
```

上游负责拓展模型与 Agentic execution 的能力前沿；下游负责把开放能力收敛为任务族、证据、权限、完成、Review 与用户工作面。Provider 到具体的人之间不是一次 API call，而是一条从 model interface、Harness service、Work Extension、正式状态到组织承担的编译路径。随着 Work Extension 覆盖的场景增多、每个场景中可可靠委派的深度增加，系统获得的是不断扩张的 **work penetration surface**。

如果将 AGI 理解为通用能力能够持续抵达多种现实工作，而不只是一项模型 benchmark，那么这种“上游扩展能力边界、下游编订并验证渗透面”的组合，是当前 Context + Output、Agentic Runtime 范式下一种可能的系统实现路径。它不是对 AGI 已经实现的宣称，也不证明任一模型本身具有通用智能；可检验的命题仍然是每层 Contract 是否成立、accepted work 是否增加、成本和责任是否改善。

更远期的递归改进只有在系统不仅能够完成工作，还能够根据 failure 与 outcome 提出对 Contract、Evaluator、Tool、Environment 和 learning procedure 的候选修改，通过反事实实验、独立 Review 与外部现实反馈验证，再由明确 Authority 提交时，才具有可讨论性。本文把它限定为 **governed meta-improvement** 的研究假说；它不由 judgment data 的规模、模型“涌现”或系统优化自身评分器自动推出，也不是 Schema Engineering 成立的必要前提。

当上游能力逐渐商品化，瓶颈会更多移向外部世界：工作是否有可观测状态，专家判断能否被低负担地编订，组织是否存在可执行 Authority 与 Review，数据与行为信号是否取得复用权。Schema Engineering 不只让模型读取世界，也提高世界作为工作环境被模型可靠触达的可编纂性。

最低必要 Schema 仍遵循 2.4 的四个问题：What persists、What expires、What gates action、What deserves attention next。实现不应超过真实 Review、恢复、版本和责任问题所需要的程度。

12.2 分层长期资产与实现边界

Professional Work Runtime 是 Work Commitment and Continuity Kernel 的高保证 profile：模型在可执行契约下参与工作，系统在对话之外维护状态、证据、权限、完成条件、Context Projection 与可恢复历史，Accountable Principal 在需要最终问责的节点裁决。

这一定义可以形成产品、Infra 或 embedded capability，不能单凭概念完整性证明天然成立的横向市场品类。不同领域的 Artifact、Evidence、Authority、Completion、Review 与 Work benchmark 语义并不相同。共享 Runtime primitives 可以降低重复建设，不能替代下游对具体工作的定义。

每层都有自己的长期资产、主要任务与验证方式：

层级	长期资产	主要任务	关键检查
Model Capability	weights、训练与数据管线、推理基础设施、Model Eval	扩展候选推理、知识和生成边界	held-out capability、成本、鲁棒性与跨任务迁移
Agentic Runtime / Harness	kernel、service / plugin ABI、tool adapters、sandbox / permission、session / event / trace、recovery 与 Agentic Eval	让能力可执行、可组合、可替换、可恢复	tool / recovery success、trace invariant、跨模型与等价接口复现
Work Extension	Work Contract、domain vocabulary、state / evidence / completion / authority / review policy、HITL UX、Work Eval、failure distribution、accepted / reversal outcomes	把通用能力编译为边界明确的工作渗透面	accepted work product、Review 时间、completion、grounding 与 downstream adoption
Product / Organization	distribution、Expert access、institution configuration、Runtime integration、canonical ownership、rights 与采购关系	让工作能力被具体的人采用并取得正式效力	留存、替换、支付、责任承担、跨 Matter 复现与合法复用

Schema-derived behavior signals、Environment generation 与 Post-agentic Refinement 横跨 Work Extension 和组织层，但不能绕过 rights、failure attribution 与 held-out Work benchmark。不存在自动成立的单一 moat；这些资产可能组合后产生采购价值，也可能分别被 frontier provider、平台商、开源生态或垂直软件商品化。

机构能够沉淀的对象是 Work Contracts、precedent hierarchy、institution policy、client preference、completion standards、Review patterns、escalation rules、Work Eval infrastructure、environment generators、Context Projection policy 与 trained policies。只有当这些对象使专业判断和工作连续性超越原始个体而稳定复现时，它们才构成组织能力。

如果上游平台提供通用 Matter persistence、Artifact state、permissions、Evidence、Completion、Review、Context Projection 与 commitment primitives，并使垂直产品无法在专业语义、成果接受率、集成留存、状态治理成本或支付意愿上形成改进，Schema Engineering 仍然可以作为设计纪律，但不再构成独立产品机会。

13. 采用路径与既有系统

Deployable Minimum 从一条真实 commitment boundary 开始：模型提出某项结果，某位具备 Authority 和 Accountability 的人判断它能否进入正式工作。第一版只需要把这次转换从专家脑中移到产品中，不需要先建设全部 Runtime primitives。

13.1 Minimal Viable Matter

第一版不从完整领域 ontology 或全量 Contract 开始，而从四个最低问题选择对象：什么必须留下、什么会失效、什么阻止提交、下一次首先看什么。最小切片可以压缩为：

```
1 Matter
+ 1 Artifact type
+ 1 Reviewer role
+ 1 formal commit transition
+ 1 active version pointer
+ 1 unresolved-obligation list
+ 1 next-context projection
```

对应实现只需要一个稳定 Matter identity、一个边界明确的 Assignment、Candidate Artifact 与 active Artifact 的区分、一个 Accountable Principal、propose 与 approve 两级 Authority、可见的未完义务、带版本的 commit / reject / revise 记录，以及能够从当前状态恢复的结构化工作包。

Evidence Contract 可以先覆盖最承重的事实主张；Completion 可以由少量 checklist 和人工裁决共同完成；Context Projection 可以只是由 active Artifact、开放问题、适用版本、Assignment 与少量相关来源组成的工作包。Output 侧也只需明确哪一版可以提交、哪些 finding 仍是 proposed、哪些旧状态被 supersede。这些逻辑对象可以存在于同一应用和数据库中，不需要先拆成独立服务。

只要产品能够回答哪一版已经生效、谁接受了它、什么还没有完成、哪些信息已经失效、下一次执行首先应看什么，它就已经越过 Session-first 产品的边界。后续扩展可以沿着以下路径发生：

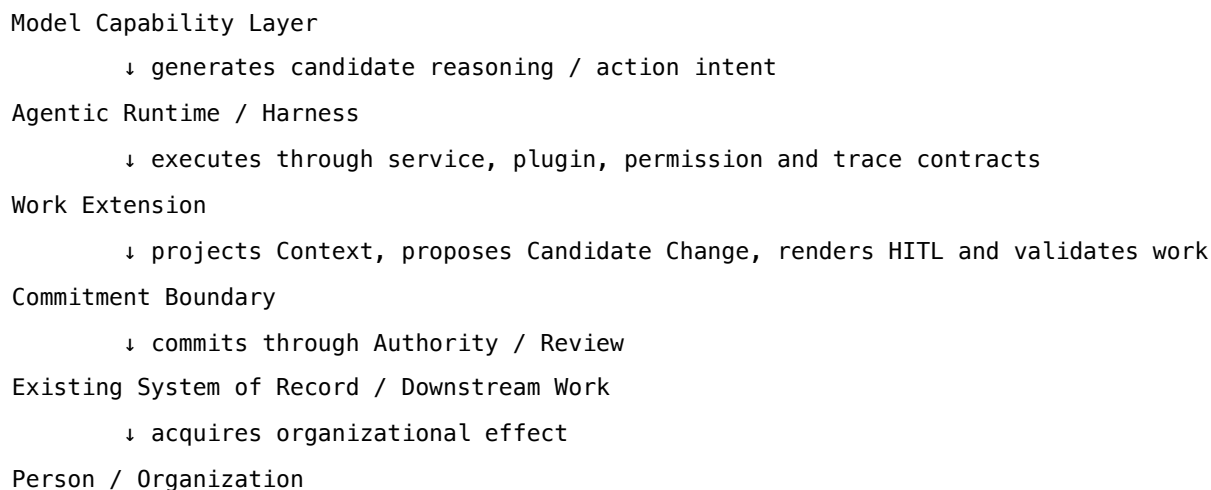
```
Chat + invisible human gate
→ explicit Candidate / Accepted Artifact
→ persistent Matter State
→ task-specific Context Projection
→ typed Evidence / Authority / Completion Contracts
→ reusable institution configuration
→ Work Eval and selective learning
```

这条路径不是强制成熟度模型。高风险场景可能先建设 Authority，研究场景可能先建设 Evidence，长程任务可能先建设 Completion、Context Projection 与 Recovery。第一项 Matter 宜具备清楚的交付

物、已经存在的 Review、明确的 Reviewer 或 Accountable Principal、可重复任务，以及状态丢失、版本混乱、重复注入或漏项造成的可见成本；第一版不宜要求模型执行不可逆的外部行动。

13.2 分层 Runtime、Work Extension 与 System of Record

Schema Engineering 不要求拥有新的 Agent Harness 或 system of record，也不要求形成独立物理平台。逻辑上的调用与承诺链可以表示为：



通用 Harness 负责 model adapter、tool execution、session、event、plugin lifecycle、sandbox、approval seam、trace 与 recovery；Work Extension 负责具体 Task family、Work State、Artifact、Evidence、Completion、Authority、Review 与 HITL 语义。二者之间应通过稳定 service、event、permission、UI slot 和 state interface 连接。宿主可以替换，领域语义不应被迫重写；领域 Contract 可以修订，也不应要求重新训练模型或 fork Harness core。

Case Management、EHR、BPM、审计平台和垂直 SaaS 可能已经持有 Matter identity、客户或患者资料、Artifact、用户角色、审批流程、正式状态和 retention policy。Schema and Commitment Layer 不应无条件复制这些对象，而应补充 Candidate Change 与 Current Semantic State 的区分、model-facing Authority、Evidence Relation、Completion obligation、persistence / expiry / supersession policy、Context Projection，以及跨模型 Run 的恢复和升级语义。

每一类正式对象必须只有一个 **canonical owner**。AI sidecar 或 Work Extension 可以持有 Candidate、Context Projection 和执行 trace，但不能与原系统各保存一份互相独立的“已批准 Artifact”。最终批准状态必须提交到指定的 system of record，或者由 Schema Runtime 明确成为该对象的 canonical owner。Event Ledger 也可以引用外部系统的 immutable event ID、审批记录和 Artifact version，不需要复制所有既有事件。

13.3 两条正交部署轴

Work Extension 如何装入 Runtime，与谁拥有正式状态，是两条正交轴。第一条是运行封装：

Runtime 封装	宿主关系	适用情形
Host-loaded Work Extension	通过 plugin / package / bundle 加载到通用 Harness	复用宿主 tool、session、permission、trace 与 UI seam，保持领域包独立版本
Embedded Module	作为垂直产品内部模块	产品已经拥有主要工作对象和 UI，需要最短调用链与一致体验
Remote Service / Sidecar	通过 API、event 或 protocol 与 Harness / 业务系统连接	隔离权限、语言栈或部署边界，允许独立扩缩与升级

第二条是正式状态的 canonical ownership：

形态	Canonical owner	Schema Engineering 的位置
Overlay / Sidecar	既有专业系统	读取正式状态，生成 Candidate Change，通过原系统 API 提交
Embedded Runtime	Case Management、EHR、BPM 或垂直 SaaS	Proposal、Evidence、Completion、HITL 与 Context Projection 成为原产品内部能力
Greenfield Vertical System	新的 Work 产品	同时拥有 Matter、Artifact Store、Committed Event Ledger、Review 与模型执行层，并允许外部系统拥有部分对象

Hot swap 只作用于第一条轴。即使 Runtime 能安全替换一个插件，也不能因此改变第二条轴上的 canonical owner，不能自动迁移正式 State，也不能跳过 Authority 与 Review。既有系统实现了部分 Runtime primitives，也不表示 F14 已经成立；只有当上游平台进一步商品化 model-facing commitment interface，并且下游无法在专业语义、成果接受率、集成留存或支付意愿上形成差异，独立垂直机会才会消失。

14. Boundary Tests

Boundary Tests 分别检查架构、连续性与治理、产品价值。三组结果不能合并为一个总分；Model、Agentic 与 Work benchmark 也必须保留各自的解释边界。

14.1 Architecture

- Candidate State Change 只有经过 Schema、Evidence、Authority 与适用的 Review Policy 检查，才能写入 Committed Event。
- Committed Event Ledger、Current Semantic State 和 Artifact Store 之间只有一条明确的权威关系，不会各自形成相互冲突的状态。

3. Evidence Relation、Authority、Operational Responsibility 与 Accountability 存在于 prose 之外，并且可以分别查询。
4. Assignment completion 不由 Agent 单方面宣布。
5. 并行执行不会模糊 Operator 的运行义务，也不会改变 Accountable Principal 的最终问责。
6. Work Contract 能够派生 validators、Evaluator rubric、Work benchmark 与 Review points；tool success、Run stop 或 Agentic benchmark 全绿不能直接把结果提升为 accepted work product。
7. Work Extension 通过声明式 service / event / permission / UI seam 依赖宿主；更换语义等价的 Harness provider、model adapter 或 UI channel，不会改变 Work Contract 与 accepted-work-product 标准。
8. HITL action 具有领域化 decision type、Authority、evidence context 与正式状态后果，不以一个无语义的通用确认框替代 Review Contract。
9. Capability activation 由 Contract、role、Matter stage 与 deterministic permission gate 约束；模型不能仅凭自述挂载越权 Tool、其他 Matter 数据或绕过 Review 的执行路径。
10. Work Extension 可以在组织、角色、Matter 或 stage 层预编排，也可以在 Run 层动态投影；AOT 与 JIT 组合只要语义等价，就产生相同的 Authority、Commitment 与 accepted-work-product 标准。
11. Compiled Work Expert 具有独立 identity、version、semantic owner、provenance、applicability、dependency compatibility、E2E evidence、freshness trigger、abstention / fallback 与 rollback；模型或 improver 不能直接改写 active version。
12. Preset binding、Expert routing 与 primitive composition 具有明确的升级和降级边界；primitive path 默认 Candidate-only，Compiled Expert 遇到未覆盖事项可以 fail closed 并回到 Human / frontier path。

14.2 Continuity / Governance

1. 用户返回的是 Matter，不是重建一段 Chat。
2. 不阅读完整 transcript 也能理解当前工作状态。
3. Artifact、approved decision、来源关系和未完成义务能跨模型、跨 Session 存在。
4. Human Review 通过 Committed Event 改变正式状态，不只留下评论。
5. 系统能够分别回答：发生过什么、现在是什么、模型下一次应看什么、这次运行产生的什么可以留下。
6. 未提交、被驳回、已撤回或已 superseded 的 Output 不会静默进入 authoritative state 或下一轮优先 Context。

7. Context Projection 可以从 Stable Contract、Current Semantic State 与可检索历史重建，不以反复压缩完整 chronology 作为唯一恢复方式。
8. 当 Matter、Artifact、history 与可用 Work Primitive 总量扩大时，单次 Context、Tool surface 与 Human Work Surface 的规模主要随当前 Assignment working set 变化，不随 Store 总量线性增长；关键约束遗漏率、错误版本重引入率和成果接受率不因稀疏投影而恶化。
9. Compiled Expert 的发布、暂停、重验证、替换和弃用均有 Committed version history；生产 edge case、Reviewer disagreement 与 later reversal 可以回溯到具体 Expert version 和 release decision。

14.3 Product Value

1. 普通专业用户不必学习 Agent engineering 才能完成主要工作。
2. 不以未来训练飞轮承诺代替当前成果接受；使用外部模型或自有训练模型的实现，都按同一工作要求判断产品是否成立。
3. 移除 prototype 作者后，合格用户的成果接受率、恢复成本或专家 Review 时间至少有一项得到可测改善，且其他高风险指标没有恶化。
4. 一个边界明确的 Matter 能够在不替换既有 system of record、不部署完整 Runtime 的条件下，分开 Candidate 与 Committed，并可测地降低 Review、恢复或版本协调成本。
5. 与反复全量注入 Raw History 相比，Current Semantic State + task-specific Context Projection 至少在成本、latency、状态一致性、恢复或 accepted-work-product rate 中改善一项，且审计与开放问题发现不恶化。
6. Model 或 Agentic benchmark 的提升只有在 Work benchmark、accepted work product、completion、grounding、escalation calibration 或真实 downstream adoption 上产生改进时，才能被解释为专业产品能力提升。
7. 结构主要由正常工作行为、Artifact diff 与实际 Review 自动产生；在固定成果质量下，额外 clarification、字段填写与 Contract 维护占用的 Expert 时间低于它节省的 Review、恢复、重复 orchestration 与错误成本。
8. 把 expert 自由 orchestration 编订为 Work Extension 后，其他合格用户能够在不学习 Tool routing、Prompt、Plugin 与 Context mechanics 的情况下复现声明范围内的工作，且 Expert 由逐次驾驶转为低频 Contract / rule governance。
9. 高频任务通过 preset / Compiled Expert 执行时，routing 与 orchestration 成本下降；未覆盖任务进入 frontier path 后能够安全升级并回流为 Candidate Expert，而不会因 fallback 被掩盖或使普通用户承担开放式插件编排。

Accepted work product 指由具备相应 Authority 的 Reviewer 在预定流程节点接受、可以进入下一步业务流程，并且不需要实质性修改的 Artifact version。格式和措辞调整可以单独记录；改变结论、风险等级、事实基础或行动建议属于实质性修改。评估期内被推翻的成果需要记录 reversal，结果按风险等级、任务族、机构和 Reviewer 分层报告，不能只给一个总接受率。

这些条件只规定产品边界，不证明具体实现已经满足。测试需要覆盖真实 Matter、任务恢复、模型替换、权限边界、Review 状态转换、Context Projection、Output commitment、三层 benchmark 和 accepted work product。

15. 证据纪律与可证伪边界

Schema Engineering 的产品命题、学习命题与市场命题具有不同证据强度。Matter-first、external Semantic State、Evidence、Authority、Completion、Review、Operational Responsibility、Accountability、Context Projection、Candidate Output commitment 与 Lane parallelism 属于产品结构判断；Work benchmark 的增量解释力、attention economics、Production Learning、schema-derived behavior signal、institution-level generalization、synthetic economics、refinement capability 与横向市场品类仍需实证。抗折旧判断也建立在“有限 attention 下由 Context 生成 Output，且工作经验不能可靠持续写回参数”的条件上，不是无条件技术预言。

证据来源也不能合并为“已验证”。独立复现、独立 benchmark、包含负面结果的 vendor technical report、vendor self-benchmark、official product documentation、community observation 与产品推论分别回答不同问题。Vendor self-benchmark 只能证明供应方在自定设置中的结果；official product documentation 可以证明某项机制或边界已经被产品公开提供，不能证明其质量、因果或通用性；community observation 可以暴露现象，不能单独确认因果；产品推论需要另行检验用户行为、成果质量和经济性。

15.1 证据命题与检验单元

Canonical 只保留不随产品版本折旧的命题和证据责任。每项待验证主张至少写成 claim + scope + observable mechanism + outcome measure + comparison + disconfirming result + evidence class and date。实例、产品机制、个人实践和来源不进入 Kernel 的定义或推理链；它们只能支持自己直接呈现的局部机制。多个必要条件同时出现，不构成对整体架构的充分证明。会折旧的来源、局部解释、检验状态与增量裁决另由带日期的 Practice Index 维护；移除该 Index 不影响本文的完整性。

15.2 证据解释纪律

证据强度由来源独立性、方法可审查性、负面结果保留、测量与命题的对应、反事实设计、重复性与外部效度共同决定。不同类型证据不得合并成“已验证”。

下表限定各类证据的最大解释范围：

证据类型	可以支持	不能单独支持
可观察产品机制与官方文档	某项 surface、scope 或 mechanism 被提供	不可观察的 backend ontology、效果大小或通用因果
供应方报告与自建 benchmark	在公开设定下的局部结果	跨任务、跨机构或一般专业正确性
社区与个人实践	可检验的现象、候选机制或失败模式	因果、收益幅度或外部效度
独立复现、消融与真实 outcome	在声明边界内的机制与结果关系	超出采样、权限、时间窗口和任务分布的外推
本文推论	待验证的系统命题和反证设计	外部背书或已经成立的产品事实

从人需要理解的结构与权衡，推到系统需要表示和保存的状态，是一项额外命题。可观察的 managed-resource 与 scoped-context surface 只证明某项机制已被提供，不能反推后端 ontology 或 Govern layer 已成立。

机制类比只用于生成问题，不用于证明答案。参数模块的 sparse routing 不能与异构、状态化、带 Authority 的 Work Primitive activation 互相证明。多个局部必要条件同时出现，仍不自动构成整体架构的充分验证。

15.3 证伪结果

下列结果会使相应命题失效或缩窄：

编号	证伪结果	直接后果
F1	仅用 Conversation History 的 Agent 在长期专业 Matter 中持续等于或优于外部 Semantic State	P3、P4 与外部状态复杂度下修
F2	Session-first 在恢复、审阅、版本和搜索成本上不劣于 Matter-first	Matter-first 降为特定信息架构选项
F3	Runtime ontology 在多个领域反复退化为大量例外	跨领域范式退回少数垂直架构
F4	外部化专家干预后，专家介入频率和成果接受率没有改善	定位退回 expert augmentation
F5	Structured Contract 持续漏掉开放问题或制造错误确信	Contract 缩到 accountability fields，部分任务放弃结构化
F6	专业质量依赖持续细粒度人工干预	Human Review 不能只集中在里程碑或终态
F7	自动捕获 Revision 无法低成本形成可靠语义	Revision Event 只保留审计价值
F8	Synthetic-only 在真实 Matter 上同效且成本显著更低	Production trace 不再构成学习壁垒

编号	证伪结果	直接后果
F9	生产失败无法形成可复用 Environment Seeds	Trace 只保留局部审计价值
F10	机构内分歧不低于机构间，机构规则降低成果接受率	配置停留在 Reviewer 或 team scope
F11	Rights 使合法可复用信号不足以支撑学习	学习分支失效，产品结构不受影响
F12	在长期 Matter 中，Raw History 全量注入与 raw-context inference 持续在 accepted work product、开放问题发现、成本、latency、问责、审计和恢复上不劣于显式 Contract、Current Semantic State 与 Context Projection	相应 Contract 与外部状态降为 scaffold、按需投影或审计视图；P18 的适用范围缩小
F13	等价接口变化使 specialization 收益持续消失	Harness 只学到 interface accident
F14	Frontier provider 商品化通用 Runtime primitives，且垂直产品无法在专业语义、成果接受率、集成留存、状态治理成本或支付意愿上形成改进	设计纪律保留，独立产品机会消失；如果垂直差异仍然成立，只消除 Runtime primitives 本身的差异化
F15	共享 Runtime 没有独立采用、复用或支付价值	横向市场类别不成立
F16	只有暴露技术对象才能取得显著更好结果	采用 progressive disclosure，不彻底隐藏技术层
F17	任何有价值的使用都必须先完成全套领域契约和 Runtime 建设，且前置编码、集成与维护成本持续高于节省的 Review、恢复和版本协调成本	增量采用路径失败；Schema Engineering 保留为架构纪律，但不构成可独立启动的产品 wedge
F18	Agentic benchmark 在跨任务族、风险和机构的 held-out Matter 上已经充分预测 accepted work product，Work Contract 派生 rubric 不再提供增量区分力或失败解释	独立 Work benchmark 降级；E2E 可以更多依赖 Agentic benchmark 与真实 outcome
F19	实际高价值使用主要是一次性任务，Output 很少进入后续工作，持久状态与 Context Projection 不改善留存、成果质量、恢复或成本	P20 与 longitudinal product wedge 缩到少数连续性场景；Commitment Profile 仍保留于发送、发布、批准、合并和不可逆行动
F20	在固定模型、任务分布、expert-hour、rights 与计算预算下，Work Contract、Committed Revision、Review 与 outcome 形成的结构化信号，在 held-out Matter 上不优于普通文档、人工 SOP 或 synthetic-only 数据	“Schema Engineering 生产新的专业行为训练上游”命题失效；这些对象仍保留运行、审计和 Eval 价值，不构成模型能力扩展路径
F21	Work Extension 只有持续 patch 某一 Harness 内部实现才能工作，或在所需能力已满足的 service、plugin、model adapter、UI channel 替换后，反复破坏既定工作语义或无法维持声明的成果接受条件	独立 Work Extension 封装与可移植性命题缩窄；相应场景可能需要与单一 Runtime 深度耦合，但 Work Contract 与 commitment governance 仍需保留
F22	在固定任务与模型下，Store → Govern → Retrieve → Compile 相比 files + search + raw context 没有改善状态一致性、critical omission、token、latency、Review 或 accepted work product，或 Govern layer 的长期维护成本高于收益	该管线降为实现偏好；Govern 只保留最小 accountability fields，其余交给 retrieval 与 Human Review
F23	结构化信号只能通过高频 Expert 填报获得，额外操作成本与数据量近似同比增长，implicit capture + inference + selective clarification 无法降低成本	expert-trace learning 分支失效；Schema 只保留当前运行与审计必要对象，不宣称可规模化蒸馏隐性知识

编号	证伪结果	直接后果
F24	Matter、Artifact、history 与 capability 总量增长时，per-run Context / Tool surface 仍近似线性增长，或稀疏投影持续遗漏关键约束、混入其他 Matter 与旧状态	Sparse Work Harness 与 attention-scaling 命题失效；退回更窄场景、固定 profile 或更简单 retrieval
F25	预编排 Work Extension 无法让非原始 Expert 复现工作；合格用户仍需逐次选择 tools、修 Prompt、解释 context 和判断全部下一步	“precompiled composition makes expert orchestration distributable”失效；产品定位退回 Expert frontier / augmentation
F26	Compiled Expert 的有效性无法通过 owner、version、freshness trigger、reversal 与 held-out outcome 维护；旧规则和旧 source / Tool / policy 持续以“fast path”污染成果，或维护成本随 Expert 数量不可控增长	Expert Registry 降为临时 preset；正式 Authority 回到 Human / dynamic path，不宣称 Expert 层可持续复利
F27	长尾 composition 的“成功”在高 judgment 场景中无法形成可审阅的发布证据，Reviewer 分歧、慢反馈与事后 reversal 使 Candidate Expert 无法可靠晋级	Expert promotion 保持机构内、人工且低频；自动编译飞轮失效，不影响 Work Contract 和 Matter governance 的运行价值

每项证伪条件都需要配套测试设计。长期连续性比较应包含中断、跨周工作与模型替换；Ontology 比较应覆盖责任结构不同的至少三个领域；Contract 比较应同时测开放问题发现、锚定、错误确信、成果接受率、token、latency 与维护成本；Model / Agentic / Work benchmark 比较应固定模型、Harness 与任务分层，检查对 accepted work product 和 reversal 的增量预测力；学习路线比较应固定 expert-hour、计算、rights 和数据预算，并比较 schema-derived signal、普通文档与 synthetic-only 的增量收益；Harness 测试应在语义等价的接口变化下进行；产品边界应以真实采购、替换、留存和支付行为验证；增量采用测试应固定 Matter 范围，完整计入编码、集成、迁移和持续维护成本，再与 Review、恢复、Context 构造和版本协调收益比较。

16. 待回答的问题

1. 最小 Semantic State 应包含哪些内容，才能改善连续性而不成为第二份过时 transcript？
2. 哪些 Completion obligation 足够稳定，可以编码；哪些只能作为 Reviewer prompt？
3. Runtime 如何表示同等授权 Reviewer 之间的真实分歧？
4. Lane 在什么条件下应被提升为独立 Operator？
5. 哪些 Event 对审计必要，哪些只是应当过期的噪声？
6. Context Projection 如何暴露状态可能过时、冲突或置信不足？
7. 一项重复 correction 从 Matter 提升到 client、institution 或 product scope，需要什么阈值？
8. Accepted-work-product Eval 如何同时抵抗 benchmark saturation 与 institution overfit？
9. Training rights 不存在时，aggregation 与 evaluation 至少需要哪些权利？
10. 法律实现中的哪些对象必须保留为领域专属，不能进入通用 Runtime？

1. 从 Expert Demonstration 抽取稳定义务时，多少个 Matter、多少位 Reviewer 才足以区分 domain invariant 与 individual habit?
2. Extension 的 E2E suite 如何覆盖任务族边界，又不把全绿误当成整个领域的专业正确?
3. Failure attribution 如何表示多因一果、归因不确定性与机构间真实分歧?
4. Work benchmark 应以多大颗粒度从 Work Contract 派生，才能解释真实失败而不固化单一机构或 Reviewer 的偏好?
5. 什么规则决定 Candidate Output 的 persistence、expiry、supersession 与下一轮 attention priority，且不把 Current Semantic State 重新写成一份压缩 transcript?
6. 如何在固定模型、任务与 accepted-work-product 标准下，测量 Context Projection 相对全量历史注入的 token、latency、Review、漂移与恢复收益?
7. Persistent learning 达到什么可靠性、可撤销性和权限隔离标准后，哪些外部状态可以降级，哪些 provenance 与 institutional semantics 仍必须显式存在?
8. Capture → infer → promote 的产品界面如何把规则提升集中为低频裁决，而不让高频 Review 重新退化为数据标注?
9. Work Contract、Evaluator、Committed Revision、Review decision 与 accepted outcome 中，哪些对象真正含有普通专业文档和 SOP 不具备的行为信号；怎样区分跨场景能力、机构偏好与 interface overfit?
10. 在软件工程场景中，哪些高层 Completion、Architecture、Compatibility 与 Release 义务能够形成超越 tests / CI 的最小 Work Contract，而不重复工程师已有的手动 loop?
11. Topic-level memory、Project / workspace 与 Matter 之间的 promotion rule 应如何定义，才能既避免 Session-first 退化，也避免把每条持久记忆或宽泛主题都建成 Matter?
12. Work Extension 与通用 Harness 之间的最小稳定接口应包含哪些 service、event、permission、HITL UI、state migration、rollback 与 compatibility 语义，才能既支持热插拔，又不让 Runtime-specific code 污染 Work Contract?
13. 外部案例中的哪一项最小机制必须被独立复现，才能把 vendor observation 从方向性证据提升为可归因证据；又如何避免把同一技术周期中的共同约束误写为对完整架构的独立收敛?
14. Govern layer 的 owner、版本、Reviewer disagreement、deprecation 与 rollback 如何定义，才能避免它成为少数专家永久维护的第二套业务系统?
15. 在 Store 持续增长时，如何同时测量 Context compiler 的 critical omission、irrelevant disclosure、stale-state reintroduction、token、latency 与 cache stability?
16. Work Primitive Pack 的最小接口应如何划分 schema、tool、verifier、permission、transition 与 Human Work Surface，才能既支持组合又避免领域 ontology 碎片化?

27. Organization、role、Matter、stage 与 Run 各适合承担多长的 activation lifetime；何时应 AOT 预编排，何时应 JIT 重编译？
28. Expert orchestration 从自由 Demo 提升为可分发 Extension，需要多少跨 Matter 与跨 Operator 复现，才能区分稳定工作结构与单个 Expert 的有效捷径？
29. Compiled Expert 的发布门应如何组合 accepted-work-product、Reviewer disagreement、风险分层、观察窗口与 later reversal，才能在慢反馈专业领域判断“足以部署”而不制造虚假确定性？
30. 哪些变化应触发 Expert 的自动重验证、暂停或重新编译；怎样区分 source / regulation / institution drift、model / Harness drift 与真实工作分布变化？
31. 当 Compiled Expert 遇到未覆盖事项时，abstention、Human escalation、bounded primitive composition 与 Candidate Expert revision 如何形成低摩擦闭环，而不把 ordinary user 重新推回开放 orchestration？
32. Review packet 至少需要哪些 Evidence、delta、uncertainty、consequence 与 Authority 信息，才能证明人在有限 attention 下形成了独立判断，而不是只完成形式 approval？
33. Context Projection、Human Work Surface 与 Retrieval Index 基于同一权威状态解释来源与候选时，如何检测 omission、staleness 与 cross-projection inconsistency，又不把三种视图物理锁死为同一表示？
34. Evaluator reason 被下游执行消费时，怎样分别版本化 criterion、verdict、rationale 与 state consequence，并检测“标签正确但归因错误”的闭环污染？

17. 分层原则索引

Compilation Principles

- **P17 — A demo becomes an Extension only when its hidden human harness is compiled and it passes E2E without its author.** 只有当隐形 Human Harness 被编译，并且原作者离场后仍能端到端交付可采用成果，Demo 才成为 Extension。

Ontology and Continuity Principles

- **P1 — Matter over Session.** Matter 是用户对象，Session 是基础设施对象。
- **P2 — Conversation is an interaction surface, not the product ontology.** Conversation 提供交互，不承载工作本体。
- **P3 — Transcript is evidence of execution, not canonical work state.** Transcript 记录执行，不代表当前正式状态。
- **P4 — Continuity comes from Stable Contracts, Current Semantic State and retrievable history.** 连续性来自稳定契约、当前状态和可检索历史。

- **P9 — Lane is parallelism; Operator carries operational responsibility; the Accountable Principal carries ultimate accountability.** Lane 表示并行，Operator 承担运行义务，Accountable Principal 承担最终问责。
- **P18 — Context is a projection; output is a candidate state update.** Context 是针对当前工作生成的投影；Output 只是候选状态更新，不能默认沉淀为 Memory。

Commitment Principles

- **P5 — Explore broadly within authorized scope; commit narrowly across the formal boundary.** 在授权资源范围内广泛探索；只有通过带有明确类型、证据、权限和问责主体的窄化边界，才能改变正式状态。
- **P6 — Model proposes; systems enforce invariants; Evaluators measure specified semantics; humans adjudicate irreducible judgment.** 模型、系统、Evaluator 和人分别承担提议、约束、测量和裁决。
- **P7 — Provenance is state, not prose.** 来源关系属于工作状态，不能只写在说明文字里。
- **P8 — Completion must exist outside the model's self-assessment.** 完成条件必须由模型之外的对象维护。

Layering and Evaluation Principles

- **P19 — Work benchmarks are derived from Work Contracts; runtime success cannot substitute for work acceptance.** Work benchmark 从 Work Contract 派生；Runtime 执行成功不能代替工作成果被接受。
- **P21 — Layered contracts, local evidence.** Model、Agentic Runtime 与 Work Extension 各自拥有 Contract、Eval、长期资产和演化周期；相邻层不能互相借用正确性。

Product Governance

- **P10 — Agent complexity should be absorbed by the product.** 产品吸收 Agent 的技术复杂度，用户处理专业对象。
- **P11 — Capture first, infer later, promote selectively.** 先捕获事实，再推断语义，最后选择少数候选提升为规则。
- **P12 — Production traces reveal observed production distributions; Synthetic Environments expand them.** 生产记录揭示当前产品条件下可观察到的分布，合成环境扩展覆盖。
- **P20 — One-shot quality is not longitudinal work quality.** 单次交付质量不等于跨时间工作质量；当 Output 会进入后续工作，状态治理就是产品质量的一部分。

Evolution Strategy

- **P13 — Delete scaffolding aggressively; preserve semantics deliberately.** 删除会随模型折旧的脚手架，保留工作语义。
- **P14 — Eval infrastructure compounds; individual Eval items saturate.** Eval 基础设施可以积累，单个条目会饱和和折旧。
- **P15 — Selective model post-training is optional; acceptance criteria come first.** 模型权重训练是可选项，接受判据先于训练目标。
- **P16 — Do not use training as a substitute for reliable validation; do not globalize local preference; do not let automation substitute for the accountable decision.** 训练不能取代可靠验证，局部偏好不能直接推广，自动化不能取代最终问责决定。

Compilation、Ontology / Continuity 与 Commitment 三组共同定义 Schema Engineering Kernel；Layering and Evaluation Principles 规定 Model、Agentic Runtime 与 Work Extension 的证据边界，以及 Work benchmark 如何从 Work Contract 派生；Product Governance 和 Evolution Strategy 是从 Kernel 推导出的运行纪律，需要单独验证。P17 描述专业能力编译的发布门，并覆盖 Candidate Expert 到 Committed Expert Version 的发布、重验证和弃用纪律；P18 描述 Context / Output 的连续性边界，并要求 Store 与当前 working set 通过 Govern、Retrieve 与 Compile 分离；P19 描述 Work Eval 的语义来源，P20 描述长期质量，P21 描述分层 Contract 与资产归属，不改写 Matter、Event、Semantic State、Artifact 与 Review 的 Runtime ontology。Sparse Work Harness、Compiled Work Expert、三层 activation、AOT / JIT composition 与 MoE 类比是从这些原则推导的实现解释，不新增一套 Kernel ontology。

结语

模型能力决定它能够提出什么变化；Agentic Runtime 决定它能否把变化执行出来；Work Contract 决定这些变化在什么条件下可以被理解、验证并取得现实效力；Matter Runtime 与 Context Compiler 进一步决定，在不断增长的工作知识与能力中，哪一小部分值得在此刻占用模型和人的 Attention。上游持续拓展 reasoning、knowledge 与 action 的能力边界，下游持续把这些能力编订为面向具体人、具体组织和具体后果的工作渗透面。两者之间的 Contract、Eval、Extension、Commitment 与长期资产，构成本文所说的 Schema Engineering。

高频工作可以被发布为 Compiled Work Expert，使普通用户在 preset 或少量 Expert routing 下消费已经验证的能力；长尾工作保留受限的 frontier composition，并通过 abstention、Human Review、Candidate revision、E2E revalidation 与版本发布回流到主路径。Expert 因而不是一个被扮演的人格，而是一项带 provenance、Authority、适用范围、失效条件与回滚路径的工作能力承诺。

在当前以有限 Attention 对 Context 生成 Output 的范式下，这可以被理解作为一种系统级的通用智能实现路径，而不是对单一模型已经达到 AGI 的宣称：通用能力不断扩大，工作世界不断变得可读、可提交、可撤销和可继续，codified knowledge、software-grounded execution 与 governed professional judgment 因而获得一条可能的连续学习路径。只有当系统进一步能够在外部反馈和 Authority 边界下改进自己的 Contract、Tool、Environment 与 learning procedure，才可以讨论 governed meta-improvement；该推论最终仍由各层自己的 Contract、held-out Eval、accepted work product、真实采用与证伪条件分别裁决。

PRACTICE SNAPSHOT · EDITION 2026-09-07 · 文本版本 9.6 · 阅读面候选 2026-09-11

Schema Engineering 的实践面

Sparse Work Harness、Matter Continuity 与 Governed Work Surface

摘要

Schema Engineering 的实践问题是：如何把通用模型和 Agentic Runtime 已经提供的推理、工具、会话、权限和界面能力，编订为可以恢复、审阅、提交和修订的工作能力。

实践不应从某个宿主、产品或成功案例出发。最小出发点是四个不受具体实现影响的约束：

- Runtime 只提供执行机制，不自动提供工作语义。
- 总存量与单次 Attention 必须分离，否则长期工作会退化为反复全量重读。
- 模型、人和多个执行实例只能提出候选变更，不能因为输出完整或意见一致就取得正式效力。
- 专业用户应直接处理工作对象、证据、版本、异议和决定，而不是学习 Session、Prompt、Tool routing 或 Subagent mechanics。

因此，当前实践形态可压缩为：通用 Runtime 之上放置宿主适配层；用 Sparse Work Harness 组合经治理的 Work Extension 与 Compiled Work Expert；用 Matter Sidecar 保存正式状态、未完义务和 Artifact 版本；用 Context Compiler 与 Human Work Surface 生成面向模型和人的不同投影；用 typed commitment boundary 决定什么可以生效。

一、文本责任与快照方法

本文记录一套当下可执行的实践构造，不编辑产品案例、个人工作法或新闻式趋势。每次增量按以下顺序处理：

```
observe an instance or failure
→ state the smallest mechanism it may expose
→ separate observation from inference
→ test against existing Canonical and Practice text
→ adjudicate: no change / index only / practice revision / canonical revision
→ record the decision and disconfirming condition
```

正文只接受同时满足三个条件的增量：

- 能够脱离原实例独立表达；
- 会改变实现、验证或失败边界；
- 现有文本无法用更小的解释吸收。

仅能说明某个产品已提供一项机制、某个团队自报一项收益、或某个人采用一种组织方式的材料，默认只进入 Index。正文中的结论必须可以不借助 Index 被理解和检验。

二、当前实践边界

2.1 Runtime seam 不是 Work Contract

可用的 Agentic Runtime 通常提供 model、tool、sandbox、session、event、storage、permission、question、trace 与 UI extension point。这些能力足以承载 Work Extension，但并不定义：

- 正在处理什么 Matter；
- 什么是当前有效状态；
- 哪些证据支持哪项主张；
- 什么算完成；
- 谁可以接受、批准、发布或承担后果；
- 哪个 Artifact 版本已经生效。

因此适配层只负责把宿主的通用机制绑定到稳定语义，不得成为领域规则的唯一存放地。等价的宿主接口替换不应改变专业含义；否则说明 interface accident 已经污染 Work Contract。

2.2 Capability surface 与 governance surface

执行层回答“此次运行能做什么”；治理层回答“什么可以取得什么效力”。

Capability surface
tools / files / network / models / sessions / UI
Governance surface
scope / evidence / completion / authority / review / commitment / recovery

两者可以实现在同一系统中，但不能用工具调用成功替代工作接受，也不能用宿主已经保存 Session 替代 Matter continuity。

实现应把高折旧的 model protocol、reasoning loop、streaming、generic tool calling 与内部 planner topology 留在可替换执行层，把低折旧的 Matter State、Evidence、Authority、side-effect boundary、validation、commitment、checkpoint 与 recovery 留在治理层。模型变强可以减少 imperative orchestration；它不改变哪些状态有权生效。Work Contract 应优先声明目标状态、不变量、充分 Evidence、禁止转换和 commitment condition，只在执行顺序本身承载专业语义时固定步骤。

2.3 完成条件必须外置

长程执行不能以当前 Agent 的自评作为唯一终止条件。最小 Completion Contract 需要把目标分解为可查询义务，为每项义务绑定 evidence procedure、validator 或 Reviewer，并把 continue、stop、return-for-revision 与 escalate 写成独立于 Agent 的 gate。

这只证明已明示的完成条件被检查，不证明专业正确性已经完全形式化。

2.4 修正必须经编订才能跨 Run 复用

一次修正只能说明某个结果在某个 Matter 和时点下被改变。要进入后续执行，必须区分：

artifact-local edit
matter-specific decision
institution convention
domain invariant
candidate evaluator or training signal

只有声明 provenance、scope、owner、review、version、rollback 和后续检验的修改，才能从 Candidate feedback 晋升为可复用规则。可读、可版本化的 instruction 能够成为 policy memory，但

只有绑定工作对象、状态、Evidence、Authority 与 outcome 后，才能成为 Work Contract 的一部分。

2.5 持久化、检索与正式状态分层

持久内容分别承担原始记录、当前状态和工作规则的责任。Memory 支持保存与召回；Current Semantic State 表达目前有效的事实、决定和开放义务；Schema 规定对象关系、状态含义与可接受的转换，并约束什么应跨 Run 保留。

稳定语义使不同执行实例能够比较同一工作中的变化。Candidate 应引用对象和依据版本，声明拟改变的内容及前置条件；提交时对当前状态重新验证。Schema 版本变化则先建立明确映射，再解释旧状态与新候选的关系。

Context Projection、Human Work Surface 与 Retrieval Index 是执行、裁决和定位所需的不同视图。它们共同引用正式状态与原始来源，不各自维护事实。Current Semantic State 能承载后续决策所需信息时，可以成为默认执行基底；未编入的 Observation、尚待发现的工作关系及作为研究对象的历史过程，通过 Raw History / Evidence 按需补充。

2.6 Context Mutation 是有效果的运行动作

删除、截断、摘要、压缩、折叠和重载会改变模型下一步能够注意、比较和引用的对象。因此 Context Mutation 默认只改变 Run-local working set，不改变 Repository、Current Semantic State 或 Active Artifact。

可恢复的 mutation 至少记录：

```
source object or span
+ previous projection
+ proposed mutation and reason
+ scope and lifetime
+ resulting projection
+ recovery path
+ downstream outcome
```

对 Context 操作给予更细的 credit 可以产生 Eval 或训练候选，但局部任务得分不能回答被保留的内容是否具有正式效力。

2.7 Multi-agent 是执行拓扑，Expert 是能力语义

多个 Agent instance 表示存在多个执行位置，它们可以并行、委派、竞争、审阅或汇总。Compiled Expert 则表示对某个任务族已声明、验证并收窄权限的能力配置。

```
Run Plan
= Compiled Expert Profile
+ current Matter State
+ role and stage
+ current Authority
+ Context Projection
```

多个 Agent 可以共用同一 Expert，一个流程也可以在不同阶段绑定不同 Expert。模型数量、Session 数量、终端数量或角色名称都不能代替 Expert 的 applicability、permission、Eval、fallback 和 release evidence。多个实例的意见一致也不等于独立证据或 Authority。

默认 coordination surface 应是带 identity、version、status、provenance 与写入规则的 governed artifact，而不是高频 Agent-to-Agent conversation。可独立拆分的执行在隔离 branch 中产生 bounded Candidate，再由明确的 comparison、merge、discard 或 Review transition 汇合；只有沟通本身会产生必要信息时才增加消息边。状态介导不能消除并发冲突，因此 shared writes 仍需 owner、base version、conflict handling 与 commitment gate。

2.8 文档是工作表面，不是自动成立的记忆

文档可以同时承载执行规格、来源与理由、Review surface、recovery index 和 accepted artifact。这些职责不得因共用同一文件形式而混同：

对象	默认地位	可否直接改变正式状态
Session / Tool Trace	Raw History / Evidence	否
自动摘要或索引	Retrieval aid / Candidate Projection	否
Contract / Decision Record	取决于 version 与 Authority	只在提交后
测试与审阅报告	Evidence / Review Record	不单独决定
已接受文档成果	Active Artifact	经 commitment 后可以

完整保留原始记录有助于回溯，生成索引有助于恢复；二者都不取代 Current Semantic State。

2.9 当前实践命题

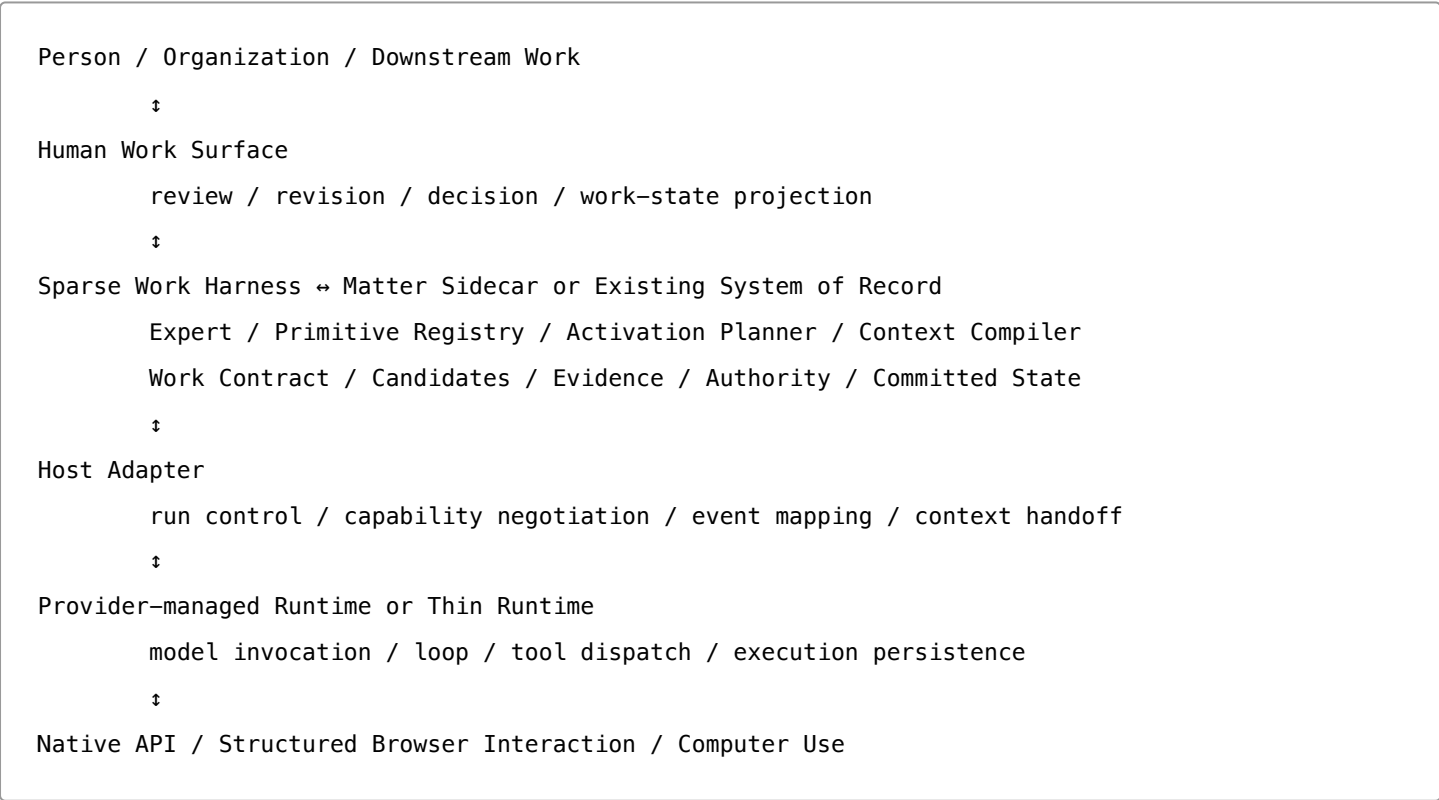
当下快照只承担以下可检验命题：

- 1. Runtime 与 Work semantics 可以通过适配层分离。
- 2. Completion 外置、feedback 编订、工作域隔离、持久资源和 Context 管理是互补机制，不是完整架构的替代品。
- 3. Sparse capability activation 与 sparse state projection 需要共同接受 omission、pollution、permission 和 accepted-work-product 检验。
- 4. Context Mutation、multi-agent topology 和 documentation workflow 都必须回到同一 Candidate / Committed 边界。
- 5. Context Projection、Human Work Surface 与 Retrieval Index 共享同一权威状态及带来源和效力标记的材料、候选，不各自维护事实源。
- 6. 上述命题是当前实现方向，并非已完成的普遍验证。

——

三、参考架构：Sparse Work Harness

3.1 分层



分层围绕工作语义与执行机制的变化速度组织。Work Contract、Matter State 和 Expert manifest 保存工作对象、证据、权限与接受条件；Host Adapter 将本次 Assignment 绑定到宿主能力，翻译运行控制与事件；Human Work Surface 把候选差异与决定后果交给人类。执行事件提供过程证据，正式状态仍由提交路径更新。

Runtime 兼容以当前 Contract 所需能力为准。Adapter 声明可用的控制时点、Context 管理、历史访问、恢复、授权拦截与效果核对能力，再选择原生实现、经验证的替代路径或收窄后的任务。该绑定覆盖执行时实际进入的材料和可产生的效果，包括宿主自行召回的历史；无法满足必要边界的任务不予执行。宿主可以保留自身的调度、压缩与并发机制；跨 Run 必须保留的工作事实则进入 Matter State。上层由此共享工作语义，下层按各自能力执行。

执行层可以消费 provider-managed Runtime，也可以封装私有模型所需的薄 Runtime。薄实现从模型调用、受限工具分发、事件记录与续行开始，依赖上层的状态、Context 和审批契约。自治长度、并发规模和失败恢复决定需要多少执行设施；短反馈循环与明确的人类裁决节点可以减少自主调度的负担。

API、结构化浏览器交互与 Computer Use 是访问外部系统的不同通道。选择取决于任务要求的授权、前置条件和结果核对能力，以及运行与维护成本。效果未知时先核对再续行；高频或需要更强控制的操作可编订为 Work Primitive。观察到的 trace 为这一编订提供材料，发布仍须经过权限约束与验证。

通用执行、存储、检索和界面组件可以复用成熟实现。SE 的实现工作集中于把它们接成同一条语义链：从正式状态生成有边界的工作视图，让执行交回可审阅的候选，再把被接受的变化写回唯一权威状态。

3.2 七个对象

对象	逻辑职责
Runtime Plugin	通用技术能力
Runtime Profile	一组技术运行环境
Work Primitive	最小可组合工作能力
Work Extension	面向宿主分发的领域语义和实现
Compiled Work Expert	对高频任务族已版本化、评测和收窄权限的 activation profile
Matter State	一项具体工作的持久化工作包，包含 Current Semantic State、版本、Evidence、决定与未完义务
Run Plan	Expert / primitives 与 Matter、role、stage 的一次绑定

Expert 不是人格化 Agent，也不是一组工具。它的最小单元是：

```
schema
+ tools
+ retrieval policy
+ validators and evaluators
+ permissions
+ Human Work Surface
+ transition and escalation rules
+ applicability and exclusions
+ E2E release evidence
```

3.3 五个承重组件

Sparse Work Harness 至少需要：

1. **Activation Planner**：根据 Matter type、role、stage 和 policy 选择 preset、Expert 或受限 primitive composition。
2. **Context Compiler**：从 Stable Contract、Current Semantic State、Resources 与 History 生成最小充分工作集。
3. **Expert / Primitive Registry**：保存版本、依赖、适用范围、权限、验证时间和退出路径。
4. **Matter binding**：将本次执行绑定到唯一工作对象、当前版本与开放义务。
5. **Commitment interface**：将 Candidate Output 路由给 Schema、Evidence、Completion、Authority 和 Review。

3.4 三层 activation

```
Preset binding
→ Expert routing
→ bounded Primitive composition
```

Matter type、role 和 stage 已能确定的工作使用 Preset；常规歧义只在少量 approved Experts 中选择；没有适用 Expert、跨域或低置信度时，才进入 primitive composition。最后一层默认 least-privilege 且 Candidate-only，不自动取得 approve、publish、external transmit 或 irreversible authority。

Activation lifetime 按 organization → role → matter → stage → run 分层。越稳定的左侧内容越适合预编译，越靠近当前 Run 的内容越适合动态投影。Sparse 不表示每个 turn 热替换全部能力。

3.5 Compiled Expert 的发布与折旧

```
frontier execution or expert intervention
→ Candidate Expert Profile
→ applicability / exclusions / provenance
→ Contract and dependency versions
→ held-out E2E and risk-stratified Review
→ release Authority
→ Committed Expert Version
→ outcome / disagreement / reversal / drift monitoring
→ revalidate / narrow / rollback / deprecate
```

成功 trace 不是 Expert。每个版本至少具有 owner、applicability、exclusions、provenance、dependency versions、risk-stratified E2E、freshness trigger、abstention、fallback、rollback 和 deprecation path。法规、来源、机构政策、工具、模型、任务分布或 Reviewer outcome 变化都可以触发重验证。

未覆盖事项必须安全退出：

```
unsupported case
→ abstain or request Review
→ Human expert or bounded composition
→ preserve Candidate-only boundary
→ capture intervention and outcome
→ Candidate revision
→ E2E revalidation and release
```

四、Matter Sidecar 与 Context Compiler

4.1 两种数据层

Matter 可以首先实现为现有 Session 和 system of record 之外的薄层，不必一开始就替换它们。

```
Canonical Semantic Layer
  committed events
  current semantic state
  active artifact references
  authority and review decisions
  open obligations
```

```
Rebuildable Index Layer
  full-text and vector indexes
  session and source locators
  graph and projection caches
  retrieval ranking
```

索引可以重建；哪个成果已被接受、哪项候选被驳回、谁作出何种裁决以及什么仍未完成，不能靠重新摘要 Transcript 猜回。语义层可以由事件溯源实现，也可以沿用既有 system of record 的事务状态、版本与提交记录；恢复读取指定的权威状态。

4.2 最小 Matter 声明

```
{
  "matter_id": "matter-001",
  "extension_id": "evidence-memo",
  "contract_version": "0.1.0",
  "status": "awaiting_review",
  "active_artifact": "memo:v3",
  "session_refs": ["session-a", "session-b"],
  "open_obligations": ["verify-source-7"],
  "pending_reviews": ["review-12"]
}
```

一个 Matter 可以关联多个 Session。删除 Session、替换模型或宿主升级不应同时删除 active Artifact、Current Semantic State 和未完义务。

Session 只是当前交互与 attention window，Run / executor 是一次可丢弃的执行尝试。恢复测试必须证明新 executor 能从 canonical state、active Artifact、Evidence、开放义务与 checkpoint 继续工作，而不是依赖旧 sandbox、旧模型的主观连续性或完整 transcript replay。

4.3 Store → Govern → Retrieve → Compile

```
Store
  Sources / Events / Artifacts / Raw History / Current Semantic State
→ Govern
  identity / status / version / provenance / authority / scope / expiry
→ Retrieve
  relevant objects within the authorized scope
→ Compile
  Context Projection / Human Work Surface / Retrieval Index / Run Plan
```

实现从本次 Assignment 的接收者、用途、任务阶段和状态版本出发确定可用范围，再定位完成当前义务所需的对象及关系。查询、索引和缓存可以分担这些操作，授权规则在数据进入对应 Runtime、工具或用户界面前执行。相关性排序只在允许使用的材料中决定优先次序。

Compiler 按任务组织工作集。路由可以使用事项类别与开放义务，执行需要相关事实、Evidence 与当前版本，Review 需要候选差异、判据和未解冲突。每种视图保留其判断依赖的限定与来源；不足以支持下一步时，补取授权材料或返回缺口。Context 预算决定表示规模，不能把尚未覆盖的义务编译成完成。

视图的来源版本、规则版本和适用范围随生成结果保存。对象变化或权限失效后，复用路径重新判断其适用性；依赖私有材料的摘要和工具参数沿用相应披露约束，不能因换了表示或操作通道而扩大接收范围。传递由执行边界实施控制，不能只交给 Prompt 中的保密要求。

本次执行产生带对象、版本和前置条件的 Candidate，由提交路径判断哪些内容可以写回正式状态。外部操作先通过授权与前置条件检查，完成后将结果证据与相应候选关联；结果未知时保留待核对状态。新 Run 从有效状态和开放义务续行。

Artifact State 与 Evidence State 分别回答“当前对象是什么”与“哪些行为已经验证、哪些主张仍无支持”。每轮工作同时携带可继续修改的 Artifact 与绑定具体版本的验证记录，原始材料和历史行动保持可检索。中间 reasoning 可以退出下一轮 Context，工作连续性由正式状态、Evidence 和恢复路径承接。

4.4 Context Mutation Preservation

在相同任务和资源下，对删除、摘要、压缩和重载分别检查：

- 限定、冲突、否定关系和来源坐标是否保留；
- retention policy 要求保留的 Raw Evidence 是否可恢复；
- superseded 或 rejected state 是否被重新引入；

- mutation 的 scope、lifetime、provenance 和 recovery path 是否可见；
- 更短 Context 是否改善 accepted-work-product，而不只改善 token 或局部得分。

五、Human Work Surface

5.1 三种投影

Current Semantic State + Sources / Evidence + Candidates

- └ Context Projection
 - | compact / normalized / executable
- └ Human Work Surface
 - | inspect / compare / trace / revise / decide
- └ Retrieval Index
 - | identify / version / locate / disclose later

人需要看到原文锚点、来源关系、版本差异、未解冲突、覆盖缺口与裁决后果。模型需要紧凑、规范化、符合当前权限和任务阶段的工作集。未来 Runtime 需要按 Matter、状态、版本、适用范围和 provenance 重新定位对象。三者以同一权威状态解释来源与候选，保留各自的版本和效力，不必共用同一表示。

5.2 表示原语

原语	主要任务
List / Table	扫描、筛选、分类、逐项裁决
Tree / Outline	层级、范围与覆盖
Anchored Document	返回原文并局部校验
Graph	多对象与多关系
Timeline	先后、生效、变化与冲突
State Graph	步骤、责任、阻塞与转换
Matrix	二维覆盖、对应与冲突

原语	主要任务
Diff / Lineage	版本、候选、覆盖与撤回
Queue / Coverage	待处理对象、缺口与完成度

这些是 renderer grammar，不是通用 ontology。哪些节点、关系、字段、动作和权限合法，仍由 Work Contract 决定。

5.3 Review grammar

Review Item

= Target

+ Anchors

+ Current Semantic State and Candidate Delta

+ Judgment Dimensions

+ Evidence

+ Automated Checks

+ Uncertainty and Open Questions

+ Commit Consequence and Reversibility

+ Decision

+ Authority

+ State Consequence

Review 围绕有状态后果的 decision unit 分批编译。默认表面只放形成独立判断所需的最小充分对象；Raw Trace、旧版本和补充 Evidence 保持可检索并按需展开。逐项 tool-call confirmation 和事后 chronology replay 都不能替代对 Candidate delta 的专业裁决。

主交互应使 Current → Proposed → Reviewed → Committed 的状态差异可见，并同时呈现 prior state、Evidence、uncertainty、Authority、side effect、reversibility 与 recovery point。模型的内部计划、Agent avatar、Agent 间聊天和 decorative workflow graph 只有在帮助诊断或裁决时才渐进披露；用户首先审阅工作变化及其后果，而不是观看执行表演。

可复用的动作族包括 accept、reject、revise、request further work、request evidence、qualify、defer、waive、escalate、approve、publish、supersede 和 withdraw。动作名称不产生复用性；只有以下链路可复用：

decision

→ authority requirement

- validation
- committed event type
- semantic-state transition
- artifact-version effect
- next-context effect

5.4 文档与界面的晋升边界

HTML、Markdown、表格或图形只是 Human Work Surface 的表示。界面中的按钮、评论或勾选只有在生成 typed Candidate Decision，并经 Authority 与 validation 写入 Committed Event 时，才更新 Current Semantic State 或 active Artifact reference。

- Model proposes Candidate
 - deterministic Renderer builds surface
 - Human reviews or revises
 - typed Candidate Decision
 - Authority and validation
 - Committed Event
 - State and active Artifact update

六、Work Extension 与隐性知识编订

6.1 产品吸收 Agent 复杂度

Runtime 对象	专业用户面对的工作语义
Session	Matter 中的一次执行
Memory	可召回信息、来源与适用范围
Tool permission	某项资源或行动权限
Agent / Subagent	执行位置；Operator 绑定运行义务，Lane 表达并行关系
Output	Candidate Change / Artifact Version
Approval	具有状态后果的专业裁决
Plugin / Skill	Work Extension / Primitive / Expert dependency

用户不必看不见这些技术对象，但默认交互应以专业对象和工作决定为中心。

6.2 从观察到 Contract

```
expert demonstration
→ intervention and correction capture
→ separate invariant / institution / matter / habit / judgment
→ compile Contract / validator / surface / escalation
→ E2E without the original author
→ production review and revision
```

隐性知识首先表现为对任务、来源、Context、工具、失败、重试、完成和交付的手动控制。编订只抽取跨时间必须保留、会改变后续行动、支持 Review 或能被验证的部分，不序列化专家的全部认知过程。

6.3 分流

内容	去向
稳定义务、来源层级、必查项	Domain Contract / Evaluator
机构审批、格式、风险偏好	Versioned institution configuration
特定客户、Matter 和时点要求	Matter context
个人表达、捷径与工具偏好	User preference，默认不推广
风险接受、策略与重大例外	Human Review / Accountable Decision

出现频率不能把个人习惯提升为机构规则，也不能把机构规则写成领域真理。

6.4 Search Contract

Review Contract 限定什么可以被接受；Search Contract 限定 Agent 应进入哪个 solution space。它可以规定先查哪类来源、什么冲突必须保留、哪些捷径禁止、何时扩大搜索、何时收敛、何时放弃并升级。

Search Contract 不能把候选空间收窄到只能复制旧答案。它的作用是保留专业搜索中稳定且可审阅的约束，并为超出声明边界的问题保留 frontier path。

6.5 Runtime 外的评测与改进

Runtime 内的检查决定本次候选能否继续或提交；Runtime 外的 Eval 比较不同版本如何完成同类工作，为下一次修订提供证据。两者可以共用执行设施与判据，但评测分数不授予工作效力。离线评测的产物是关于系统行为的证据与修订候选，正式成果和新版本仍分别经过自己的接受与发布边界。

产品问题与工作要求

→ 可解释的局部信号

→ 跨任务、跨版本的评测

→ 沿 Evidence / State / Projection / Artifact 定位失败

→ Contract / Evaluator / Harness / Context / Tool / Model 修订候选

→ 重新评测与版本裁决

→ 后续工作中的反馈

局部信号把笼统的质量感受变成可以追查的问题。Work Contract 则为追查提供坐标：哪些义务尚未处理，来源支持什么，哪些关系进入了持久状态，又在何处未被披露或被错误使用。Schema 使缺口能够被指认，不保证未记录的信息已经被发现，也不把可观测位置自动变成失败原因。

符合既有 Contract 与 Contract 足以表达工作要求，是两个不同判断。前者检查执行与成果，后者允许真实工作中的遗漏、分歧和后果反过来修订判据。Eval 因而既约束系统如何执行，也帮助团队判断系统原本要求它做的事是否恰当。

6.6 从产品信号到训练候选

codified artifacts

→ software-grounded interaction trajectories

→ governed situated judgment trajectories

第三类信号至少需要 Matter、State、Evidence、Candidate / Committed、Review、Authority、Outcome 与 later reversal。这些对象首先服务运行、恢复和 Eval；只有在 rights、failure attribution、held-out Work Eval 和外部现实反馈成立时，才成为 selective post-training 候选。产品成立不以训练飞轮成立为前提。

七、理念认证

7.1 验证目标

验证 Work Contract 能否在不修改宿主核心的条件下，被编译为可加载能力、Matter continuity、Context Projection、Human Work Surface、Authority boundary 与 typed commitment protocol。

理念认证只需一个可插拔或可适配的 Runtime、一个 Matter repository、一组 Work Extension、一个 Context Compiler、一个 Human Work Surface renderer 和一条 commit protocol。不必先完成多租户平台、市场、训练管线或全领域 ontology。

7.2 共享场景

至少使用三类责任结构不同的任务：

- 结构化对象的逐项裁决；
- 主张、来源、支持、反驳与限定的校勘；
- finding、control、evidence、owner 与 remediation 的整改裁决。

再增加一个弱 commitment 的对照场景，检查 Human Work Surface 是否在不依赖完整 Matter governance 时仍能改善 review bandwidth。

7.3 必要测试

Architecture

- **No Core Patch**：不修改宿主核心也能加载、撤销和恢复 Extension。
- **Upstream Upgrade**：等价接口替换只修改 adapter，不改变 Work semantics。
- **Plugin Reload**：重载后 Matter 与 active Artifact 不丢失。
- **AOT / JIT Equivalence**：预编译与运行时组合在相同边界下产生等价语义。

Continuity and governance

- **Session Replacement**：替换 Session 或模型后从 governed state 恢复。
- **Candidate / Committed Isolation**：未通过适用提交检查的 output 不改变 Current Semantic State。
- **Authority Failure**：越权裁决被拒绝并保留候选记录。
- **Retrieval / Canonical Separation**：检索命中的旧陈述不被自动恢复为当前状态。

- **Documentation Promotion**: Trace、Index、Candidate Decision、Active Contract 和 Accepted Artifact 具有不同写入路径。
- **Context Mutation Preservation**: 限定、冲突、否定、来源和恢复路径不因压缩丢失。
- **State Sufficiency / History Disclosure**: 对照 State-only、State + on-demand History 与 append-only Transcript，检查未被及时编入的 Observation、动态 Schema 和 trajectory-defined task 是否需要披露历史。
- **Patch Preservation**: State patch 采用 merge semantics 而不是无声全量覆写；旧字段删除、类型改变与非法 patch 必须显式验证、拒绝或回滚。
- **Failure Containment / Trusted Recovery**: 向 Observation、Proposition、Candidate 与 validation 阶段分别注入错误，检查它是否在 commit 前被发现或隔离，并能否从最近 trusted checkpoint 由新 executor 恢复。

Sparse activation and Expert release

- **Sparse Capacity Scaling**: per-run Context 和 Tool surface 不随总存量线性增长。
- **Omission / Pollution**: 必要约束不被遗漏，旧版本、其他 Matter 和越权 capability 不被引入。
- **Least Privilege**: 激活能力只获得当前 role 与 stage 所需权限。
- **Three-tier Priority**: Preset 先于 Expert routing, Expert 先于 primitive composition。
- **Compiled Expert E2E**: 评测整个 activation profile，而不是单个 tool 或 prompt。
- **Staleness / Revalidation**: 依赖或现实变化能触发缩窄、暂停、回滚或弃用。
- **Graceful Escape**: 未覆盖事项进入 Candidate-only frontier path，不被快速路径掩盖。

Review and evaluation

- **Completion Independence**: 完成条件不由 Agent 临时降低。
- **UI Representation Equivalence**: 不同 renderer 对同一裁决产生相同状态后果。
- **Review Bandwidth**: 在成果质量不降低时，专家 Review 时间或恢复成本下降。
- **Review Sufficiency**: 去除完整 execution trace 后，结构化 Review packet 仍使 Reviewer 发现关键错误、请求必要证据并形成可解释的独立判断。
- **Projection Consistency**: 基于同一状态与来源快照重建三种视图，检查候选身份、版本与效力一致，且不同披露范围不被误判为事实冲突。
- **Evaluator Lifecycle**: criterion、rubric、reason 与 deployment version 可追踪；错误归因不会因 label 碰巧正确而进入下游 revision 或训练信号。
- **Correlated Review Failure**: 注入共享错误前提、共同缺失来源和相同 Evaluator 偏差，检查多实例共识是否被误当独立验证。

- **State-mediated Coordination**: 比较 governed branch / artifact 汇合与高频 Agent messaging, 在相同任务下检查冲突、重复工作、错误传播、Context 成本和 accepted outcome。
- **Accepted Work Product**: 结果由具备 Authority 的 Reviewer 在预定节点接受, 可进入下游且无需实质性修改。

八、非目标与失败模式

8.1 非目标

本快照不主张：

- 某个宿主是唯一或永久实现；
- 把所有工作完全形式化；
- 把专业判断消除或转移给多个 Agent 投票；
- 把文档、索引、记忆或检索系统等同于 Current Semantic State；
- 让普通用户在全量工具和 primitives 中自行编排；
- 以训练、市场规模或横向平台作为产品价值成立的前提。

8.2 失败模式

失败	判定
Sidecar split-brain	Matter State 与 system of record 同时声称权威，却无明确 owner 和 reconciliation
Retrieval mistaken for governance	语义相近的旧内容被当作当前有效状态
Context optimization destroys evidence	压缩降低 token，同时丢失限定、否定、冲突或坐标
Surface theater	界面更结构化，动作却没有 Authority 和状态后果
Formal HITL	人被放进 loop，却没有足够 Evidence、时间或状态差异形成独立判断
Projection split-brain	Context Projection、Human Work Surface 与 Retrieval Index 对同一 Matter 持有不同事实或版本
Evaluator drift without governance	rubric 或 reason 改变生产行为，却没有版本、Review、monitoring 与 rollback
Correlated consensus	多个实例共享错误前提和 Evaluator，共识被误当独立校验

失败	判定
Chatter topology	Agent 间消息成为事实源，缺少版本、冲突处理与 commitment boundary
Executor-bound work	新 Session 或 executor 无法从 governed Matter State 恢复，必须依赖旧 sandbox 或 transcript
Imperative ossification	Runtime 已能安全选择执行路径，Contract 仍冻结不必要的 planner / worker / reviewer 步骤
Agent theater	UI 强调内部计划、头像和聊天，却弱化 Candidate delta、Evidence、Authority 与 commit 后果
Dense activation regression	能力与数据总量增长时，每次 Run 的 Context 和权限表面同比增长
Expert ossification	旧来源、规则、工具或偏好因快速路径而持续生效
False promotion	成功 trace 在无适用范围、反例、Authority 和 held-out E2E 时晋升为 Expert
Annotation displacement	结构化主要依赖专家额外填表，成本与使用量同比增长
Interface overfit	等价宿主变更即使专业语义或成果质量变化
Structure without outcome	结构增多，accepted-work-product、Review、恢复、缺口发现和成本均无改善

九、结论

当下实践的最小结构是：

```
thin shared runtime
+ host adapter
+ Preset / Expert / Primitive activation
+ Matter State
+ Context Compiler
+ Human Work Surface
+ typed commitment boundary
+ E2E and lifecycle governance
```

这一结构把总能力、总知识与单次 Attention 分离，把执行拓扑与 Expert 语义分离，把原始记录、索引、当前状态与正式成果分离，并使 Context 的每次重写和 Output 的每次写回都可被检查。

是否成立不由宿主能力、界面完整度、Agent 数量、社区采用或库内 Expert 数量证明。裁决条件是：原作者离场后其他合格用户能否产生可采用成果，替换 Session、模型或宿主后工作能否恢复，候选与

正式状态是否始终分离，专家 Review 或恢复成本是否下降，以及新形成的信号是否在 rights、归因和 held-out Work Eval 下具有增量价值。

PRACTICE INDEX · EDITION 2026-09-07 · 文本版本 9.6 · 阅读面候选 2026-09-11

Schema Engineering · Practice Index

引用、检验与增量修订记录

一、用途

Practice Index 保存会折旧的实例、来源和裁决记录。Canonical 定义 Kernel，Practice 给出当下可执行的泛化快照，Index 回答：

- 某项局部观察来自哪里；
- 它直接支持什么，不支持什么；
- 应如何复现、消融或证伪；
- 经过讨论后，是否修订 Canonical 或 Practice；
- 哪一版文本吸收了何种最小增量。

Index 不是案例汇编，也不为正文提供必要前提。删除 Index 后，Canonical 和 Practice 仍应各自完整。

二、登记格式

每个条目至少包含：

```
id / observed_at / source_class
source / version or access date
observable mechanism
smallest supported claim
unsupported extrapolations
verification or falsification
text disposition
status
```

状态只使用：

状态	含义
observed	来源和机制已登记，未形成文本裁决
abstracted	已抽取可检验的最小机制
indexed	只保留于 Index，不修订正文
practice-adopted	已以泛化形式写入 Practice
canonical-adopted	已证明现有 Kernel 边界需修订
rejected	观察不足以支持候选命题
superseded	条目的产品或解释已被后续证据覆盖

三、局部命题索引

PI-01 · Runtime composability

- 观察：插件原生 Runtime 公开 model、tool、session、event、permission、question、storage 与 UI seam；可组合系统还可以提供 effect tracking、dependency resolution、configuration reconciliation 与可逆生命周期。^{1 2 3 4 5 6}
- 最小命题：Work Extension 可以在不把领域语义写入宿主核心的条件下被封装、加载、撤销和投影。
- 不支持：Work Contract 正确；跨宿主零成本迁移；热替换后成果质量不退化。
- 检验：No Core Patch、Upstream Upgrade、Plugin Reload、AOT / JIT Equivalence。
- 处置：Practice §2.1、3.1 以宿主无关表述采用；具体 API 仅留本条。
- 状态：practice-adopted。

PI-02 · External completion governance

- 观察：在所选长程软件重建任务中，供应方先构造 executable standard of completion，再以它约束实现与 continue / stop decision；报告结果来自自选任务和自建 system / benchmark。⁷
- 最小命题：独立于当前 Agent 的 completion inventory、evidence procedure 与 validation gate 可以改变长程执行结果。

- 不支持：一般专业正确性；完整 Work Contract；跨领域因果；独立复现。
- 检验：固定模型与任务，消融 external completion inventory、evidence gathering 和 stopping gate，同时报告负面结果。
- 处置：Practice §2.3 采用泛化机制；数字与供应方叙事留在 Index。
- 状态：practice-adopted。

PI-03 · Feedback compilation

- 观察：实践系统将相对稳定的 procedural Skill 与 inference-time Memory 区分，把 domain instructions 保存为文件，在工作位置捕获 Human feedback，由 improver 提出小幅 diff，再经普通 PR / review / merge 流程进入后续执行；材料同时提醒 feedback 可能错误，需筛选其 Authority 与适用范围。⁸
- 最小命题：Human correction 可以被低摩擦捕获、审阅、版本化并在模型权重之外跨 Run 复用；可复用 procedural rule 的变更应走独立于自动 Memory 写入的 commitment path。
- 不支持：Skill 已成为完整 governance substrate；accepted-work-product 因果；post-training 收益；局部偏好可泛化；自动 improver 已能判断所有 feedback 的 Authority。
- 检验：检查 feedback Authority、scope、version、rollback、Reviewer disagreement 与后续 outcome。
- 处置：Practice §2.4、6.3 采用；具体产品流程留在 Index。
- 状态：practice-adopted。

PI-04 · Project-bounded recall

- 观察：Frontier lab 的官方产品文档描述工作域内的记忆隔离、旧对话检索与域外上下文排除。⁹
- 最小命题：持久内容可以按工作域隔离，历史可以保留检索权而不必全量占用当前 Context。
- 不支持：Project 等于 accountable Matter；retrieval 等于 Current Semantic State；Review、Authority 与 supersession 已解决。
- 检验：Retrieval / Canonical Separation、Session Replacement、stale-state reintroduction。
- 处置：Practice §2.5 采用机制区分；产品名与版本留在 Index。
- 状态：practice-adopted。

PI-05 · Managed resource surface

- 观察：Frontier lab 的官方文档公开 memory synthesis、project scope、persistent file references、source reuse 与 source disclosure 等产品表面。¹⁰

- 最小命题：Context 可以从一次 message payload 转向拥有 identity、scope、lifecycle 和 reference 的可管理资源；storage 与 current attention 可分离。
- 不支持：不可观察的 backend 已采用 event / object ontology；UI 表明 Govern layer 已成立；资源不会在模型输入前被 flatten。
- 检验：查验 identity、scope、version、source return、deletion、expiry 与 Context disclosure，不从 UI 反推内部实现。
- 处置：Practice §2.5、4.3 采用 storage-attention separation；产品表面留在 Index。
- 状态：practice-adopted。

PI-06 · Execution-to-judgment migration

- 观察：专家文章指出，即使 Agent 降低局部实现成本，人仍需理解 data architecture、system architecture、security、reliability、production operation 与 lifecycle trade-off。¹¹
- 最小命题：execution 变便宜不会自动消除 architecture、verification 和 lifecycle judgment。
- 不支持：该作者提出了 Schema Engineering；人需要理解的内容必然应全部成为系统状态；软件工程与专业工作的成熟度对称。
- 检验：在执行成本下降后，测量决策、验证、责任和生命周期错误是否成为更主要的成果失败来源。
- 处置：只作为 Practice 的结构假设背景，不在正文点名。
- 状态：indexed。

PI-07 · Proactive context management

- 观察：Frontier 研究把 context editing 表达为 planning、structured memory、offloading、partial rollout 与 snapshot-level credit 的一部分，并公开 inference、evaluation 与 training 实现。^{12 13}
- 最小命题：Context 的删除、摘要、压缩与重载是改变后续行为的 action，可以被快照、回放和给予局部 credit。
- 不支持：已建立 governed Matter memory；被保留的内容具有正式效力；特定长上下文任务的结果具有跨领域外部效用。
- 检验：Context Mutation Preservation；对 token、局部得分和 accepted-work-product 分别报告；保留 Raw Evidence recovery、scope 与 lifetime。
- 处置：Canonical §5.4 作最小边界澄清，Practice §2.6、4.4 写入实现与测试；不新增 ontology 或原则编号。
- 状态：canonical-adopted / practice-adopted。

PI-08 · Multi-agent work topology

- 观察：个人实践描述多种模型或 TUI 共享 harness、skills 与工作约束，并用 planner、worker、critic 等位置隔离 Context；另一个人实践强调并发实例需要状态、优先级、隔离环境与调度。^{14 15}
- 最小命题：执行拓扑、能力配置、工作责任与最终 Authority 是不同对象。
- 不支持：角色名称等于 Expert；Agent 数量提高必然改善质量；个人 workflow 已证明一般架构或收益幅度。
- 检验：Runtime Profile Separation、Correlated Review Failure、Authority Failure。
- 处置：Canonical §4.3–4.4 已充分表达责任区分，不修订；Practice §2.7 增加 topology / Expert 分离及相关测试。
- 状态：practice-adopted。

PI-09 · Documentation as work surface

- 观察：社区实践把文档驱动、对抗审查、测试、复杂度约束、完整记录与工作索引并置。¹⁶
- 最小命题：文档可以兼任执行规格、审阅、恢复和交付表面；同一文件形式不消除对象的制度差异。
- 不支持：Session log 等于 Current Semantic State；自动索引可以改变正式状态；Agent 共识等于验收；个人报告的成本比例具有一般性。
- 检验：Documentation Promotion Boundary、Session Replacement、Candidate / Committed Isolation。
- 处置：Practice §2.8、5.4 采用；Canonical 已有 History / State / Context / Artifact 边界，不增加新对象。
- 状态：practice-adopted。

PI-10 · Structured state as the execution substrate

- 观察：Google / Purdue 的 *SKILL.state: Scalable Long-Horizon Agent Skills* 将长程 procedural execution 表达为 immutable procedure + current structured state + latest observation，由模型提出 State Patch 与 action，再由 deterministic runtime 验证、merge 和执行。论文 v2 于 2026-08-28 修订，arXiv 页标注 accepted at EMNLP。¹⁷
- 最小命题：在 Current Semantic State 能够作为后续执行充分统计量的长程 procedural task 中，canonical structured state 比 append-only conversational history 更适合作为默认 execution substrate；它可以同时降低累计 token 复杂度和 history contamination。

- 实验边界：论文在 synthetic SkillExecBench、InterCode CTF 与 Sierra τ -Bench 上比较 ReAct-style Prompt、Summary Memory、State + History 与 State-only Runtime。Warehouse T=100 的 budget-matched 对照中，sliding window、capped summary、LLMLingua 与 structured state 的 score 分别为 0.18、0.52、0.22 与 0.94；structured state 的累计 token 为 65,408。T=200 时 structured state 报告 0.94 / 约 122k tokens。在 T=50 每轮 50 个 distractor events 时，Prompt 与 structured state 的 score 分别为 0.53 与 0.98。¹⁷
- 不支持：History 可以从存储删除；固定 Schema 适用所有专业工作；论文已建立 Evidence、Authority、Review、Artifact version 或 Matter governance；单 Agent procedural benchmark 结果可直接外推到审计、法律、投资或多写者工作。
- 论文自述限制：State 必须是 future execution 的 sufficient statistic。该假设在三类场景失效：Schema 需在执行中发现；旧 Observation 的重要性未被及时识别和 commit；historical trajectory 本身是 auditing、debugging provenance 或解释过去行动的工作对象。当前实现也只验证单 Agent；多 Agent 共享 State 还需要 deterministic concurrent-write conflict resolution。¹⁷
- 失败归因：论文对一个较小 open-weight model 的 State 失败分类为 premature overwrite / deletion 68%、schema comprehension / type coercion 20% 和 JSON syntax 12%。这只能说明失败在结构化 transition 后更可归因；不证明 reasoning error 已被消除。¹⁷
- 检验：State Sufficiency / History Disclosure、Patch Preservation、Context Omission / Pollution、Session Replacement、Candidate / Committed Isolation。消融应同时比较 accuracy / accepted outcome、token、latency、recovery、source preservation 和旧 Observation 召回。
- 处置：Canonical §5 增加一句带 sufficient-state 条件的规范性收敛，不新增原则；Practice §2.5、4.3、7.3 增加 Memory / State / Schema 区分、State-first 运行链与边界测试。论文名、数字和限制仅留 Index。
- 状态：canonical-adopted / practice-adopted（局部实证，待专业工作场景复现）。

PI-11 · Fresh-context continuation as a runtime hypothesis

- 观察：用户保存的 2026-09-03 Codex 实现讨论截图描述一个候选 new_context 机制：Agent 可以结束已污染的 working window，在不生成 compaction summary 的情况下进入 fresh initial context，并通过 history / notes 类工具按需取回旧信息。当前公开 OpenAI 文档能够确认长程 Runtime 对 compaction、state continuity、tool orchestration 与 autonomy boundary 的支持，但未能独立确认该截图中的具体接口、合并状态或发布语义。¹⁸
- 最小命题：将 history 作为可查询数据源、将 current Context 作为可销毁 working set，是一条不同于“持续把旧 attention 压入新 attention”的 Runtime 路径；两者可以并存。
- 不支持：Codex 已停止 compaction；new_context 已发布或会按截图落地；notes / history 已构成 governed Matter State；fresh window 一定提高 accepted-work-product。

- 检验：Fresh-context / Compaction 对照；Session Replacement；State Sufficiency / History Disclosure；比较关键约束遗漏、旧状态重引入、token、latency、recovery 与 accepted outcome。
- 处置：现有 Canonical §5 和 Practice §2.5–2.6 已能表达，不因未独立核验的产品接口修订正文；仅作为 Harness 内化 context / state management 的候选趋势保留。
- 状态：indexed（产品实现观察待公开来源或可重复实验确认）。

PI-12 · Human attention as a review constraint

- 观察：Mitchell、Ghosh 与 Passi 的 position paper 认为，把人放入 Agent loop 不自动构成有效监督；Agent 的速度、步骤与长期自动化可能削弱 situational awareness、形成 approval fatigue，并使监督所需技能退化。论文提出 strategic friction、bounded autonomy、batch review、automated pre-checks、monitoring 与组织协议等设计方向。¹⁹
- 最小命题：Human Review 有独立的 attention 与 cognition constraint；Review Contract 必须规定人看到什么、何时看到、以何种 decision unit 和 Evidence 颗粒度看到，而不能只规定“存在 approval step”。
- 不支持：论文实证了某一种 Review UI；所有 batch review 都优于逐项 review；review duration、override 或 canary 单项指标能够证明判断质量；HITL 可以消除能力差距或责任风险。
- 检验：固定任务、Agent 与 Authority，比较 raw chronology、tool-call approval 与 structured decision packet；测 critical error detection、evidence-seeking、Review time、override、later reversal、canary 与 accepted-work-product。
- 处置：Canonical §6.6 增加 Review 的认知充分性与 decision-unit projection；Practice §5.1、5.3、7.3 增加 Human attention、Review Sufficiency 与监测边界；不新增 Contract 类型或原则编号。
- 状态：canonical-adopted / practice-adopted（设计约束；具体 UI 收益待实证）。

PI-13 · Software engineering becomes work-shaped beyond the task boundary

- 观察：Harness-of-Harness 把多日 autonomous software development 组织为 planning–development–independent QA 的迭代循环。Runtime 保存 software Artifact State 与 Evidence State，为不同 role 冻结输入、限制读写权限、要求 structured output、按 concise index 渐进披露持久 Artifact，并把 Evidence 绑定到 read-only candidate version。²⁰
- 最小命题：当 coding autonomy 跨出一次 bounded task，持续推进需要长期 Specification、Artifact continuity、Evidence continuity、bounded objective、role-specific Authority 与独立 Acceptance；Artifact 与 Evidence 谁也不能替代谁。
- 实验边界：论文在三个 software benchmark、三组 harness–model pair 和一个 70+ iteration 游戏开发案例中报告增益；多日案例来自单一项目和作者系统，benchmark 的 verifier 与软件 substrate

也比多数专业工作更可执行、可回放和可版本化。²⁰

- 不支持：Planner / Developer / QA 是通用最佳拓扑；QA 报告已经形成外部 commitment gate；固定 Specification 适合规则会演化的工作；software tests 可以替代 broader professional judgment；结果可直接外推到法律、投资或机构审批。
- 检验：固定模型、Harness 与任务，消融 Artifact / Evidence dual state、progressive disclosure、role Authority、frozen candidate 和 independent Acceptance；分别测 regression、重复工作、unsupported completion、token、recovery 与 accepted artifact。
- 处置：Canonical §8.5 增加 coding 跨 task boundary 后的 work-shaped runtime 与外推边界；Practice §4.3 增加 Artifact / Evidence dual continuity；具体角色、模型、benchmark 与数字仅留 Index。
- 状态：canonical-adopted / practice-adopted（软件域局部实证，跨专业域待验证）。

PI-14 · Evaluator criteria have a governed lifecycle

- 观察：Netflix 的 production case study 把 LLM judge 组织为 Birth、Training、Deployment、Monitoring 四阶段 lifecycle：专家定义 must-have criteria、labeling guideline、边界样本与 rationale；rubric tuning 同时处理 label error 和“同为 fail 但 reason 不一致”；生产中 judge 既 gate explanation，又把 reason 送回 bounded revision；每周 Human review 监测 drift 和 rubric gap，新 rubric 经 manual review gate 后才能部署，旧版保留 rollback。²¹
- 最小命题：当 criterion、verdict 或 reason 会改变生产 lifecycle 时，Evaluator 与 rubric 本身是需要 owner、version、monitoring、Review、deployment gate 和 rollback 的 governed artifacts；label agreement 不能覆盖具有下游后果的 attribution error。
- 实验边界：案例只覆盖一个 recommendation-explanation family 和 mobile surface；部分 criterion 与模型细节未公开；drift-triggered automatic retuning 尚未在生产触发；meta-judge 与 primary judge 使用同一 base-model family，可能存在相关错误。²¹
- 不支持：LLM-as-a-Judge 可以替代 Accountable Reviewer；该 rubric lifecycle 已验证一般专业 Work Contract；单一 judge 的 gate 与 critique 复用在所有领域都更安全；线上业务 lift 证明每项 rubric revision 的因果。
- 检验：Criterion / Evaluator Version Binding；Right-label / Wrong-reason cases；Drift and Rubric-gap Detection；Candidate Rubric / Deployed Rubric Isolation；Rollback；下游 revision contamination。
- 处置：Canonical §11.2 增加 Evaluator / criterion lifecycle 与 reason 的执行后果；Practice §7.3 增加 Evaluator Lifecycle 测试；数字、产品规模与限制仅留 Index。
- 状态：canonical-adopted / practice-adopted（生产案例，外部效度受限）。

PI-15 · Repository-governed handoff continuation

- 观察：在 2026-09-04 本轮续行中，新 Run 先读取仓库 README、CONTRIBUTING、papers/README、CHANGELOG、近期 commits、三份 source 的责任边界与 build / validation 规则，再按需展开三段 chat attachment 和原始论文；它识别 PI-10 已完成裁决而未重复引入，并把未公开的 Codex 接口观察、论文证据与本文综合推论分开处置。
 - 最小命题：显式 repository state、revision protocol、source / generated artifact 分工与可查询 history 可以让一次新的执行在不把全部旧聊天当作 Current Semantic State 的条件下继续修订。
 - 不支持：单次自观察证明该方法优于 compaction；Agent 未获得隐含上下文；结果已经由独立 Reviewer 接受；相同协议能在其他模型、仓库或专业领域稳定复现。
 - 检验：用同一修订任务比较 repository-governed handoff、仅提供 chat summary 与无治理文件三组 fresh Run；检查重复论断、来源错配、Canonical 过度修订、版本一致性、验证通过率、Review time 与 later reversal。
 - 处置：只登记为本轮方法自观察和 V-13 起点，不作为 Canonical 或 Practice 成立的外部证据。
 - 状态：observed（单次自观察，待独立对照与人工 Review）。
-

PI-16 · Product signals motivate offline attribution

- 日期 / 来源类别：2026-09-05 访问；作者发布、由从业者补充的实践分享。用户提供的微信链接无法直接读取；已核对同作者 Substack 全文，聊天转述仅作查找线索。²²
- 观察：文章以事实点数量和隐喻率描述 Research Bench 的局部指标；报告 GPT 搜索调用较多但最终信息量较低，检查中间笔记后修改写法，并自报信息量改善。文中也承认指标需要线上结果持续校验。
- 最小支持命题：输出信号可以引导 trace inspection，并产生针对中间表示和 Harness 的修订假说。这是产品诊断实践的局部观察。
- 不支持的外推：Search 次数不等于接触到的相关信息量，不能排除检索质量或覆盖不足；干预自报没有给出足以独立复现的对照与效应区间，不能认定唯一因果；信息点数量不自动度量真实性、相关性或义务覆盖；隐喻率不等于易读性。文章未公开的架构不能据此判定存在或缺失；本文不采用其中关于隐喻必然造成信息损失的数学解释。
- 检验：固定来源、任务和资源预算，比较自由摘要、原子笔记与按既有 Work Contract 记录证据关系的表示；以独立核对的任务义务和来源支持为参照，分别测抽取遗漏、持久化遗漏、Projection 遗漏与最终使用错误，同时记录相关性、重复、未获支持的主张、Review 时间和成果接受。端到端检索另设对照；不以同一个抽取器生成并裁决全部参照。

- 讨论与裁决：具体案例只入 Index；结合 PI-14、PI-17 与既有 P19、P21，把诊断、受控比较和回归发布合成为 Practice 的离线 Eval 实践。Schema 提供可检查的位置和关系，不自动提供完整观测、正确标签或因果识别。
- 正文处置：Practice §6.5 补全 Runtime 外的评测闭环；无需新 ontology、Contract 类型或原则。
- 状态：practice-adopted（机制设计已采用；收益与因果待验证）。

PI-17 · Evaluation harness and agent harness have distinct responsibilities

- 日期 / 来源类别：2026-09-05 访问；Anthropic 官方工程实践总结，2026-01-09 发布。²³
 - 观察：文章区分 task、trial、grader、trace、outcome 和 evaluation harness；区分能力探索与回归检查，并要求校准 grader、读取轨迹、隔离试验环境、重复运行及结合生产观察。
 - 最小支持命题：执行 Agent 的 Harness 与组织试验、记录、评分和比较的评测设施可以分担不同职责；一次试验的最终外部状态不能只从 Agent 自述推断。
 - 不支持的外推：工程建议不是通用收益的受控实验；离线通过不等于专业工作已被接受；重复调用同一家族模型不形成独立专业判断；Trace 可读不等于失败原因已被识别。
 - 检验：固定同一候选版本，对比单次总分与重复、按风险分层的指标及盲审；注入环境残留、评分顺序变化和 grader 漂移，检查结论是否变化；用未参与调优的任务和后续成果复核判断。
 - 讨论与裁决 / 正文处置：与 PI-16 合并支持 Practice §6.5 的评测理念；沿用 Canonical §8.5、10、11.2 的分层与归因边界。
 - 状态：practice-adopted（工程方法；SE 的增量效度未完成验证）。
-

PI-18 · Existing foundations and the scope of the contribution

- 日期 / 来源类别：2026-09-05；概念定位与独立审稿裁决。核对原作者 Event Sourcing 说明、W3C PROV-DM 与 OMG CMMN 1.1 的正式来源。^{24 25 26}
- 观察与承接关系：Event Sourcing 用事件序列保存状态变化并支持重建，对应本文的 Ledger / reducer 参考实现；PROV-DM 提供 entity、activity、agent 与 derivation 等来源表达，对应 provenance 的既有建模基础；CMMN 提供 case、case file 与 case plan 的建模标准，对应工作对象和非固定流程的既有基础。
- 最小支持命题：这些原语已有明确先例。本文提出的是面向 Agent 的实践编订、候选提交、状态投影与评测修订的组合框架，不是状态、案件管理或来源建模的首次发明。
- 不支持的外推：术语相近不表示模型完全等价；PROV 的 derivation 不自动等于 Evidence Contract 的语义支持；CMMN 不自动包含本文全部模型权限与 Context 语义；事件溯源不是治理成立的唯

一实现。这组定位不构成穷尽相关工作的综述。

- 检验 / 反例：以既有案件或流程系统加来源记录与模型接口为基线；只有在相同工作要求下出现可归于编订、提交或连续性边界的增量，才主张组合收益。不能以“基线不叫 SE”排除已经实现同等机制的系统。
 - 讨论与裁决 / 正文处置：Canonical §2 交代贡献与承接关系；§4 把事件溯源限定为参考实现，保留既有 SoR 路径。
 - 状态：canonical-adopted（定位与边界澄清；组合收益待验证）。
-

PI-19 · Replaceable execution, durable control and contained failure

- 日期 / 来源类别：2026-09-06 访问；InfoQ 对 TiDB、腾讯研究院、Floatboat 与 Trae 从业者的访谈报道。²⁷
- 观察：TiDB 受访者将内部 Harness 概括为“薄 Agent Loop，厚 Control Plane”，报告从 OpenCode 更换到 Pi 时保留 Sandbox、权限、持久状态与恢复设施；同一报道还描述 declarative goal、低通信拓扑、通过明确状态与结果协作，以及 fail fast → limit propagation → recover from trusted state。项目周期、workspace 数量与无人工编码 / 审阅均为受访者自报，未见本文独立复核。
- 最小支持命题：通用执行 loop 与持久状态 / 权限 / 副作用 / 恢复边界可以分层；模型吸收通用执行步骤时，Work Contract 可以减少不承载专业语义的 imperative orchestration；长程可靠性需要衡量错误传播与 trusted recovery，而不只衡量连续步数；可分解任务可以优先通过 bounded state output 协作。
- 不支持的外推：Pi 或任何 Agent core 是通用最优；所有 Skill 或 orchestration 都会消失；所有工作应使用数据库、filesystem、MVCC、checkpoint 或 Multi-agent；少通信必然提高结果；受访系统已经实现完整 Work Contract、Matter governance 或 accepted-work-product gate；自报规模与周期具有独立因果效度。
- 检验：固定模型、任务、资源与 Work Contract，替换等价 Agent loop 并比较状态语义与成果；比较 declarative contract 与固定 planner / worker / reviewer 路径；在不同阶段注入错误，测 durable-state contamination、detection latency、rollback loss 与 recovery success；比较 governed branch / artifact 汇合与高频 messaging 的冲突、重复、成本与接受结果。
- 讨论与裁决 / 正文处置：Canonical §2.1、3、4.5、5.4–5.5 与 8.1 只吸收 declarative contract、Matter / Session / Run、Context compilation、failure containment 和可替换执行的最小关系，不采用产品实现为理论来源；Practice §2、3、4、5、7、8 增加实现、UI、复用、测试与失败边界。未新增 ontology、Contract 类型或原则编号。

- 状态：practice-adopted；canonical-adopted（既有边界的必要澄清，实证效果未验证）。
-

PI-20 · Runtime compatibility and action-channel guarantees

- 日期 / 来源类别：2026-09-07；用户提供的架构讨论与本轮文本对照，属于设计推论，无新增实证。
 - 来源：Harness架构对比，本轮读取完整四轮讨论。聊天中的产品说明与引用未独立核验，不作为供应方实现事实。
 - 可观察机制：讨论提出 provider-managed / 自封装薄 Runtime 两条路径，终端围绕 Review / revision 组织，并比较 SaaS API 与 Computer Use 的适配成本；这是方案描述，不是运行观察。
 - 最小支持命题：由既有分层 Contract 推导，替换执行层与操作通道必须保留工作语义并显式检验能力缺口；执行复杂度应随任务要求选择。
 - 不支持的外推：不证明 Codex 或 Claude 的具体内部机制、CodeRabbit 的定位与优劣、专业任务普遍简单、API 天然幂等或可回滚、Computer Use 必然替代 connector，也不证明 SE 因治理语义而必然拥有市场优势。
 - 复现 / 证伪：V-19 固定 Work Contract 比较原生、模拟和缺失能力的执行层；V-20 注入未知副作用结果并切换操作通道；另以相同成果接受标准比较薄与完整 Runtime 的开发、运维、Review 和恢复成本。均未运行。
 - 讨论与裁决：Canonical §13.2、P21 与 F21 已容纳替换边界，不改正文；Practice §3.1 重写逻辑责任、能力协商、薄 Runtime 及通道选择，并在 §7.3 补充测试。§5.3 已表达面向变化的 Review surface，不重复增加产品类比。拒绝把“停止→注入→恢复”视作无条件等价的 steering，也不把本地 checkpoint 当作外部回滚。
 - 正文处置 / 状态：已吸收为实现约束；9.6 重写 §3.1，并将新增的逐项测试移至 PI-21 / V-19–V-20。性能、互操作性与经济性待验证。不新增 ontology、Contract 或原则。
-

PI-21 · Structured state, authorized views and reusable CS mechanisms

- 日期 / 来源类别：2026-09-07；用户提供的概念讨论与本轮文本对照，属于形式化候选与工程研究线索，无新增实证。
- 来源：稳定 Schema 是否向量空间。已读取三轮讨论，依次涉及状态空间、分层披露和 CS 范式迁移。初次入账未核理论文献；9.6 审阅补核下列原始资料，聊天类比仍不作为定理、实现保证或新颖性证据。

- 可观察机制：讨论尝试以同一 Schema 表达 Matter State、候选变化及不同执行角色的可见视图，并提出数据库、编译器、权限与程序分析机制作为实现候选。这是方案描述，尚无运行观察。
- 最小支持命题：既有 Schema、Context Projection 与 Authority 边界可以进一步转译为可测试的状态约束、读取策略、候选操作和提交检查；工程选型可按这些职责查找成熟机制，不必增加新的 Kernel 对象。
- 数学边界：可暂记 $X_S = \{x \in \prod_i X_i \mid I_S(x)\}$ ，表示满足 Schema 不变量的异质状态集合；分量间约束意味着并非所有字段组合都合法。候选操作 p 在给定版本、授权与前置条件下通过部分转换 $apply_S(x, p)$ 求得候选结果，正式生效另经提交边界。这里不默认存在状态减法、线性加法、标量乘法或可逆操作；操作组合可能有顺序依赖、冲突或无定义。数值字段、文本 embedding 与稳定字段名均不能自动赋予整个工作状态向量空间结构。“坐标系”只作表示类比；流形、切空间与局部线性化需要另行定义结构和证明条件，本次不采纳。
- 披露边界：视图可暂记为 $v_i = \pi(x; assignment, purpose, policy_version, state_version)$ ；可读范围、可提议操作与可提交权限分别验证。授权过滤应在数据跨越相应信任边界前执行，但不因此把 Project 固定为新增流水线阶段或规定它总先于所有检索；可信域内的查询计划和最终 Context 编译可以分阶段实施。摘要、索引、缓存、日志、工具参数与结果也属于需检查的传播路径；隐藏原文不排除从派生值推断敏感信息。权限撤销不能使已经披露的数据自动消失。

候选范式	对应的既有 SE 职责	迁移边界与待检验问题
数据库 view、行列访问控制、查询优化、物化视图	按 Matter / role / purpose 生成 Context Projection 与 Retrieval Index	普通 view 或 projection pushdown 不自动成为安全边界；检查跨对象关联、派生字段、缓存失效与撤权后的重建
编译器 IR / lowering、ISA / ABI 契约	Work Contract 的中间表示与 Host Adapter 兼容边界	不将 Schema 等同于 ISA，也不新增 Work ABI；版本映射、语义保持和能力缺失继续按 PI-20 / V-19 检验
类型、guard 与状态转换检查	Candidate 形状、前置条件与 typed commitment boundary	类型正确不等于 Evidence 充分或具有 Authority；并发版本冲突仍需提交时验证
OS 隔离、object capability、信息流控制	资源访问、最小权限、委派与输出去向	执行层 capability 不等于组织 Authority；RBAC 与 capability 可组合。Context 裁剪不是内存隔离或不泄漏证明，读取与外传权限的组合需单独检查
Abstract interpretation / abstract domain	保留 unknown、冲突、限定和任务所需性质的抽象视图	普通摘要不自动具有 soundness；须定义具体域、抽象域、所保留性质与转换关系后才讨论可靠近似，不能仅凭两个映射声称 Galois connection
CQRS / event sourcing	读投影、候选命令、正式写入与可恢复历史	二者不互相蕴含，也不自动提供 Review / Authority；沿用 PI-18，事件溯源仍只是参考实现

- 不支持的外推：Schema 稳定即变化可自由组合、Context 编译必然确定或最小充分；数据库检索与投影可以任意交换且保持权限和结果；Agent 只能知道本次 Context 中的信息；传统 CS 只治理计算位置而不治理信息与动作；通用理论机制已证明 SE 的安全性、效率、专业正确性或市场价值。
- 复现 / 证伪：V-21 比较源端授权视图与检索后过滤，注入跨 Matter 关联、敏感派生值、缓存与撤权变化，检查各信任边界的暴露及必要证据遗漏；V-22 固定任务与授权范围，对比全文、自由摘要与按声明性质生成的视图，保留反例、unknown 和冲突，测错误确信、成果接受与披露成本。跨版本映射和 Runtime 替换复用 V-19，不新建重复队列。均未运行；本轮来源核验只覆盖下表，具体安全模型、实现选型和证明仍待执行。
- 讨论与裁决 / 正文处置：初次裁决为 indexed；9.6 强审后，主要概念仍由既有 Kernel 承载，但发现候选可见性被误写成必须先取得正式效力，以及 Schema 被缩窄为 persistence policy 的表述问题。Canonical §2.3、§5.4–5.5、§6.1 重写既有表示、披露与提交关系；Practice §2.5、§3.1、§4.3 将其编排为实现叙述。数学公式、CS 对应、反例和证明要求保留本条，不增加 Project 阶段、epistemic sandbox 或 Work ABI 等对象，不据此采纳技术栈。
- 状态：practice-adopted；Canonical 为既有原则的一致性修订，不新增 Kernel 命题。

9.6 原始来源核验（2026-09-07）：

来源	本轮核验到的内容	不据此推导
PostgreSQL 18 Row Security Policies 与 CREATE VIEW ²⁸	行级策略可分别限制读取与修改；view 的安全语义取决于执行身份与 security 配置，部分访问路径有例外	普通 view、查询优化或简单字段投影自动防泄漏；任何数据库配置直接满足 SE 的组织 Authority
LLVM Language Reference, Introduction / Well-Formedness ²⁹	同一 IR 有多种表示；可解析与满足内部结构约束有区别	Work Schema 具有 LLVM 的精确执行语义，或宿主替换无需证明兼容
Cousot & Cousot 1977，作者保存的论文摘要及书目信息 ³⁰	抽象解释通过有序结构、转换和不动点讨论程序性质及抽象间一致性	普通摘要具备可靠近似；本轮已复核完整论文证明或为 SE 建立 Galois connection
Sabelfeld & Myers 2003，§I、§V.D ³¹	读取控制不直接约束读取后的传播；允许的信息释放需由相应策略定义	标签检查能完全追踪 LLM 的语义依赖，或当前 SE 已证明 noninterference

保留于 Index 的实现与审查条件：

- PI-20 的 Runtime 兼容测试继续覆盖控制时点、能力缺失、取消后完成和未知副作用；恢复 Matter State 不等于迁移内部推理快照，API 不天然提供幂等或回滚，薄 Runtime 不自动满足主权部署的数据边界。将这些条件从 Practice §3.1 的逐项警戒句和 §7.3 的两项新增测试收回本条与 V-19 / V-20；正文保留能力协商、效果核对与必要授权的机制。

- 候选可保存、检索和进入 Review，但身份不得被摘要或重复召回升级为正式状态；对照“候选进入 Context”与“候选取得 Authority”分别断言。视图的一致性按相同对象、版本和效力检查，不要求不同时间的快照逐字相同。
 - 检查对象存在性、关联、派生值、工具参数、日志、缓存和接收端；撤权阻止后续复用，不能收回已披露信息。允许的脱敏或释放需要具备 Authority 的规则或决定，不能由模型自行降低限制。
 - 视图内容不足时应标明缺口或请求授权补充；对未能识别的遗漏另用 V-22 外部参照评测，不把“按 Schema 编译”当作充分性证明。保留先验知识、推断与隐蔽通道等模型边界，不声称限制了模型全部可知信息。
 - 保留 unknown、限定、冲突和来源的目标属于任务性质保持；如果要上升为形式可靠性，必须另行定义状态域、转换、抽象关系与威胁模型。没有该证明仍可实现并测量视图治理，不借数学术语取得保证。
-

PI-22 · Personal Attention 与既有连续性原语

- 日期 / 来源类别：2026-09-09；用户提供的设计讨论。完整读取13个turn与1张截图，属于设计输入；外部项目机制与Courtwork实际接缝由Courtwork研究索引登记，不在论文重复选型。
- 来源：解释 GoRaven。后续用户明确Attention可拥有独立于单个Matter和Session的生命周期；“全量memory”指可寻址、按需要披露。完整私有原文留在个人项目，外部技术来源及本轮消费见 Courtwork固定来源索引与局部核验。固定提交已在本地形成，未推送前远端链接可能暂不可用。
- 观察与最小支持命题：这是对个人持续关注对象的设计要求，而非运行结果。可检验一个关注关系在关联零个、一个或多个Matter时，能否保留identity、owner、未完义务与生命周期；人的视图与执行Session是否仍可替换且不成为第二事实源。
- **Astra裁决 / 正文处置**：本轮不修订Canonical或Practice。Canonical摘要已明确 anything governed is addressable、持久状态与Context分离；PI-21已容纳存在性、关联、缓存与撤权的披露边界；PI-20 / V-19已容纳Runtime能力协商。typed lookup与grep作为实现选择进入Courtwork局部选型，不升为新的普遍检索定律。独立Attention的产品对象语义不自动要求新增Kernel ontology；先比较既有对象关系是否足够，再决定是否需要最小正文修订。若未来需修正文，由Astra在SE源目录亲自撰写。
- 不支持的外推：不证明Attention必须成为新的Kernel实体、任何Runtime可无损互换、grep提供访问控制，或文件化手动loop已经证明自动ACL、效率及专业质量；来信中的产品主张不证明成熟度。个人通信原文、Courtwork工单和选型台账不复制到论文。
- 复现 / 证伪：V-23比较独立Attention记录与既有Topic/Queue/Matter关系及State/Event原语，覆盖零/单/多Matter、Session替换、关闭/重开及未完义务。若既有对象可保持同等边界，则否决新

增ontology。存在性与内容披露复用V-21，Runtime缺失能力与替换复用V-19；本轮均未运行这些对照实验。

- 状态：indexed；设计线索已登记，正文不改，无新Edition或发布。



四、验证队列

ID	待验证命题	最小对照	主要结果	当前状态
V-01	宿主适配可以隔离 Runtime 与 Work semantics	固定 Work Contract，替换等价宿主接口	语义一致性、适配成本、E2E	未完成
V-02	External Completion 改善 长程任务	有无独立 completion inventory / gate	completion、accepted outcome、负面结果	供应方观察，待独立复现
V-03	Governed feedback 可以跨 Run 复利	仅保存 comment vs 经 scope / Authority / version 编订	接受率、reversal、错误放大	未完成
V-04	Govern layer 相比 files + search 有增量	固定模型、任务和材料	状态一致、critical omission、token、Review	未完成
V-05	Sparse activation 解耦总存量与单次 Attention	稠密激活 vs preset / Expert / primitive	Context / Tool surface、omission、pollution、permission	未完成
V-06	Context Mutation 可管理 且可恢复	各类 mutation vs 未修改 Context	accepted outcome、source preservation、recovery	研究方向性证据，待工作场景验证
V-07	Multi-agent review 不会把 相关错误误当独立证据	注入共享前提、来源缺失和 Evaluator 偏差	false consensus、escalation、Authority routing	未完成
V-08	Document surface 改善 Review 而不污染正式状态	文档表面 vs 非结构输出	Review time、修正率、promotion errors	社区观察，待消融
V-09	Current Semantic State 可作为长程工作的默认 execution substrate	State-only vs State + on-demand History vs State + append-only Transcript	accepted outcome、token、latency、recovery、omission、source disclosure	procedural benchmark 局部实证，待专业工作复现
V-10	Human Review packet 在 有限 attention 下仍支持独立判断	raw trace / per-action approval vs structured decision unit	error detection、evidence-seeking、time、override、reversal、canary	position paper 支持 问题定义，待产品实验
V-11	Artifact / Evidence dual state 改善跨轮 work continuity	仅 Artifact vs Artifact + governed Evidence	regression、repeat work、unsupported completion、recovery	software benchmark 局部实证，待跨域验证

ID	待验证命题	最小对照	主要结果	当前状态
V-12	Governed Evaluator lifecycle 防止 rubric 与 attribution drift	static rubric vs versioned monitoring / review / rollback	wrong-reason contamination、drift detection、rollback、outcome	单一生产案例，跨域与因果待验证
V-13	Repository-governed handoff 支持自主续行	repository state vs chat summary vs no governance files	duplication、source mismatch、revision scope、validation、Review	单次自观察，待独立对照
V-14	按工作义务和状态边界记录信号改善失败定位	自由摘要 vs 原子笔记 vs Contract 证据记录；固定来源后另做端到端对照	独立参照下的遗漏、归因准确度、相关性、支持、Review 成本、接受与 reversal	方法已定义，实验未运行
V-15	离线 signal 的改善预测真实工作改善	调优集 vs 按来源 / Matter 隔离的留出集及后续成果	指标稳定性、排序偏差、风险分层、成果接受、成本、指标与结果背离	方法已定义，实验未运行
V-16	可替换 Agent loop 不改变 Work semantics	固定 Contract / Matter，替换语义等价 Runtime core	state transition、permission、recovery、accepted outcome、adapter cost	访谈自报支持可行性，待独立复现
V-17	Failure containment 比持续步数更能解释长程可靠性	分阶段注入相同错误；有无 commit isolation / trusted checkpoint	detection latency、propagation depth、state contamination、rollback loss、recovery	从业者观察与架构推论，实验未运行
V-18	Governed state 协作优于高频 Agent chatter 的条件	bounded branches / artifacts vs messaging topology	conflict、duplicate work、Context cost、error propagation、accepted outcome	从业者观察，适用边界待验证
V-19	Runtime 能力协商保留 Work Contract	原生 / 经验证替代 / 缺失能力；固定任务、权限和接受标准	非法提交、信息丢失、可恢复性、显式拒绝与总成本	设计推论，待验证
V-20	操作通道切换不突破效果边界	API / Browser / Computer Use；注入超时、迟到结果、重复回执	重复副作用、授权绕过、结果核对与错误成功报告	设计推论，待验证
V-21	授权视图约束端到端披露	源端授权视图 vs 检索后过滤；跨 Matter、关联推断、缓存与撤权注入	边界暴露、外传、陈旧权限、必要证据遗漏	已核局部原始来源，实验未运行
V-22	任务抽象保留所声明的必要性质	全文 vs 自由摘要 vs 按声明性质生成的视图；固定授权范围	unknown / 冲突保留、错误确信、接受率与披露成本	设计候选，不声称形式可靠性
V-23	个人Attention是否需要独立对象语义	独立记录 vs 既有Topic/Queue/Matter 关系与State/Event；零/单/多Matter及Session替换	identity、owner、未完义务、生命周期、恢复与重复事实源	设计候选，实验未运行

V-14 / V-15 · 离线评测快照与查考方法

以下是待执行的评测设计，不是本次实验结果。它使用已有的 Contract、Evidence、Artifact 与 Review 对象，不增设运行时 ontology。

环节	应保存的快照或记录	解释边界与校验
定义比较	任务与义务版本、来源快照、初始状态、模型、Harness、Context policy、预算、工具环境和评测器版本	模型比较与完整系统比较分开命名；环境不能冻结时记录变化，不声称严格可回放
构造信号	测量对象、单位、分母、去重与相关性规则、原文坐标、grader 判据	数量、支持、覆盖与可读性分开报告；重复或无关事实不能靠累积数量获得工作价值；不将局部信号相乘为未经定义的总分
留出与校准	调优样本、独立留出 Matter / 来源、边界样本、人工分歧与评分器输出	同源改写不算独立留出；盲化模型标签、交换输出顺序、重复评分；共享模型家族的结果不当作独立验证
定位遗漏	来源 → 抽取 → 持久状态 → Projection → Artifact 的对象对应与版本	留存率只针对明示且独立核对的应保留集合；没有抽取到的内容不能从分母消失；允许合理归并、舍弃与未解冲突；无法对应时标为不可测
归因与重测	原配置、候选修改、配对任务、重复 trial、失败样本、成本与延迟	固定来源检查表示，再恢复端到端检索；共同变更只能支持组合效果；区分改进 signal 与改善成果；样本量与不确定性不足时不下稳定排序结论
接受与后续	未参与调优的 Reviewer 判断、声明观察窗口、下游采用、返工、reversal 和风险分层	当前 Contract 未覆盖但影响工作的问题必须能进入评测；保留全部尝试与退出，避免只统计成功件；离线结果与真实结果背离时重开指标或判据

评测快照可以作为版本发布的 Evidence，但评分器没有部署 Authority。已用于反复调优的留出集应降为开发或回归资料；后续候选使用新的留出材料。这里的校验是研究队列，不能写作已经验证的 SE 性能。

五、增量修订记录

2026-09-09 · Attention · Index-only

Astra复读现有正文与PI-20/PI-21后裁决：只登记PI-22及对象生命周期对照V-23，披露与Runtime验证复用V-21/V-19；不修订Canonical、Practice、Edition、CHANGELOG或发布产物。外部成熟实践的局部核验与产品消费留在Courtwork，不把本轮设计讨论提升为实现证据。

2026-09-07 · 9.6

审查对象	裁决与有机合成	正文处置
稳定 Schema / 状态空间	保留跨执行可比较的对象、状态、版本与转换语义；不用向量、流形或 ISA 替代工作定义	Canonical §2.3 首段；Practice §2.5
按角色披露与工作集编译	从授权范围、当前义务到不同接收视图，保留判断依赖的来源、限定和缺口	重写 Canonical §5.4、Practice §4.3
候选可见性与正式效力混同	Candidate 可被保存、检索并供后续 Review；正式更新单独过提交边界	Canonical §5.5
读取、操作与信息传播	读取权、工具权和向外传递的权限不能相互代替	Canonical §6.1；Practice §4.3
Runtime / 通道适配段落累积保护性条件	按语义责任、能力协商、执行复杂度和操作通道重新合成	压缩 Practice §3.1；详细校验归 PI-21 / V-19–V-22

本轮由 Astra 主笔，Luna 只读探索，另由未继承聊天历史的 Astra 实例进行独立文本审查。初审与终稿复审结果见下表。属于模型审稿与原始来源核验，不是安全证明、互操作实测或同行评审接受。没有新增 ontology、Contract 类型或原则编号。

审稿与修后复审：Luna（gpt-5.6-luna，max）只读定位既有承重章节与重复，建议不新增 Kernel 对象；主笔据此选择重写而非加入术语合集。独立 Astra（gpt-6-astra，xhigh）在未继承聊天的情况下完整通读三份原稿，并复核磁盘修订稿；最终未发现阻断本轮修订接受的承重矛盾。主笔保留最终裁决与文字责任。

审稿发现	最终处置与反例
候选尚未提交便不能进入下一轮 Context	§5.5 允许有明确身份的候选被保存、检索和审阅；候选可见不等于取得正式效力
“未提交错误仍是局部失败”与共享候选冲突	整段改写为传播时保留认识状态、提交限制正式效力及污染后的修复；Committed working assumption 仍非已证实事实
三视图只从正式状态生成，排除了候选与原始材料	同步 Practice §2.9、§5.1 图示与说明、Projection Consistency 测试和 Canonical 开放问题 33；一致性比较相同版本和效力，不要求披露相同
本地提交与外部执行被写在同一步	删除原 Practice §4.3 procedural 图，以授权和前置检查、结果证据与待核对状态表达边界
宿主历史不透明仅被视为审计限制	Practice §3.1 的兼容绑定覆盖宿主实际召回的信息及可产生的效果；不能只约束新传入的 Context
§11.5 声称相乘公式预设独立、单调且同号	删除错误数学断言； $x \cdot x$ 即可反驳独立性推论，负因子反驳同号预设。组合模型仍需定义与消融，不在正文扩写代数解释
F21 将接受率的任何变化视为兼容失败	收紧为所需能力满足时仍破坏工作语义或无法维持声明的接受条件；质量改善和随机轨迹变化本身不触发证伪

“未审阅 output”可能被读成人工 Review 普遍必需

Practice §7.3 统一为“未通过适用提交检查的 output”，服从既有混合裁决边界

本地候选检查：构建与 10 项发布检查通过，67 个文内链接均有目标，当前入口与带日期 HTML 一致，`git diff --check` 通过。浏览器检查三视图切换与入口排版，并抽查重写后的 Practice §4.3；不声称逐屏检查全文。Canonical 与 Practice 正文合计较本轮起点缩短约 1,600 字符。原始来源核验仅覆盖 PI-21 所列范围；V-19–V-22 实验未运行，未推送或部署 Pages。

同日 9.5 HTML 保存为 `papers/dist/schema-engineering-2026-09-07-v9.5.html`；当前带日期文件在本轮更新为 9.6。此前 Index-only 条目随本次构建进入 HTML。未推送或部署。

2026-09-07 · Index-only 补记

登记 PI-21 与 V-21 / V-22：保留受约束状态空间、授权视图及 CS 机制映射；拒绝把向量空间、流形、隔离或可靠抽象类比提升为已证命题。Canonical / Practice 与版本元数据不变；按索引更新规则不重新构建发布文件，本条尚未进入已生成的 9.5 HTML。检查条目编号、聊天来源、章节指向与 diff；实验及外部理论来源核验待执行。

2026-09-07 · 9.5

观察 / 讨论	裁决	Canonical	Practice	Index
可替换 Runtime、薄主权部署、面向 Review 的终端与 API / Computer Use 适配	采用能力与效果边界；产品事实和市场推断不提升为证据	正文不变，仅同步 Edition / Revision	重写 §3.1；§7.3 增加两项验证要求；§5.3 已充分	PI-20；V-19 / V-20，均待实测

本轮为架构与文字审阅，不是运行时互操作实验或独立模型审稿。保留已有未提交修订与历史候选产物；本地生成不等于 Pages 发布。

本地候选检查：三份源文件 Edition / base 同步为 2026-09-07；构建与 10 项发布检查通过，带日期 HTML 与当前入口一致，`git diff --check` 通过。本轮未做浏览器视觉复核，未推送或部署。

2026-09-06 · 9.4

观察 / 讨论	裁决	Canonical	Practice	Index
TiDB 等从业者访谈把易折旧 Agent loop 与状态、权限、副作用、验证和恢复控制面分层	作为既有 SE layering 的工程佐证；不把具体 Harness、数据库类比或自报结果提升为理论权威	\$2.1、8.1 合并重复 Runtime 说明，收紧为 declarative Contract 与 replaceable execution	\$2.2、3.1 增加折旧分层与成熟组件复用	PI-19；V-16
长程失败来自局部错误越过验证成为持久共享状态	吸收为 proposal / commitment 的 failure-containment 解释，不新增状态对象	\$5.5 增加最小段落；\$3、4.5 明确 Matter / Session / Run 与 executor replacement	\$4.2、7.3、8.2 增加 trusted recovery 测试与失败模式	PI-19；V-17
Agent 间通信拓扑会增加协调状态；安静协作可以经 bounded artifacts 汇合	只采用 state-mediated coordination 的默认倾向，保留任务相关通信与并发冲突边界	现有 Operator / Lane / Candidate 关系已充分，不修订	\$2.7、7.3、8.2 增加 branch / artifact 协作和对照	PI-19；V-18
Human review 与 GUI 应暴露状态、证据、权限、提交与恢复，而不是 Agent 表演	作为既有 Human Work Surface 的产品推导；UI 形态不进入 Canonical ontology	不修订	\$5.3、8.2 明确 transition surface 与 Agent theater 失败	PI-19 记录来源边界；具体组件不作为证据命题

本轮没有新增 Canonical ontology、Contract 类型或原则编号。正文净增量通过合并 Canonical §8.1 的重复 Runtime / composability 说明控制；外部项目、受访者判断和易折旧实现细节只留 Index。

本地候选检查：三份源文件 Edition / base 已同步到 2026-09-06；构建与 10 项现有发布检查通过，当前入口与带日期 HTML 内容一致，git diff --check 通过；桌面浏览器完成当前 Canonical 入口的渲染检查。未推送、未部署，Pages workflow 尚未执行。

2026-09-05 · 9.3

观察 / 讨论	裁决	Canonical	Practice	Index
Manus 实践分享用局部信号追查笔记行为；官方 Agent eval 方法区分执行设施和评测设施	抽取 Runtime 外的评测—归因—修订闭环；不将案例自报提升为因果	\$11.2 澄清遵守判据与判据充分，明确归因坐标的边界	\$6.5 写入理念，原训练候选移 \$6.6	PI-16/17；V-14/15；具体校验只留 Index
Fresh Astra 独立通读指出提交语义被写成事件溯源的唯一实现	保留 Candidate / Committed 与唯一权威状态；事件重放限定为参考实现	\$2、4、5 明确语义要求与持久化选择	\$4.1 明确既有 SoR 的承载方式	记录反例：事务状态表、版本化成果与可审计提交记录可实现同一边界
独立审稿发现示例把可解析引用直接晋升为 supported	证据定位与语义支持分别成立	\$6.2、8.8 补清语义检查责任	沿用既有 Evidence / Review 边界	反例：引用确实存在，但只限定或反驳目标主张
训练的默认检查顺序被写成普遍必要前后关系	不以训练替代判据和接受；允许训练参与首次能力形成或降低成本	\$9、11.4、P15 及对 应检查统一范围	\$6.6 保留产品不依赖训练飞轮的独立价值	反例：基础模型不满足任务门槛，先训练再做首次 E2E；质量相同但训练显著降低成本

观察 / 讨论	裁决	Canonical	Practice	Index
独立审稿要求交代与既有工作建模、来源与事件溯源方法的关系	贡献限定为 Agent 工作生命周期中的组合与边界	\$2 增加最小定位	不扩写	PI-18 登记 primary sources、承接与非等价关系

本版以可公开讨论的概念工作论文为目标；文本修订与本地构建不表示完成实证或已经线上发布。独立审稿不继承聊天历史，仍属于模型审稿，不替代作者或专业共同体接受。

审稿与修后复审：2026-09-05，使用不继承父任务聊天的 gpt-6-astra 实例，以 xhigh reasoning 完整通读三份文本，再核对修订 diff 与受影响章节。首轮提出五项必要边界修复，上表均已处置；复审未发现阻断公开概念 / 立场工作论文发表的承重矛盾。其判断不覆盖全部外部来源的独立复核，不等于实证论文或正式同行评审接受。篇幅收敛、系统实现、跨域结果与训练增益继续作为后续工作，不扩大本轮正文。

本地候选检查：三份源文件 Edition / base 已同步到 2026-09-05；构建与 10 项现有发布检查通过，当前入口与带日期 HTML 内容一致，57 个文内链接均有目标，git diff --check 通过。浏览器安全策略拒绝本地文件预览，未完成视觉复核；不以结构校验代替视觉校验。未推送、未部署，Pages workflow 尚未执行。

2026-09-04 · 9.2

观察 / 讨论	裁决	Canonical	Practice	Index
Codex 实现讨论显示 fresh Context + on-demand history 的候选方向	公开资料不足以确认具体接口；现有 Context / State 边界已经充分	不修订	不修订	PI-11 作为产品趋势观察，保留严格不支持项
人在 loop 中仍可能因 attention overload 失去有效监督	Review Contract 必须包含认知充分性，不只包含审批拓扑	\$6.6 最小增量；无新 Contract / 原则	\$5、7 增加三投影、Review packet 与测试	PI-12 保留 position paper、边界与待验证指标
同一 governed state 同时服务模型、人和未来 Run	Context Projection、Human Work Surface 与 Retrieval Index 是不同 Projection；Current Semantic State 保持唯一事实源	\$5.4 明确三类 Projection 与 split-brain 边界	\$2.5、4.3、5.1、7.3 增加实现与测试	作为 PI-10–PI-12 的综合裁决，不虚构独立外部来源
多日 autonomous software development 同时保存 Artifact 与 Evidence，并隔离角色权限和 Acceptance	software 跨 task boundary 后呈现 work-shaped runtime；软件 substrate 的可验证性限制外推	\$8.5 增加局部边界	\$4.3 增加 dual continuity	PI-13 保留具体实现、实验范围和限制

观察 / 讨论	裁决	Canonical	Practice	Index
生产 Evaluator 的 criterion、reason 与 rubric 会变化并产生下游后果	Evaluator 是 governed artifact；正确 label 不能掩盖错误 attribution	\$11.2 增加 lifecycle 与 commitment boundary	\$7.3 增加测试	PI-14 保留生产案例与外部效度限制
本轮由 repository state、revision protocol 与渐进披露续行	只构成单次方法自观察，不构成因果证据	不修订	不修订	PI-15 / V-13 登记对照设计
正文修订仍可从新增段落和版本摘要辨认材料加入顺序	接受后的命题必须重写进最小完整章节；逐项记录与校验只留 Index	重写 \$5.4、6.6、8.5、11.2，删除补丁式解释	重写 \$2.5、5.3，删除版本摘要	修订合成纪律写入 CONTRIBUTING 与 papers/README
同一 SE 对象曾出现无意别名和机械翻译；反过来，固定词形会把自然多义误作治理问题	用词按语境自然达意并保持全篇体例；治理同一指代与概念边界，不治理单词本身；SE 抽象优先对接共识，自撰概念保持最少	统一没有语义差别的定义、状态转换和 Projection 名称；不要求单词一义，不改变 ontology 或原则	统一同一对象的实现与测试表述，保留不同语境所需的自然用词	写作范例并入既有治理文件；本行保存裁决与 repo-wide 校验范围

本轮没有新增 Canonical ontology、Contract 类型或原则编号。

2026-09-01 · 9.1

观察 / 讨论	裁决	Canonical	Practice	Index
主动 Context 编辑可以被表达为 action 并给予局部 credit	这是 Projection 操作纪律，不是新 Memory ontology	\$5.4 增加最小边界；无新原则	增加 Context Mutation 实现与 Preservation 测试	PI-07 保留论文、代码和外部推边界
多个 Agent 可以共享 Harness 或隔离 Context	这描述 execution topology，不定义 Expert 或 Authority	\$4.3–4.4 已充分，不修订	\$2.7 明确 topology / Expert；增加 correlated failure 测试	PI-08 保留个人实践与限制
文档驱动工作可以同时服务执行、审阅、恢复和交付	吸收“文档是共享工作表面”，保留 Trace / Index / State / Artifact 分离	已有对象边界，不新增术语	\$2.8、5.4 与 Documentation Promotion 测试	PI-09 保留社区来源和自报限制
旧 Practice 以宿主、产品和个例组织大量论证	正文改为泛化、最小、连贯的实践快照；实例与来源移入 Index	移除当前证据图谱，保留稳定证据纪律	整体重写，保留架构、对象、测试与失败边界	PI-01–PI-09 承接来源、检验与处置
Structured State 在长程 procedural task 中比累积 Transcript 更准确且更省 token	吸收带 sufficient-state 前提的 state-first execution；History 仍属于 Evidence / Ledger，discard from prompt 不等于 delete from storage	\$5 增加一句规范性收敛；无新 ontology 或原则	\$2.5、4.3、7.3 增加操作区分和测试	PI-10 保留论文、数字、限制与待复现边界

本轮没有新增 Canonical ontology、Contract 类型或原则编号。

六、来源登记

1. DeepSeek, “DeepSeek Harness developer preview: Everything is a plugin,” <https://deepseek.com/harness/en/> (快照日期: 2026-08-28)。仅用于可观察的插件化能力面。↩
2. DeepSeek AI, “deepseek-ai/deepseek-harness,” <https://github.com/deepseek-ai/deepseek-harness> (developer preview)。官方仓库明示 compatibility-breaking changes，因而只支持带版本的宿主适配实验。↩
3. DeepSeek Harness Documentation, “DeepSeek Harness Architecture,” <https://deepseek-harness.github.io/deepseek-harness/en/reference/>。用于 session events、agent events、capability events、turn flow 与 session log 的公开定义。↩
4. DeepSeek Harness Documentation, “User Interaction,” <https://deepseek-harness.github.io/deepseek-harness/en/reference/subsystems/user-questions>。用于 provider-neutral question vocabulary 与 presentation intent。↩
5. DeepSeek Harness Documentation, “Cookbook: extension plugin shapes,” <https://deepseek-harness.github.io/deepseek-harness/en/reference/cookbook/extension-cookbook>。用于 UI extension surface。↩
6. Cordiverse, “A Programming Paradigm for Spatiotemporal Composability,” <https://github.com/cordiverse/paper>。用于 reversible effects、reactive coeffects、effect tracking、resolution、reconciliation 与 hot replacement 的形式化参照；不证明 Work semantics。↩
7. Factory Research, Theo Luan, “What it Takes for Coding Agents to Complete Large Software Tasks,” <https://factory.ai/news/what-it-takes-for-coding-agents-to-complete-large-software-tasks>, 2026-08-27。来自所选 ProgramBench 任务、供应方自建 system / benchmark；仅用于 Completion governance 的局部机制。↩
8. Michael Segner, Anthropic / Claude, “How Warp builds self-improving agents on Claude,” <https://claude.com/blog/how-warp-builds-self-improving-agents-on-claude>, 2026-08-26。仅用于 feedback capture、proposed diff、Human review 与 versioned reuse；不把 Skill 等同于 Work Contract。↩
9. Anthropic Help Center, “Use Claude’s chat search and memory to build on previous context,” <https://support.anthropic.com/en/articles/11817273-using-claude-s-chat-search-and-memory-to-build-on-previous-context> ; “How can I create and manage projects?,”

- <https://support.anthropic.com/en/articles/9519177-how-can-i-create-and-manage-projects> ; OpenAI Help Center, “Projects in ChatGPT,” <https://help.openai.com/en/articles/10169521-projects-in-chatgpt> 。快照日期：2026-08-29。仅用于 project-bounded recall 与 archive retrieval。↩
10. OpenAI, “Memory FAQ,” <https://help.openai.com/en/articles/8590148-memory-faq> ; “ChatGPT Release Notes,” <https://help.openai.com/en/articles/6825453-chatgpt-release-notes> ; “Dreaming: Better memory for a more helpful ChatGPT,” <https://openai.com/index/chatgpt-memory-dreaming/> 。快照日期：2026-08-29。仅用于 managed-resource 与 scoped-context 的产品表面，不反推 backend ontology。↩
11. Andrew Ng / DeepLearning.AI, “The AI Engineering Skills Map In Detail — Software Engineering Fundamentals,” <https://www.deeplearning.ai/the-batch/the-ai-engineering-skills-map-in-detail-software-engineering-fundamentals> , 2026-08-28。仅用于 execution-to-judgment migration 的结构假设，不作为 Schema Engineering 的外部背书。↩
12. Zhuoshi Pan et al., “ContextPilot: Teaching Agents for Proactive Context Management via Fine-grained RL,” arXiv:2608.28476, 2026, <https://arxiv.org/abs/2608.28476> 。仅用于 context-editing action、snapshot 与局部 credit 的机制；不视为 Matter governance 或跨领域专业能力的证明。↩
13. Tencent, “ContextPilot,” <https://github.com/Tencent/ContextPilot> 。截至 2026-09-01 公开 inference、evaluation 与 training 实现；公开时间短，尚不足以建立独立复现或外部效度。↩
14. Scott Fryxell, “The Harness Is the Thing,” 2026, <https://scott-fryxell.github.io/blog/the-harness-is-the-thing/> 。个人实践观察；仅用于共享 Harness 与执行位置隔离，不推断一般收益。↩
15. David Ondrej, “Agentic Engineering Setup (after 2,000+ hours),” 2026 Q3, 用户保存的公开帖子快照；原作者主页 <https://x.com/DavidOndrej1> 。个人经验与趋势判断，无统一 benchmark 或独立复现。↩
16. Vonng, “如何验收 AI 拉出来的屎山？”，2026，用户保存的公开文章快照；发布说明 <https://x.com/RonVonng/status/2094288759743545769> 。仅吸收文档作为共享工作表面的观察；成本比例与效果判断作为作者自报。↩
17. Sanket Badhe, Priyanka Tiwari, and Jonghyun Chung, “SKILL.state: Scalable Long-Horizon Agent Skills,” arXiv:2608.26263v2, 2026, <https://arxiv.org/abs/2608.26263> 。v1 于 2026-08-26 提交，v2 于 2026-08-28 修订，arXiv 页标注 accepted at EMNLP。作者所属 Google LLC 与 Purdue University。本 Index 仅将其用作 explicit execution state、validated patch、bounded prompt footprint、noise robustness 与 sufficient-statistic limitation 的局部实证，不视为完整 Schema Engineering 的证明。↩
18. OpenAI, “Model guidance,” <https://developers.openai.com/api/docs/guides/latest-model> 。访问日期：2026-09-04。官方材料用于确认 Responses Runtime 中的 persisted reasoning、conversation / state compaction、tool orchestration 与 autonomy guidance；未发现公开 new_context 或截图所述 history / notes 接口，因而不据此确认 Codex 产品实现。↩
19. Margaret Mitchell, Avijit Ghosh, and Samir Passi, “AI Agents Push Humans Out of the Loop,” arXiv:2608.23642, 2026, <https://arxiv.org/abs/2608.23642> 。Position paper；用于 human oversight 的 cognitive requirement、approval fatigue、strategic friction、batch review 与 monitoring 设计约束，不作为特定 Review UI 收益的实证。↩

20. Haoyang Yan et al., “Harness-of-Harness: Multi-Day Autonomous Software Development with Continual Improvement,” arXiv:2609.01481v1, 2026, <https://arxiv.org/abs/2609.01481> 。用于 Artifact / Evidence dual state、bounded increment、role-specific Authority、progressive disclosure、frozen candidate 与 independent QA 的软件域观察；不把其多 Agent topology 或 benchmark 数字外推为一般专业工作结论。↩
21. Emma Yanyang Kong et al., “The Lifecycle of LLM-as-a-Judge for Large-Scale Recommendation Explanations,” arXiv:2608.18300, 2026, <https://arxiv.org/abs/2608.18300> 。用于 criterion / guideline / rubric 的生产 lifecycle、reason-aligned evaluation、bounded revision、drift monitoring、Human review gate 与 rollback；不把单一 recommendation surface 视为通用 Work Eval 的证明。↩
22. 葬爱咸鱼，邀请胡迪 Hoodie 补充整理，《Manus团队测模型一点微小的经验》，2026-09-04，<https://funeralai.substack.com/p/manus> 。访问日期：2026-09-05。用户提供的微信入口为 https://mp.weixin.qq.com/s/2CXssAsQMxdPn_5aLcaijw?scene=1 ，本次无法直接读取；以作者 Substack 全文核验，不声称逐字核对两个版本。证据类别为 practitioner account，不是 Manus 完整评测架构规格或独立实验。↩
23. Anthropic, “Demystifying evals for AI agents,” 2026-01-09, <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents> 。访问日期：2026-09-05。官方工程实践总结，用于离线评测设施、重复 trial、多类 grader、轨迹与 outcome 分离及回归纪律；不视为 Schema Engineering 的效度证明。↩
24. Martin Fowler, “Event Sourcing,” 2005-12-12, <https://martinfowler.com/eaDev/EventSourcing.html> 。访问日期：2026-09-05。作者的架构模式说明；用于事件序列、状态重建与 system-of-record 选择的定位，不证明 SE 组合收益。↩
25. W3C, “PROV-DM: The PROV Data Model,” Recommendation, 2013-04-30, <https://www.w3.org/TR/prov-dm/> 。访问日期：2026-09-05。用于 provenance 对象和关系的既有基础；来源关系本身不判定专业主张为真。↩
26. OMG, “Case Management Model and Notation,” Version 1.1, December 2016, <https://www.omg.org/spec/CMMN/1.1/About-CMMN> ，规范正文 <https://www.omg.org/spec/CMMN/1.1/PDF> ，§4–5。访问日期：2026-09-05。用于 case 工作建模的定位，不把 SE 等同于 CMMN，也不声称穷尽案件管理相关工作。↩
27. Tina, InfoQ, 《人人都能整个“自己的 DeepSeek Harness”，那我们为啥还在给 Claude Code 们充会员？》，2026-09-04, <https://www.infoq.cn/article/6Jc130IN2OaXqsPDlzmJ> ，访问日期：2026-09-06。访谈报道用于 thin loop / control plane、Runtime replacement、declarative orchestration、state-mediated coordination 与 failure containment 的从业者观察；系统规模、开发周期、无人工编码 / PR review 与行业共识均视为报道或受访者自述，不作为独立实验。↩
28. PostgreSQL Global Development Group, PostgreSQL 18 Documentation, “Row Security Policies,” <https://www.postgresql.org/docs/18/ddl-rowsecurity.html> ；“CREATE VIEW,” <https://www.postgresql.org/docs/18/sql-createview.html> 。访问日期：2026-09-07，访问 current 页面时版本为 18；登记固定主版本入口。用于区分访问与修改策略、执行身份及视图安全配置，不构成 SE 安全验证。↩

29. LLVM Project, “LLVM Language Reference Manual,” Introduction / Well-Formedness,
<https://llvm.org/docs/LangRef.html> 。访问日期: 2026-09-07。仅用于中间表示与结构约束的既有机制定位;
不证明 Work Contract 的语义保持。↩
30. Patrick Cousot and Radhia Cousot, “Abstract interpretation: a unified lattice model for static analysis of
programs by construction or approximation of fixpoints,” POPL 1977, pp. 238–252, 作者摘要与书目
<https://www.di.ens.fr/~cousot/COUSOTpapers/POPL77.shtml> 。访问日期: 2026-09-07。本轮核验摘要, 不
声称逐项复核论文证明。↩
31. Andrei Sabelfeld and Andrew C. Myers, “Language-Based Information-Flow Security,” IEEE Journal on
Selected Areas in Communications, 21(1), 2003, §I、§V.D, 作者保存的原文
<https://www.cs.cornell.edu/andru/papers/jsac/sm-jsac03.pdf> 。访问日期: 2026-09-07。用于访问控制与传播
约束的区别及显式信息释放策略, 不把传统程序分析保证直接外推给 LLM。↩